

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра информатики

К защите допустить:

Исполняющий
обязанности заведующего
кафедрой информатики
_____ С.И.Сиротко

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к дипломному проекту
на тему

**ПРОГРАММНЫЙ КОМПЛЕКС ДЛЯ ПОДГОТОВКИ К
ЦЕНТРАЛИЗОВАННОМУ ТЕСТИРОВАНИЮ И
ЦЕНТРАЛИЗОВАННОМУ ЭКЗАМЕНУ**

БГУИР ДП 1-40 04 01 00 032 ПЗ

Студент

Б.Я.Дмитрук

Руководитель

С.В.Снисаренко

Консультанты:

*от кафедры Информатики
по экономической части*

С.В.Снисаренко
О.А.Матяс

Нормоконтролер

А.Д.Комаровская

Рецензент

???

Минск 2026

Решением рабочей комиссии
допущен(а) к защите дипломного проекта

Председатель рабочей комиссии
_____ (_____)
Подпись Инициалы и фамилия

«___» июня 2026 г.

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИНФОРМАТИКИ
И РАДИОЭЛЕКТРОНИКИ

Кафедра информатики

УТВЕРЖДАЮ

И.о. заведующего кафедрой
информатики

С.И. Сиротко
«20» марта 2026 г.

ЗАДАНИЕ
на дипломный проект

Обучающемуся Дмитруку Богдану Ярославовичу
(фамилия, собственное имя, отчество (если таковое имеется))

Курс 4 Учебная группа 253504

Специальность 1-40 04 01 «Информатика и технологии программирования»

Тема дипломного проекта

Программный комплекс для подготовки к централизованному
тестированию и централизованному экзамену

(наименование темы)

Утверждена приказом ректора 23.02.2026 № 409-с

Исходные данные к дипломному проекту

Тип операционной системы: Windows 11 23H2, Android 13.0, Языки программирования:

C# 12, Kotlin 2.3.0, Python 3.13. Перечень выполняемых функций: создание, прохождение и
модерирование тестов, просмотр статистики учащихся, модерирование учебных групп
преподавателем. Перечень таблиц базы данных: пользователи, роли, предметы, темы,
задания, тесты, назначения студентам, попытки прохождения, связи преподаватель-ученик.

Оформление расчетно-пояснительной записки и графической части дипломного проекта
выполнить в соответствии с СТП 01-2024.

Перечень подлежащих разработке вопросов или краткое содержание
расчетно-пояснительной записки

Титульный лист. Реферат. Задание. Содержание

Введение

1 Обзор и анализ существующих решений.

2 Выбор и обоснование технических требований.

3 Проектирование программного комплекса

4 Проектирование и разработка базы данных

5 Программная реализация комплекса

6 Руководство по установке (развертыванию) и использованию программного комплекса

7 Технико-экономическое обоснование

Заключение. Список использованных источников

Приложения (обязательные): отчет о проверке на заимствования в системе «Антиплагиат»;
листинги программного кода; графический материал, поясняющий разработанное
программное средство; ведомость дипломного проекта

Перечень графических материалов (с точным указанием обязательных чертежей и графиков):

Общая архитектура приложения. Схема программы – формат А4, лист 1.

Схема алгоритма – формат А4, лист 1.

Пользовательский интерфейс. Плакат – формат А4, лист 1.

Диаграмма прецедентов. Плакат – формат А4, лист 1.

Инфологическая модель базы данных. Плакат – формат А4, лист 1.

Консультанты по дипломному проекту (с указанием разделов, по которым они консультируют)

Матяс О.А. – раздел 7.

Снисаренко С.В. – разделы 1, 2, 3, 4, 5, 6

Гриценко Н.Ю. – нормоконтроль.

Примерный календарный график выполнения дипломного проекта

Наименование этапов дипломного проекта	Срок выполнения этапа	Примечание
1-я опрoцентoвка (разделы 1, 2, 3)	23.03.2026–24.03.2026	30 %
2-я опрoцентoвка (разделы 4, 5, 6, 7)	16.04.2026–17.04.2026	60 %
3-я опрoцентoвка (разделы «Введение», «Заключение», «Список использованных источников», графический материал)	11.05.2026–12.05.2026	90 %
4-я опрoцентoвка (полностью готовый(ая) проект(работа))	18.05.2026–19.05.2026	100 %
Консультации по оформлению графического материала и пояснительной записки, нормоконтроль	с 16.03.2026	Еженедельно согласно графику
Итоговая проверка готовности дипломного проекта на заседании рабочей комиссии кафедры информатики и допуск к защите в ГЭК	25.05.2026–28.05.2026	100% Согласно графику
Рецензирование дипломного проекта	01.06.2026–12.06.2026	Согласно распоряжению
Защита дипломного проекта	15.06.2026–28.06.2026	Согласно приказу

Дата выдачи задания «20» марта 2026 г.

Срок сдачи законченного дипломного проекта «28» мая 2026 г.

Руководитель дипломного проекта _____ С. В. Снисаренко

Подпись обучающегося _____

Дата «20» марта 2026 г

РЕФЕРАТ

АВТОМАТИЗИРОВАННАЯ ИНФОРМАЦИОННАЯ СИСТЕМА ОБРАБОТКИ ЗАКАЗОВ КОРПОРАТИВНЫХ КЛИЕНТОВ : дипломный проект / Б.Я. Дмитрук. – Минск : БГУИР, 2026, – п.з. – 109 с., чертежей – 5 л. формата А4, плакатов – 3 л. формата А4.

В данном дипломном проекте представлены этапы разработки программного комплекса для подготовки к централизованному тестированию и централизованному экзамену. Рассмотрены требования к системе, архитектурные и технологические решения, а также основные подходы к реализации процессов прохождения тестирования, сбора статистики и формирования персональных рекомендаций.

Цель данного проекта – разработать программный комплекс, объединяющий мобильное приложение для учащихся, веб-инструмент для преподавателей и серверную часть, для организации эффективной подготовки к экзаменам с учетом специфики национальной системы образования. Система обеспечивает доступ к базе тестов прошлых лет, адаптивную генерацию смешанных заданий, автоматический анализ ошибок и контроль успеваемости учеников преподавателем.

Разработанный комплекс обеспечивает автоматизацию процессов проверки знаний и взаимодействия между учеником и учителем, поддерживает масштабируемую клиент-серверную архитектуру, а также позволяет осуществлять персонализированное обучение за счет подсистемы рекомендаций. Использование современных технологий (ASP.NET Core, Kotlin, PostgreSQL) гарантирует отказоустойчивость системы, безопасность персональных данных и высокую производительность при одновременной работе большого числа пользователей.

Данная система ориентирована на массовое использование выпускниками школ, абитуриентами и преподавателями Республики Беларусь и может быть внедрена в образовательный процесс учреждений общего среднего образования для организации пробного тестирования и мониторинга качества знаний.

В рамках работы выполнено технико-экономическое обоснование проекта, в котором рассчитаны инвестиции в разработку программного средства, оценены показатели экономической эффективности его реализации на рынке, а также доказана целесообразность вложений при выводе продукта на массовый рынок образовательных услуг.

СОДЕРЖАНИЕ

Введение.....	7
1 Обзор и анализ существующих решений	9
1.1 Онлайн-платформы подготовки к экзаменам	9
1.2 Мобильные приложения.....	12
1.3 Обоснование разработки программного комплекса.....	15
2 Требования к программному комплексу	17
2.1 Общая постановка задачи.....	17
2.2 Функциональные требования.....	17
2.3 Нефункциональные требования	21
2.4 Требования к интерфейсам	22
2.5 Требования к безопасности	23
3 Проектирование программного комплекса	25
3.1 Архитектурные решения	25
3.2 Модульная структура системы	26
3.3 Взаимодействие компонентов	29
3.4 Сценарии использования системы	33
3.5 Обеспечение расширяемости и сопровождения	48
4 Проектирование и разработка базы данных.....	49
4.1 Обоснование выбора СУБД	49
4.2 Логическая модель данных	49
4.3 Физическая модель данных.....	57
5 Программная реализация.....	66
5.1 Выбор средств и инструментов разработки	66
5.2 Структуре серверной части.....	67
5.3 Структура мобильного приложения.....	69
5.4 Структура веб-инструмента преподавателя	71
5.5 Структура административной панели.....	73
5.6 Поддерживаемые платформы и окружения	73
6 Тестирование и проверка работоспособности программного комплекса....	75
6.1 Цели и методы тестирования	75
6.2 Модульное тестирование серверной части	75
6.3 Функциональное тестирование веб-инструмента преподавателя.....	76

6.4 Нагрузочное тестирование REST API.....	78
6.5 Результаты тестирования	79
7 Руководство по развертыванию приложения.....	80
7.1 Обзор компонентов и требований	80
7.2 Подготовка окружения	80
7.3 Развертывание базы данных	80
7.4 Развёртывание серверного API.....	81
7.5 Развёртывание панели преподавателя	81
7.6 Развёртывание мобильного клиента	82
7.7 Проверка работоспособности	82
7.8 Обслуживание и резервное копирование	82
8 Руководство пользователя.....	83
8.1 Общие шаги авторизации.....	83
8.2 Ученик. Прохождение тестов и анализ своих результатов	84
8.3 Преподаватель. Создание тестов, управление учениками и контроль успеваемости	87
8.4 Администратор: управление системой	90
9 Экономическое обоснование разработки и реализации на массовом рынке программного комплекса для подготовки к централизованному тестированию и централизованному экзамену	92
9.1 Характеристика программного средства	92
9.2 Расчет инвестиций в разработку программного средства	93
9.3 Расчет экономического эффекта от реализации программного средства на рынке	95
9.4 Расчет показателей экономической эффективности разработки и реализации программного средства на рынке	97
Заключение	100
Список литературных источников	102
Приложение А справка о проверке на заимствования	104
Приложение Б листинг программного кода	105

ВВЕДЕНИЕ

Система образования в Республике Беларусь характеризуется последовательным развитием в направлении стандартизации и цифровизации. Ключевым этапом в образовательной траектории выпускников средней школы является прохождение централизованного тестирования, а в последние годы – централизованного экзамена, который выступает основным инструментом отбора абитуриентов в учреждения высшего образования. Успешность сдачи данного экзамена определяет возможность продолжения образования, что предопределяет высокую значимость вопросов организации эффективной подготовки.

Существующие традиционные средства подготовки включают использование печатных сборников тестов и индивидуальные занятия с преподавателями. Однако данные подходы имеют ограничения. Печатные материалы не обеспечивают адаптивности, отсутствуют средства систематического анализа ошибок и построения статистики результатов. Занятия с репетитором позволяют восполнять пробелы в знаниях, но требуют значительных затрат и не обладают массовой доступностью. В условиях развития информационных технологий и широкого распространения мобильных устройств актуальным становится использование специализированных цифровых инструментов, которые способны обеспечить доступность, гибкость и регулярность подготовки.

На данный момент на рынке образовательных решений присутствует ряд приложений, ориентированных на подготовку к международным экзаменам и отдельным предметным дисциплинам. Однако данные решения, как правило, не учитывают специфику белорусской системы итоговой аттестации и не предоставляют средств взаимодействия между учеником и преподавателем. В большинстве случаев функциональность ограничивается представлением набора готовых тестов без возможности их комбинирования, анализа динамики подготовки или формирования индивидуальных подборок заданий. Данный факт обосновывает необходимость разработки программного комплекса, ориентированного на специфику национальной системы образования.

Целью настоящей работы является разработка программного комплекса, включающего мобильное приложение для операционной системы Android, серверную часть на платформе .NET и административную панель на базе Django, предназначенного для организации подготовки выпускников к централизованному экзамену. Разрабатываемое решение предусматривает хранение и предоставление тестов прошлых лет, возможность формирования смешанных и пользовательских тестов, сбор и анализ статистики решений, формирование рекомендаций по повторению тем, а также поддержку роли

преподавателя, включающей создание собственных тестов, контроль прогресса учеников и обмен материалами.

Для достижения указанной цели решаются следующие задачи: организация базы тестовых заданий и её структурирование по предметным областям и типам заданий; реализация механизмов адаптивного формирования тестов; разработка подсистемы статистики и аналитики для учащихся и преподавателей; обеспечение функциональности взаимодействия между пользователями с различными ролями; реализация механизмов защиты и безопасного хранения данных.

Практическая значимость разработки заключается в создании доступного цифрового инструмента, обеспечивающего повышение эффективности подготовки к централизованному экзамену. В отличие от существующих аналогов, предлагаемый программный комплекс ориентирован на специфику белорусской системы образования и сочетает функции самостоятельной подготовки учащихся и контроля со стороны преподавателя. Использование комплекса позволит учащимся систематизировать знания и объективно оценивать динамику подготовки, а преподавателям – эффективно организовывать образовательный процесс и осуществлять мониторинг результатов. Оформление пояснительной записки и графического материала выполнено в соответствии с требованиями стандарта предприятия [1].

Дипломный проект выполнен самостоятельно, проверен в системе «Антиплагиат». Процент оригинальности соответствует норме, установленной кафедрой информатики. Цитирования обозначены ссылками на публикации, указанные в «Списке использованных источников». Процент оригинальности работы представлен на рисунке 1 приложения А.

1 ОБЗОР И АНАЛИЗ СУЩЕСТВУЮЩИХ РЕШЕНИЙ

1.1 Онлайн-платформы подготовки к экзаменам

1.1.1 Онлайн-платформа Adukar.

Одним из наиболее известных белорусских ресурсов для подготовки к централизованному тестированию и централизованному экзамену является платформа Adukar. Сайт функционирует в формате образовательного портала и предоставляет учащимся широкий спектр инструментов для подготовки к поступлению в высшие учебные заведения. Основная функциональность платформы сосредоточена вокруг онлайн-решения тестов по различным предметам, а также предоставления методических материалов и статей, связанных с подготовкой к экзаменам.

Ключевой возможностью Adukar является доступ к реальным вариантам тестов прошлых лет. Учащиеся могут выбрать интересующий их предмет, ознакомиться с форматом заданий и пройти тест в онлайн-режиме. Система фиксирует количество правильных и неправильных ответов, что позволяет пользователю получить общее представление о своём уровне подготовки. Однако результаты представляются преимущественно в виде подсчёта правильных ответов без глубокой аналитики и детальной статистики по тематическим разделам.

Дополнительно платформа предоставляет справочные материалы, объяснения по типовым ошибкам, а также статьи о стратегии прохождения экзаменов. Это расширяет возможности ресурса, однако в большинстве случаев информация подаётся в статическом виде и не интегрирована в адаптивную систему обучения. Таким образом, учащийся получает полезные сведения, но вынужден самостоятельно выстраивать стратегию повторения тем и организации подготовки.

Сильной стороной Adukar является высокая доступность и популярность среди школьников. Также положительным фактором является соответствие тестов формату централизованного экзамена, что позволяет учащимся заранее привыкнуть к структуре заданий и регламенту.

К ограничениям платформы можно отнести отсутствие персонализации и адаптивного подхода к обучению. Система не предоставляет индивидуальных рекомендаций, не формирует подборки заданий исходя из ошибок пользователя, а также не имеет развитых инструментов для анализа динамики подготовки. Кроме того, отсутствует функциональность взаимодействия с преподавателем, что делает невозможным использование платформы в рамках полноценного образовательного процесса под руководством наставника.

В отличие от Adukar, разрабатываемый в рамках данного проекта программный комплекс предусматривает интеграцию механизмов адаптивного формирования тестов и детальной статистики по темам и категориям заданий. Пользователь сможет не только увидеть процент

правильных ответов, но и получить рекомендации по темам, требующим повторения. Также в системе реализуется функциональность для преподавателей, которые смогут создавать собственные тесты, отслеживать прогресс учеников и управлять учебным процессом. Это обеспечивает более высокий уровень гибкости и эффективности по сравнению с существующими возможностями платформы Adukar.

1.1.2 Онлайн-платформа TurboСТ.

Среди белорусских онлайн-ресурсов, ориентированных на подготовку к централизованному тестированию и централизованному экзамену, значительное распространение получила платформа TurboСТ. Данный сервис специализируется на предоставлении пользователям доступа к базе реальных тестов прошлых лет и тренировочных заданий в формате, максимально приближенном к официальным экзаменационным условиям. Основная идея платформы заключается в том, чтобы воспроизвести режим централизованного экзамена в онлайн-среде и тем самым подготовить учащихся к реальной ситуации прохождения испытаний.

Главная функциональная возможность TurboСТ заключается в прохождении полных вариантов тестов с учётом временных ограничений. Пользователь выбирает предмет, запускает тест и должен выполнить его в установленное время. По завершении работы система автоматически подсчитывает результат и переводит его в экзаменационную шкалу. Такой подход позволяет школьникам заранее привыкнуть к регламенту экзамена и тренировать навык распределения времени на выполнение заданий. Это отличает TurboСТ от некоторых других ресурсов, которые предоставляют тесты без имитации временных условий.

Сервис также предоставляет пользователям доступ к архиву тестов прошлых лет, что позволяет систематически отрабатывать типовые задания. После завершения работы можно просмотреть количество правильных и неправильных ответов, получить балл и сравнить результат с предыдущими попытками. Однако, несмотря на наличие базовой статистики, система не предусматривает детализированного анализа по тематическим разделам и не формирует индивидуальных рекомендаций для повторения материала.

К достоинствам TurboСТ можно отнести простоту использования, ориентацию именно на белорусский формат экзамена и имитацию реальной экзаменационной обстановки. Эти факторы позволяют пользователям не только проверить уровень знаний, но и психологически подготовиться к процедуре экзамена. Дополнительным преимуществом является широкая доступность: платформа функционирует в веб-браузере и не требует установки дополнительных программ.

К числу ограничений TurboСТ относится отсутствие расширенной системы аналитики, невозможность комбинирования заданий из разных тестов, а также ограниченная поддержка взаимодействия между учащимся и преподавателем. Платформа не предусматривает роли учителя, который мог бы формировать собственные варианты заданий или отслеживать прогресс

прикреплённых учеников. Кроме того, отсутствует механизм адаптивного формирования тестов, что снижает гибкость индивидуальной подготовки.

Разрабатываемый программный комплекс преодолевает обозначенные ограничения. В частности, в нём реализуется возможность генерации смешанных тестов из заданий разных лет и разных категорий, что позволяет создавать уникальные варианты, ориентированные на отработку конкретных тем. Дополнительно будет обеспечена функция персонализированных рекомендаций и подробная статистика по результатам прохождения тестов. Наличие преподавательского интерфейса позволит интегрировать самостоятельную подготовку учащегося с контролем и сопровождением со стороны педагога, что существенно расширяет сферу применения системы по сравнению с возможностями TurboСТ.

1.1.3 Онлайн-платформа 0101СТ.

Среди белорусских образовательных ресурсов, ориентированных на подготовку к централизованному тестированию и централизованному экзамену, выделяется платформа 0101СТ. В отличие от большинства аналогичных решений, данный проект позиционирует себя не только как база тестов, но и как комплексная система обучения, включающая онлайн-курсы, методические материалы и инструменты для работы с преподавателями.

Основной функционал платформы включает прохождение тестов по предметам централизованного экзамена, доступ к курсам и выполнение домашних заданий. Пользователь получает возможность работать как в веб-интерфейсе, так и через мобильное приложение. Сервис реализует систему тематической структуризации материалов: тестовые задания и учебные блоки сгруппированы по разделам, что позволяет последовательно осваивать материал. Дополнительным элементом является интеграция домашних заданий и проверочных работ, которые направляются ученикам в рамках курсовой программы.

Сильной стороной 0101СТ является комплексный подход к подготовке. Платформа сочетает тестирование с теоретическими материалами, а также поддерживает элемент взаимодействия с преподавателем. Такой формат приближает её к онлайн-школам, где учебный процесс выстраивается не только вокруг самопроверки, но и в виде последовательного освоения программы. Кроме того, наличие мобильного приложения обеспечивает удобство доступа к материалам и позволяет учащимся использовать сервис в любое время.

Однако при всех преимуществах у платформы есть ряд ограничений. Во-первых, доступ к расширенным возможностям, включая курсы и проверку домашних заданий, как правило, является платным, что снижает массовую доступность ресурса. Во-вторых, система не предусматривает свободного комбинирования заданий по выбору ученика, а учебный процесс во многом строится по фиксированным сценариям. В-третьих, несмотря на наличие взаимодействия с преподавателями в рамках курсов, отсутствует полноценная система персонализированной аналитики, которая могла бы автоматически

выявлять слабые места и формировать рекомендации.

Разрабатываемый программный комплекс отличается от 0101СТ прежде всего ориентацией на массовое использование и универсальность. В нём будет реализован механизм гибкой генерации тестов из различных источников, возможность создания кастомных заданий и тестов преподавателями, а также система рекомендаций для учащихся. Такой подход позволит совместить преимущества формального курса и свободной тренировки, обеспечив при этом доступность функционала. Дополнительно в проекте предусмотрена расширенная статистика и аналитика, что обеспечит более высокий уровень персонализации, чем в существующем решении.

1.2 Мобильные приложения

1.2.1 Мобильное приложение Adukar.

Мобильное приложение Adukar является продолжением одноимённой образовательной платформы и предназначено для подготовки школьников к централизованному тестированию и централизованному экзамену. Оно предоставляет доступ к базе тестовых заданий прошлых лет, а также дополнительным материалам, представленным на основном веб-ресурсе. Приложение ориентировано на операционную систему Android и распространяется в свободном доступе.

Ключевая функциональность мобильного приложения заключается в возможности прохождения тестов по выбранным предметам. Пользователь получает набор заданий, близкий по структуре к официальным экзаменационным материалам, и может пройти тестирование в режиме тренировки. По итогам прохождения выводится количество правильных ответов и процент успешности. Кроме того, приложение содержит справочные материалы и рекомендации, что делает его более универсальным инструментом по сравнению с исключительно веб-версией.

Сильной стороной мобильного приложения Adukar является его доступность и удобство. Пользователь имеет возможность готовиться к экзамену в любое время, используя смартфон, что значительно расширяет сценарии применения ресурса. Дополнительным преимуществом является наличие контента, соответствующего официальному формату централизованного экзамена, что позволяет учащимся знакомиться с типовыми заданиями и вырабатывать навыки работы с ними в условиях, приближённых к экзаменационным.

В то же время у приложения наблюдается ряд ограничений. Во-первых, реализована преимущественно базовая функциональность: система фиксирует количество правильных ответов, но не предоставляет расширенной статистики по темам и типам заданий. Во-вторых, отсутствует механизм адаптивного подбора тестов, что не позволяет индивидуализировать процесс подготовки. В-третьих, приложение не предусматривает возможности взаимодействия с преподавателем, отсутствует поддержка создания кастомных тестов и

совместной работы в рамках учебного процесса. Таким образом, функциональность приложения ограничена самостоятельной подготовкой без интеграции в образовательную систему.

Разрабатываемый в рамках настоящего проекта программный комплекс преодолевает данные ограничения. В отличие от мобильного приложения Adukar, он будет включать подсистему аналитики и персонализации, позволяющую автоматически выявлять пробелы в знаниях и формировать рекомендации для повторения. Дополнительно предусмотрена функциональность генерации смешанных и пользовательских тестов, что повышает гибкость подготовки. Особое значение имеет поддержка роли преподавателя, который сможет создавать собственные тесты, отслеживать прогресс учеников и делиться материалами. Таким образом, предлагаемое решение обеспечивает более высокий уровень индивидуализации подготовки и интеграцию в образовательный процесс, что выводит его за рамки возможностей существующего мобильного приложения Adukar.

1.2.2 Мобильное приложение 0101СТ.

Мобильное приложение 0101СТ является составной частью одноимённого образовательного проекта, который предоставляет учащимся комплексные курсы подготовки к централизованному тестированию и централизованному экзамену. В отличие от ряда других решений, приложение не ограничивается функцией прохождения тестов: оно интегрировано в учебный процесс и ориентировано на взаимодействие между учеником и преподавателем.

Функциональность приложения включает доступ к тестам, домашним заданиям и теоретическим материалам, которые организованы в соответствии с курсами, предлагаемыми платформой. Пользователь получает задания в структурированном виде, что позволяет двигаться по программе последовательно, изучая материал по темам. Приложение поддерживает проверку домашних заданий преподавателем, что обеспечивает обратную связь и делает подготовку более системной. Важным элементом является уведомление пользователей о новых заданиях и результатах проверки, что позволяет поддерживать дисциплину подготовки.

Сильной стороной мобильного приложения 0101СТ является комплексность подхода и интеграция в образовательный процесс. Пользователь получает не только доступ к базе тестовых заданий, но и возможность обучаться по курсу под руководством наставника. Наличие мобильного клиента обеспечивает удобство доступа к материалам в любое время и с любого устройства, а встроенные механизмы контроля помогают выработать устойчивый график подготовки.

Вместе с тем у приложения имеются определённые ограничения. Доступ ко многим функциям, включая обучение в курсовом формате и проверку домашних заданий, предоставляется на платной основе, что снижает массовую доступность ресурса. Структура заданий и последовательность их прохождения в значительной степени жёстко закреплены, что ограничивает

гибкость подготовки. Кроме того, приложение не предоставляет расширенной статистики по результатам тестирования и не реализует адаптивные механизмы подбора заданий в зависимости от индивидуальных пробелов в знаниях.

Разрабатываемый в рамках настоящего проекта программный комплекс отличается от 0101СТ ориентацией на свободное использование и доступность основных функций. В нём предусмотрена возможность самостоятельного формирования тестов, а также генерации смешанных заданий по категориям и темам. Система будет включать детальную статистику и персонализированные рекомендации, чего в текущем решении нет. Важным отличием является функциональность преподавательского интерфейса, позволяющая педагогам создавать собственные тесты, прикреплять учеников и отслеживать их прогресс без необходимости жёсткой привязки к курсовой программе. Таким образом, предлагаемое решение сочетает преимущества индивидуальной и групповой подготовки при сохранении гибкости и доступности для пользователей.

1.2.3 Мобильное приложение Gojimo.

Одним из наиболее известных международных мобильных приложений для подготовки к экзаменам является Gojimo. Данный сервис ориентирован на учащихся Великобритании и США, готовящихся к экзаменам GCSE, A-Level, SAT и аналогичным. Приложение содержит обширную базу вопросов с несколькими вариантами ответов, сгруппированных по предметам и темам.

Ключевая функциональность заключается в возможности выбора предмета, прохождения теста и получения немедленного результата с указанием правильных и неправильных ответов. Приложение предоставляет объяснения к ряду вопросов, что помогает не только проверять знания, но и разбирать ошибки. Доступ к значительной части контента бесплатный, при этом некоторые материалы распространяются по подписке.

Сильными сторонами Gojimo являются большая база вопросов, охват различных экзаменационных систем и наличие пояснений к заданиям. Интерфейс оптимизирован для мобильных устройств, что обеспечивает удобство использования. Кроме того, приложение активно применяется в разных странах, что подтверждает его востребованность среди учащихся.

К ограничениям можно отнести ориентацию на международные экзамены, что делает данное приложение непригодным для прямого использования белорусскими абитуриентами. Структура заданий и база контента не соответствуют формату централизованного экзамена в Беларуси. Кроме того, функционал приложения ограничен прохождением тестов и базовой статистикой: отсутствует система рекомендаций, взаимодействие с преподавателем и адаптивная генерация тестов.

Разрабатываемый программный комплекс, в отличие от Gojimo, будет специально ориентирован на национальный экзаменационный формат и включать механизмы персонализации. Дополнительным преимуществом станет наличие преподавательского интерфейса, позволяющего создавать

кастомные тесты и отслеживать прогресс учеников, что выводит проект за рамки возможностей существующих международных решений.

1.2.4 Мобильное приложение Examer.

Одним из приложений, ориентированных на подготовку к экзаменам в формате тестирования, является Examer. Оно позиционируется как универсальный инструмент для отработки экзаменационных заданий и поддерживает прохождение тестов по различным дисциплинам. Основная функциональность заключается в имитации условий реального экзамена: пользователь выбирает предмет, запускает тест и выполняет его в ограниченное время. По завершении система подсчитывает количество правильных ответов и формирует результат в виде общей оценки.

Сильной стороной Examer является простота интерфейса, возможность тренироваться в условиях, приближённых к экзаменационным, и поддержка нескольких предметов. Приложение ориентировано на массовое использование, доступно на мобильных устройствах и не требует сложных настроек.

К ограничениям можно отнести отсутствие развитой аналитики: система фиксирует лишь общий результат, без детализированного анализа по темам и категориям заданий. Кроме того, приложение не предусматривает адаптивного формирования тестов, отсутствует поддержка взаимодействия с преподавателем и создание кастомных вариантов заданий.

Разрабатываемый программный комплекс отличается от Examer более широким функционалом. В нём будет реализована система персонализированных рекомендаций, возможность генерации смешанных тестов по категориям, детальная статистика и роль преподавателя. Это обеспечивает более высокий уровень индивидуализации подготовки и интеграцию в образовательный процесс, что недоступно в рамках существующего мобильного приложения Examer.

1.3 Обоснование разработки программного комплекса

Анализ онлайн-платформ и мобильных приложений, предназначенных для подготовки к централизованному тестированию и централизованному экзамену, позволяет выделить ряд общих достоинств, характерных для большинства рассмотренных решений. Прежде всего, данные ресурсы обеспечивают учащимся доступ к заданиям прошлых лет, что позволяет ознакомиться с форматом экзамена и типологией вопросов. Существенным преимуществом является высокая доступность: как онлайн-платформы, так и мобильные приложения могут использоваться на различных устройствах при наличии подключения к сети Интернет, что расширяет возможности организации самостоятельной подготовки. Многие системы отличаются простотой интерфейса и низким порогом входа, что делает их удобными для массового применения. Для ряда ресурсов положительным фактором является

наличие дополнительных материалов, включающих теоретические пояснения и методические рекомендации, что способствует более глубокому освоению содержания экзаменационных дисциплин.

Вместе с тем анализ продемонстрировал наличие существенных ограничений и недостатков, которые ограничивают эффективность применения существующих решений. Наиболее распространённым недостатком является отсутствие персонализированного подхода: большинство систем фиксируют лишь общее количество правильных и неправильных ответов, не предоставляя подробной аналитики по темам и категориям заданий. Практически полностью отсутствует функциональность адаптивного формирования тестов, позволяющая строить индивидуальные траектории подготовки. В большинстве случаев не реализована роль преподавателя: пользователи не имеют возможности прикрепляться к учителю, получать от него индивидуальные задания и рекомендации, а также предоставлять доступ к своей статистике.

Выявленные недостатки определяют постановку задач для разработки нового программного комплекса. В рамках дипломного проекта предполагается реализовать систему, которая объединяет преимущества существующих решений и устраняет их ключевые ограничения. Основными задачами разработки являются:

- создание базы тестов прошлых лет с возможностью их прохождения в режиме, приближенном к экзаменационному;
- реализация механизма генерации смешанных тестов, включающего выбор количества заданий по категориям и возможность интеграции кастомных тестов;
- разработка подсистемы статистики и аналитики, обеспечивающей детализацию по темам и динамике подготовки;
- внедрение системы персонализированных рекомендаций по повторению тем на основе результатов тестирования;
- реализация функциональности преподавательского интерфейса, включающей создание собственных тестов, прикрепление учеников и отслеживание их прогресса;
- обеспечение взаимодействия между учителем и учеником в рамках единой платформы;
- интеграция push-уведомлений и напоминаний для повышения мотивации и регулярности подготовки;
- разработка мобильного клиента для Android, обеспечивающего удобный доступ к функционалу и поддержку офлайн-режима.

Таким образом, на основании анализа существующих решений сформулирована задача разработки комплексного программного продукта, ориентированного на подготовку к централизованному экзамену в Беларуси и учитывающего современные требования к цифровым образовательным системам.

2 ТРЕБОВАНИЯ К ПРОГРАММНОМУ КОМПЛЕКСУ

2.1 Общая постановка задачи

Целью разработки является создание программного комплекса, обеспечивающего эффективную подготовку выпускников школ Республики Беларусь к централизованному экзамену посредством использования мобильного приложения, серверной части и административной панели. Комплекс должен включать функциональность как для самостоятельной подготовки учащихся, так и для организации учебного процесса под руководством преподавателей.

Основная задача программного комплекса заключается в предоставлении пользователю доступа к базе тестовых заданий прошлых лет и возможности формирования новых тестов, включающих как задания из официальных источников, так и созданные преподавателями. Система должна позволять учащемуся проходить тестирование в условиях, приближённых к экзаменационным, а также использовать режимы свободной тренировки. Результаты выполнения заданий должны сохраняться и представляться в виде статистики и аналитики, обеспечивающей оценку уровня подготовки и выявление пробелов в знаниях.

Программный комплекс должен обеспечивать персонализацию учебного процесса. На основании результатов прохождения тестов пользователю должны предоставляться рекомендации по повторению отдельных тем и разделов. Для повышения эффективности подготовки необходимо предусмотреть возможность формирования смешанных тестов, позволяющих комбинировать задания из разных источников и категорий.

Особое внимание должно уделяться роли преподавателя. В системе должен быть реализован отдельный интерфейс, позволяющий учителям создавать собственные тесты, назначать их прикреплённым ученикам, отслеживать результаты их выполнения и формировать рекомендации. Таким образом, комплекс должен обеспечить интеграцию индивидуальной и групповой подготовки в рамках единой цифровой среды.

С архитектурной точки зрения программный комплекс должен представлять собой распределённую систему, включающую мобильное приложение для операционной системы Android, серверную часть на платформе ASP.NET Core, административную панель для преподавателей, реализованную с использованием Django, а также базу данных PostgreSQL для централизованного хранения информации [2]. Обмен данными между компонентами должен осуществляться по протоколу HTTP с использованием REST API.

2.2 Функциональные требования

Следующие функциональные требования сформированы на основе

анализа существующих решений и задач, поставленных в настоящем проекте, и описывают поведение системы с точки зрения пользователей различных ролей.

1 Регистрация и аутентификация пользователей. Система должна обеспечивать возможность создания нового аккаунта с указанием минимального набора данных (имя, адрес электронной почты, пароль). Должны быть реализованы процедуры входа в систему, выхода и восстановления доступа в случае утраты пароля. Для корректной работы приложения необходимо предусмотреть уникальность учётных записей и защиту от повторной регистрации с уже существующими адресами электронной почты.

2 Роли пользователей и разграничение прав. В программном комплексе должны быть предусмотрены роли «ученик», «преподаватель» и «администратор». Роль «ученик» ограничивается функциональностью прохождения тестов, просмотра статистики и получения рекомендаций. Роль «преподаватель» дополнительно включает возможность создавать тесты, назначать их ученикам, просматривать их статистику и формировать аналитические отчёты. Роль «администратор» используется для управления банком заданий и пользователями через административную панель.

3 Механизм привязки учеников к преподавателю. Для обеспечения взаимодействия необходимо реализовать функциональность назначения учеников к определённому преподавателю. Должна быть предусмотрена возможность приглашения через уникальный код или подтверждение запроса со стороны ученика. После привязки преподаватель получает доступ к истории решений и статистике данного ученика.

4 Банк тестов прошлых лет. Система обязана содержать структурированную базу тестов, включающую все задания централизованного тестирования и экзамена прошлых лет. Для каждого задания должны храниться метаданные: предмет, год проведения, номер варианта, категория или тема, уровень сложности, правильный ответ и пояснение. Должен быть реализован поиск и фильтрация по указанным параметрам.

5 Прохождение готовых тестов прошлых лет. Пользователь с ролью «ученик» должен иметь возможность выбрать полный вариант теста, запустить его с таймером и пройти в условиях, соответствующих экзаменационным. По завершении выполнения система должна подсчитать количество правильных и неправильных ответов, отобразить результат и сохранить его в истории решений.

6 Формирование смешанных тестов. Приложение должно предоставлять пользователю возможность сформировать тест с заданным количеством вопросов по различным категориям и темам. Пользователь должен иметь возможность указать предмет, количество заданий каждой категории, уровень сложности, а также решить, будут ли включены задания, созданные преподавателем.

7 Создание пользовательских тестов преподавателями. Пользователь с ролью «преподаватель» должен иметь возможность формировать тесты из

заданий банка или добавлять новые вопросы. Должна быть предусмотрена возможность редактирования и удаления тестов, а также публикации их для прикрепленных учеников.

8 Создание пользовательских тестов. Преподаватель должен иметь возможность создавать собственные пользовательские тесты через специализированную браузерную панель и передавать эти тесты для выполнения выбранным ученикам. Ученик получает уведомление о новом тесте и может пройти его в своём приложении. Результаты выполнения должны сохраняться и быть доступны как ученику, так и преподавателю.

9 История решений тестов. Для каждого пользователя необходимо хранить список всех пройденных тестов. Запись должна содержать дату прохождения, количество заданий, результат в баллах, перечень правильных и неправильных ответов, а также время, затраченное на выполнение.

10 Аналитика и статистика. Система должна формировать статистику на основе накопленной истории решений. Для ученика статистика отображается в виде распределения ошибок по темам и динамики подготовки за выбранный период. Для преподавателя предоставляется доступ к агрегированной статистике по прикрепленным ученикам с возможностью просмотра индивидуальных данных.

11 Формирование рекомендаций. На основании анализа ошибок система должна автоматически определять темы, требующие повторения. Пользователь получает список тем, упорядоченных по приоритетности, а также уведомления с напоминанием о необходимости их повторить.

12 Система уведомлений. Программный комплекс должен поддерживать push-уведомления на мобильных устройствах. Уведомления используются для информирования о назначении новых тестов, появлении рекомендаций, обновлении результатов и напоминаниях о регулярной подготовке.

Реализация указанных функций позволит обеспечить комплексный подход к подготовке к централизованному экзамену в условиях современной цифровой образовательной среды.

В завершение описания функциональных требований к системе основные варианты использования и роли пользователей в наглядной форме представлены на диаграмме прецедентов на рисунке 2.1.

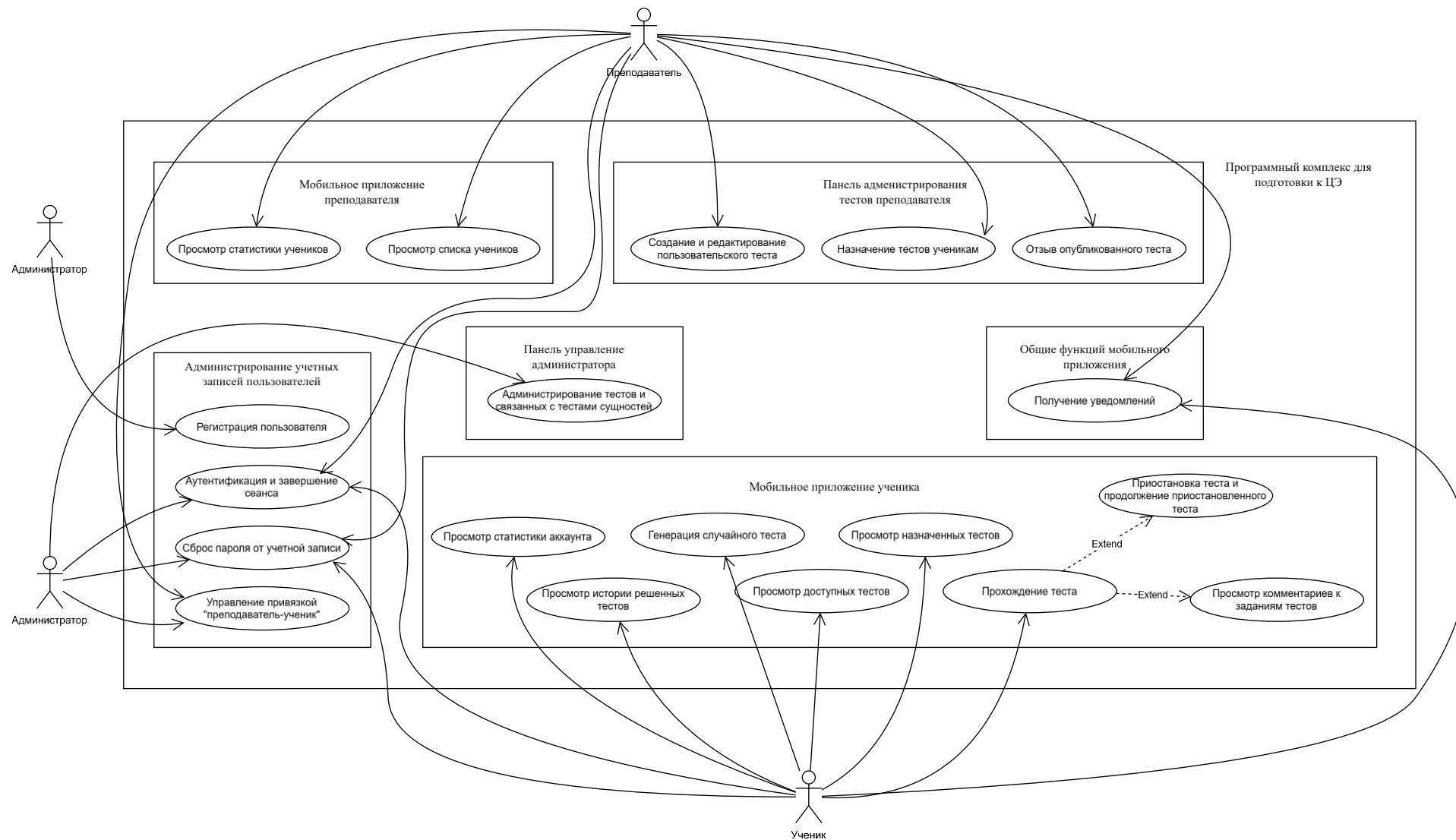


Рисунок 2.1 – Диаграмма прецедентов программного комплекса

2.3 Нефункциональные требования

С точки зрения производительности серверная часть на базе ASP.NET Core должна обеспечивать обработку стандартных запросов (получение списка тестов, отправка результатов попытки, загрузка статистики) за время не более одной секунды при одновременной работе не менее 500 активных пользователей [3]. Мобильное приложение на Kotlin должно обеспечивать мгновенный отклик интерфейса при переходах между основными экранами и отображать результаты теста не позднее чем через две секунды после отправки их на сервер.

Надёжность и отказоустойчивость являются важными характеристиками комплекса. В случае обрыва сетевого соединения мобильное приложение должно сохранять прогресс решения теста локально в SQLite и автоматически синхронизировать данные с сервером после восстановления связи. Результаты всех завершённых тестов должны храниться в базе данных PostgreSQL, где необходимо настроить ежедневное резервное копирование с возможностью восстановления не менее чем за последние семь дней.

Ключевым требованием является удобство интерфейсов. Мобильное приложение должно соответствовать гайдлайнам Material Design для Android, обеспечивая единообразие элементов и предсказуемость поведения интерфейса. Основной сценарий пользователя — выбор теста, его прохождение, просмотр результатов и статистики — должен выполняться не более чем в три шага. Веб-инструмент для преподавателей должен иметь адаптивный интерфейс на базе Django Templates, корректно отображаться в современных браузерах (Google Chrome, Mozilla Firefox, Microsoft Edge) и поддерживать разрешения экранов от 1366×768 пикселей.

Масштабируемость системы должна обеспечиваться за счёт использования модульной архитектуры серверной части и возможности горизонтального масштабирования базы данных PostgreSQL. Архитектура приложения должна позволять без доработок ядра добавлять новые предметы, категории и типы заданий. При росте нагрузки до 1000 активных пользователей система должна обеспечивать устойчивую работу без деградации производительности.

Совместимость комплекса должна гарантировать работу мобильного приложения на устройствах с Android версии 8.0 (Oreo) и выше. Серверная часть должна развёртываться на операционных системах Windows Server 2019 и Ubuntu 22.04 LTS. Для взаимодействия между компонентами должен использоваться протокол HTTPS, а обмен данными реализован в формате JSON поверх REST API.

Особое внимание уделяется безопасности. Для аутентификации и авторизации пользователей должны использоваться JWT-токены, генерируемые сервером ASP.NET Core и передаваемые в заголовках запросов. Пароли пользователей должны храниться только в виде хэшей,

сформированных с использованием алгоритма bcrypt с солью. Передача данных между клиентом и сервером должна осуществляться исключительно по протоколу HTTPS с использованием TLS 1.3. Система должна быть защищена от SQL-инъекций за счёт использования параметризованных запросов в Entity Framework Core, а также от атак XSS и CSRF средствами встроенных middleware. Доступ к функциям должен разграничиваться по ролям «ученик» и «преподаватель».

Поддержка и сопровождение комплекса должны обеспечиваться за счёт документирования API с использованием Swagger/OpenAPI, наличия юнит-тестов для основных сервисов на уровне серверной части и возможности развёртывания обновлений без потери пользовательских данных. В системе должно быть реализовано логирование ключевых операций с использованием библиотеки Serilog.

2.4 Требования к интерфейсам

2.4.1 Пользовательские интерфейсы.

Мобильное приложение для операционной системы Android должно быть ориентировано на пользователя с ролью ученика. Интерфейс обязан соответствовать рекомендациям Material Design и обеспечивать единообразие элементов управления. Основные сценарии работы должны выполняться интуитивно и без лишних шагов.

Приложение должно включать следующие основные экраны:

- главный экран с выбором предмета и типа теста;
- экран прохождения теста с таймером, навигацией по заданиям и возможностью возврата к предыдущим вопросам;
- экран результатов с перечнем правильных и неправильных ответов, общей оценкой и временем выполнения;
- раздел статистики с историей решений, аналитикой по темам и динамикой успеваемости;
- интерфейс уведомлений о новых тестах, результатах и рекомендациях.

Веб-инструмент для преподавателей реализуется на базе Django Templates и должен обеспечивать удобный доступ к функциям создания и редактирования тестов, назначения их ученикам и просмотра статистики. Он обязан корректно работать в браузерах Google Chrome, Mozilla Firefox и Microsoft Edge. Минимальное разрешение экрана — 1366×768 пикселей.

Функциональные элементы веб-интерфейса включают:

- раздел «Создание теста» с возможностью добавления заданий из базы, создания новых вопросов и их упорядочивания;
- раздел «Мои тесты» для редактирования и публикации тестов;

2.4.2 Системные интерфейсы

Взаимодействие между клиентскими приложениями и серверной частью должно осуществляться через REST API, реализованный на ASP.NET Core.

Для обмена данными используется формат JSON, транспортным протоколом является HTTPS. Документация интерфейсов должна формироваться автоматически с использованием Swagger/OpenAPI.

Основные группы методов API включают:

- аутентификацию и авторизацию пользователей;
- доступ к базе тестов прошлых лет с возможностью фильтрации по предмету и году;
- сохранение и получение результатов прохождения тестов;
- предоставление статистики и аналитики ученику и преподавателю;
- управление пользовательскими тестами;
- получение и обработку уведомлений.

Для всех операций серверная часть обязана возвращать корректные коды HTTP-ответов. Ошибки должны сопровождаться информативными сообщениями в формате JSON. Авторизация осуществляется с помощью JWT-токенов, которые передаются в заголовках запросов. Доступ к данным разграничивается в зависимости от роли пользователя.

2.5 Требования к безопасности

Программный комплекс должен обеспечивать высокий уровень защиты персональных данных пользователей, устойчивость к внешним угрозам и предотвращение несанкционированного доступа. Безопасность рассматривается на уровне клиентских приложений, серверной части и базы данных.

Защита данных пользователей предусматривает:

- хранение паролей исключительно в виде хэш-сумм с использованием алгоритма bcrypt с солью, что исключает возможность их восстановления даже при компрометации базы;
- использование механизма политики сложности паролей, включающей минимальную длину и обязательное наличие различных типов символов;
- передачу всех данных только по защищённому протоколу HTTPS с использованием TLS версии 1.3, что предотвращает перехват информации;
- шифрование критически важных данных (например, электронных адресов) с использованием встроенных средств PostgreSQL;
- минимизацию хранения персональной информации: обязательными являются только имя, адрес электронной почты и роль пользователя, остальные данные носят факультативный характер.

Аутентификация и авторизация должны быть реализованы с использованием современных механизмов управления сессиями [4]. Для доступа к API применяются JWT-токены, генерируемые сервером ASP.NET Core и передаваемые в заголовках запросов. Каждый токен должен иметь ограниченный срок действия и автоматически обновляться по мере истечения. Система должна предусматривать возможность отзыва токена при подозрении на компрометацию. Разграничение прав доступа должно строиться на основе

ролевой модели: «ученик» имеет доступ только к собственным данным и назначенным тестам, «преподаватель» получает доступ к информации прикрепленных учеников, а доступ к административным функциям создания и модификации банка заданий возможен только через защищенный веб-интерфейс преподавателя.

Безопасность серверной части и базы данных должна обеспечиваться:

- использованием ORM Entity Framework Core для защиты от SQL-инъекций и безопасного взаимодействия с PostgreSQL;
- обязательной валидацией всех входных данных на стороне сервера;
- применением встроенных механизмов защиты ASP.NET Core и Django от XSS- и CSRF-атак [5];
- ограничением числа попыток входа для предотвращения атак перебора паролей [6];
- изоляцией базы данных от внешнего доступа, её подключение возможно только через сервер приложений;
- ведением журналов событий всех критически важных операций, включая входы в систему, создание и редактирование тестов, просмотр статистики и выдачу рекомендаций;
- регулярным обновлением зависимостей и применением актуальных патчей безопасности.

Мобильное приложение должно учитывать специфику использования на личных устройствах. Доступ к профилю должен быть защищён паролем или биометрической аутентификацией при наличии соответствующего оборудования. Хранение временных данных (например, незавершённых тестов в офлайн-режиме) должно осуществляться в защищённом локальном хранилище SQLite с шифрованием. После синхронизации данные должны удаляться из локального хранилища, чтобы минимизировать риск их компрометации при утрате устройства.

Веб-инструмент для преподавателей должен защищаться дополнительными средствами. Доступ осуществляется только по имени пользователя и паролю с обязательной проверкой сложности. Сессии пользователей должны иметь ограниченное время жизни и завершаться при длительном бездействии. Должна быть реализована защита от подбора адресов URL и доступа к ресурсам без авторизации.

В разделе сформулированы и структурированы требования к программному комплексу. Определены задачи системы, функциональные возможности для разных ролей пользователей и нефункциональные параметры: производительность, надежность, удобство интерфейса и безопасность. Установлены требования к интерфейсам взаимодействия и представлена диаграмма прецедентов, отражающая основные сценарии работы. Сформированный комплекс требований является достаточным для дальнейшего проектирования и реализации системы.

3 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО КОМПЛЕКСА

3.1 Архитектурные решения

Разрабатываемый программный комплекс проектируется на основе многослойной клиент–серверной архитектуры с чётким разделением ответственности между уровнями представления, прикладной логики и хранения данных. Такой подход обеспечивает модульность, масштабируемость и упрощает сопровождение системы.

На уровне представления функционируют два клиента: мобильное приложение для учеников и веб-инструмент для преподавателей. Мобильное приложение создаётся для операционной системы Android на языке Kotlin с использованием библиотек Jetpack Compose и Retrofit для работы с интерфейсом и сетевыми запросами. Оно обеспечивает выполнение всех функций, связанных с подготовкой учащегося: прохождение тестов, просмотр результатов, доступ к истории решений и получение рекомендаций. Веб-инструмент для преподавателей реализуется на основе Django Templates и предоставляет интерфейс для создания и редактирования тестов, назначения их прикреплённым ученикам. Интерфейс ориентирован на браузеры и мобильные устройства, что обеспечивает удобство работы с системой в разных сценариях.

Уровень прикладной логики представлен серверной частью, реализуемой на платформе ASP.NET Core 8 [7]. Сервер выполняет ключевые функции: аутентификацию и авторизацию пользователей, управление ролями, доступ к банку тестов прошлых лет, обработку пользовательских запросов, формирование статистики и рекомендаций, работу с уведомлениями. Для взаимодействия с клиентами используется REST API с передачей данных в формате JSON. Авторизация реализуется с применением JWT-токенов, которые передаются в заголовках HTTP-запросов. В серверной части используется модульная структура, включающая подсистемы управления пользователями, тестами, статистикой и аналитикой. Для устойчивости и удобства сопровождения сервер документируется с помощью Swagger [8].

Уровень хранения данных реализуется в СУБД PostgreSQL версии 15 [9]. В базе данных сохраняются сведения о пользователях, тестах, заданиях, результатах попыток, статистике и уведомлениях. Доступ к базе данных осуществляется исключительно через серверную часть с использованием ORM Entity Framework Core, что обеспечивает защиту от SQL-инъекций и унифицирует работу с данными. Для повышения отказоустойчивости предусматривается ежедневное резервное копирование базы, а в перспективе – настройка репликации для масштабируемости и балансировки нагрузки.

Архитектурные решения предусматривают использование только защищённого протокола передачи данных HTTPS. Развёртывание серверной части может осуществляться как на физических серверах, так и в облачных средах: Microsoft Azure или Amazon Web Services. Для удобства

развертывания и сопровождения должно быть предусмотрено использование контейнеризации на базе Docker. Такой подход обеспечит гибкость в управлении системой и снизит риски, связанные с зависимостью от конкретной инфраструктуры.

Физическая структура программного комплекса с указанием вычислительных узлов, их аппаратного и программного окружения отражена на диаграмме развёртывания, приведённой на рисунке 3.1, а также на чертеже ГУИР.253504.001 ПЛ.

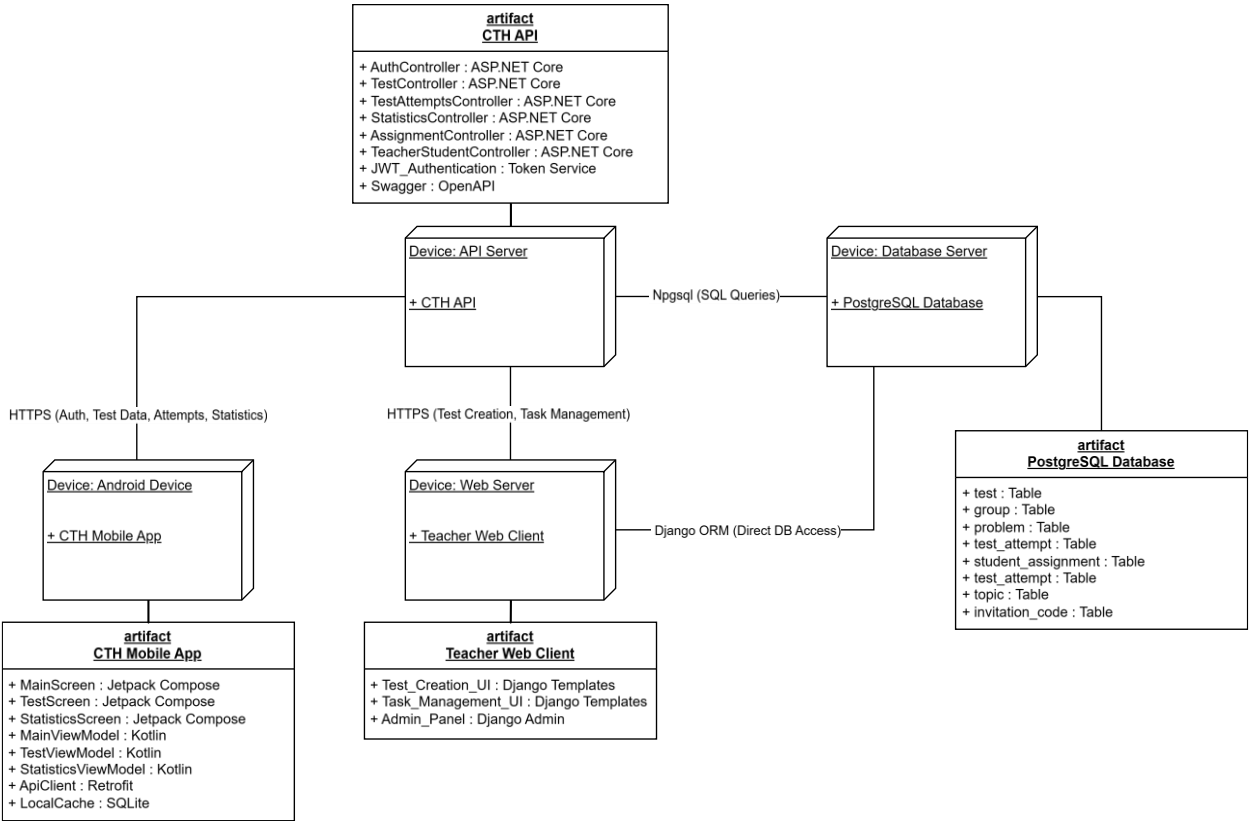


Рисунок 3.1 – Диаграмма развёртывания программного комплекса

Выбор многослойной архитектуры обусловлен необходимостью обеспечения масштабируемости и гибкости. В отличие от монолитного решения, разделение на уровни позволяет модернизировать отдельные компоненты без риска нарушения работы всего комплекса. Использование микросервисной архитектуры на данном этапе признано избыточным из-за ограниченного объёма функциональности и повышенной сложности сопровождения. Трёхуровневая архитектура является оптимальным вариантом для решения поставленных задач, так как сочетает относительную простоту реализации с возможностью дальнейшего расширения.

3.2 Модульная структура системы

Программный комплекс включает в себя ряд подсистем, каждая из которых отвечает за выполнение определённого набора функций. Модульная структура обеспечивает логическое разделение компонентов и их взаимодействие в рамках общей архитектуры.

Модуль аутентификации и авторизации обеспечивает регистрацию новых пользователей, вход в систему и контроль доступа. В данном модуле реализуются механизмы проверки учётных данных, генерации и валидации JWT-токенов, а также процедуры восстановления пароля через электронную почту [10]. Модуль поддерживает ролевую модель доступа и определяет права пользователей в зависимости от их статуса: ученик или преподаватель.

Модуль управления пользователями отвечает за хранение и обработку информации о зарегистрированных аккаунтах. В рамках этого модуля преподаватель имеет возможность добавлять учеников к своему аккаунту, просматривать их данные и при необходимости изменять статус доступа.

Модуль управления тестами реализует функциональность работы с тестовыми материалами. Он включает хранение заданий прошлых лет, поддержку поиска и фильтрации по предметам, темам и уровням сложности. В модуле предусмотрена возможность создания новых тестов преподавателями, редактирования существующих заданий, а также генерации смешанных тестов по параметрам, заданным пользователем.

Модуль прохождения тестов обеспечивает проведение тестирования учащихся в условиях, приближённых к экзаменационным. В него входят механизмы отображения заданий, отслеживания времени выполнения, сохранения промежуточных данных и фиксации финального результата. Результаты выполнения передаются в систему статистики и сохраняются в базе данных для последующего анализа.

Модуль статистики и аналитики отвечает за обработку накопленных данных о прохождении тестов. Он формирует отчёты по отдельным ученикам и группам, анализирует динамику успеваемости и распределение ошибок по темам. В рамках этого модуля реализуется подсистема автоматической генерации рекомендаций, которая определяет разделы программы, требующие повторения.

Модуль уведомлений используется для информирования пользователей о событиях в системе. Он формирует и отправляет сообщения о назначении новых тестов, появлении результатов, необходимости повторения тем и регулярной подготовке. Для мобильного приложения уведомления реализуются через push-сервисы, а веб-интерфейс использует встроенные механизмы оповещения.

Логическое разделение программного комплекса на взаимодействующие модули и подсистемы с указанием их иерархии и информационных связей схематично показано на рисунке 3.2.

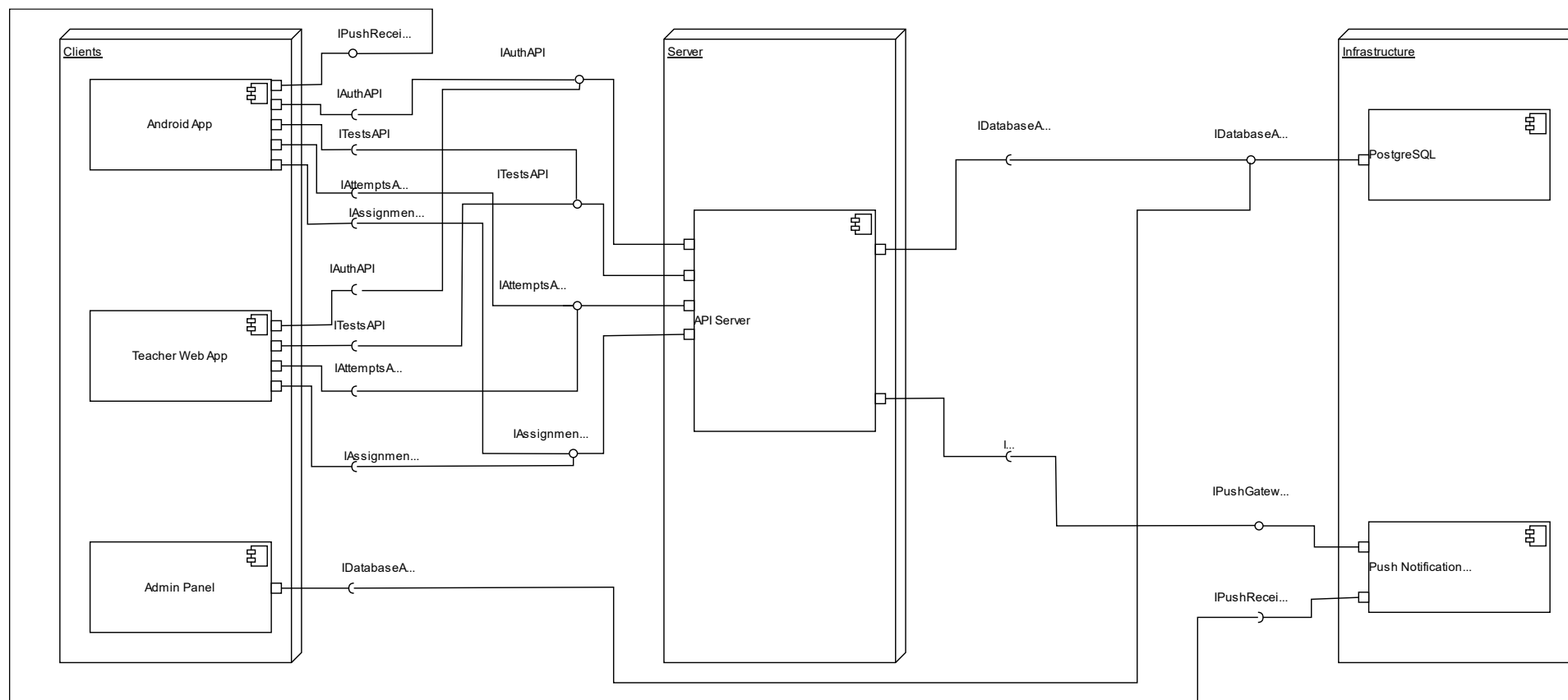


Рисунок 3.2 – Диаграмма компонентов программного комплекса

Модуль веб-интерфейса для преподавателей реализуется на базе Django Templates и предоставляет специализированный набор инструментов. Он включает функциональность создания и редактирования тестов, назначения их прикрепленным ученикам, просмотра их истории решений и анализа статистики. Интерфейс адаптирован для работы в браузере и позволяет выполнять все действия через удобные формы и таблицы.

Модуль административного управления данными используется для администрирования основных сущностей системы: пользователей, тестов, заданий. Он реализуется на основе встроенной панели Django Admin, что позволяет управлять содержимым базы данных через веб-интерфейс без необходимости создания отдельного интерфейса. Данный модуль используется разработчиками и администраторами системы, не предназначен для массового доступа преподавателей или учеников и выполняет функции технического сопровождения.

3.3 Взаимодействие компонентов

Взаимодействие компонентов программного комплекса строится по классической трёхуровневой схеме «клиент–сервер», предусматривающей чёткое разделение на уровень представления (клиентская часть), уровень бизнес-логики (серверная часть) и уровень хранения данных (база данных). Такая архитектура обеспечивает масштабируемость, независимость компонентов и упрощает сопровождение системы. Все коммуникации между клиентскими приложениями и серверной частью осуществляются по защищённому протоколу HTTPS с использованием REST API [11], что гарантирует конфиденциальность и целостность передаваемых данных. Обмен данными выполняется в текстовом формате JSON, который обеспечивает высокую скорость обработки и удобство интеграции между разнородными компонентами.

Мобильное приложение используется как учениками, так и преподавателями, при этом функциональные возможности интерфейса адаптируются в зависимости от роли пользователя. В первом случае приложение обеспечивает доступ к прохождению тестов в интерактивном режиме, автоматическому сохранению и последующему просмотру результатов, формированию детализированной статистики по каждому тесту, а также получению персонализированных рекомендаций на основе успеваемости. Во втором случае преподавательский интерфейс мобильного приложения дополняется расширенными возможностями: работа со статистикой прикрепленных учеников, просмотр динамики их успеваемости, анализ истории решений и выявление типичных ошибок. Все запросы, поступающие от мобильного клиента, передаются на сервер, где они проходят необходимую обработку, валидацию и сохраняются в базе данных.

Веб-инструмент для преподавателей реализован на основе Django Templates и выполняет ключевые функции по созданию и редактированию

тестов. Преподаватель формирует новый тест, выбирая задания из предварительно наполненного банка, либо создаёт собственные вопросы с настройкой параметров сложности, типов ответов и временных ограничений. После завершения редактирования преподаватель публикует тест и назначает его конкретному списку учеников или целым учебным группам. Все операции, выполняемые в веб-инструменте, осуществляются через REST API, что обеспечивает единый канал взаимодействия с серверной частью и согласованность данных. Следует отметить, что доступ к аналитике и детализированным результатам в веб-инструменте не предусмотрен – эти данные отображаются исключительно в мобильном приложении, что разграничивает сценарии использования и упрощает интерфейсы.

Серверная часть реализует всю бизнес-логику программного комплекса и отвечает за централизованную обработку всех запросов как от мобильного приложения, так и от веб-инструмента. В её функции входит: валидация входных данных, управление доступом на основе ролей (ученик, преподаватель, администратор), обработка результатов прохождения тестов с автоматическим подсчётом баллов, формирование статистических отчётов и рекомендаций. Для взаимодействия с базой данных используется ORM Entity Framework Core, который обеспечивает безопасный параметризованный доступ к информации, предотвращает SQL-инъекции и предоставляет централизованное хранение всех сущностей системы: пользователей, тестов, заданий, результатов и сессий.

Административная панель Django Admin взаимодействует с базой данных напрямую через встроенный ORM Django, минуя REST API. Она предназначена исключительно для администраторов системы и используется для управления основными сущностями: пользователями (регистрация, блокировка, назначение ролей), тестами (просмотр, модерация, удаление) и заданиями (добавление, редактирование, категоризация). В отличие от мобильного приложения и веб-инструмента, административная панель не подразумевает использования API и ориентирована на задачи технического обслуживания, сопровождения и оперативного исправления данных без участия конечных пользователей.

Временной порядок взаимодействия компонентов программного комплекса для различных сценариев удобно представить в виде диаграмм последовательности. На рисунке 3.3 изображён сценарий аутентификации и авторизации пользователя, включающий обмен JWT-токенами.

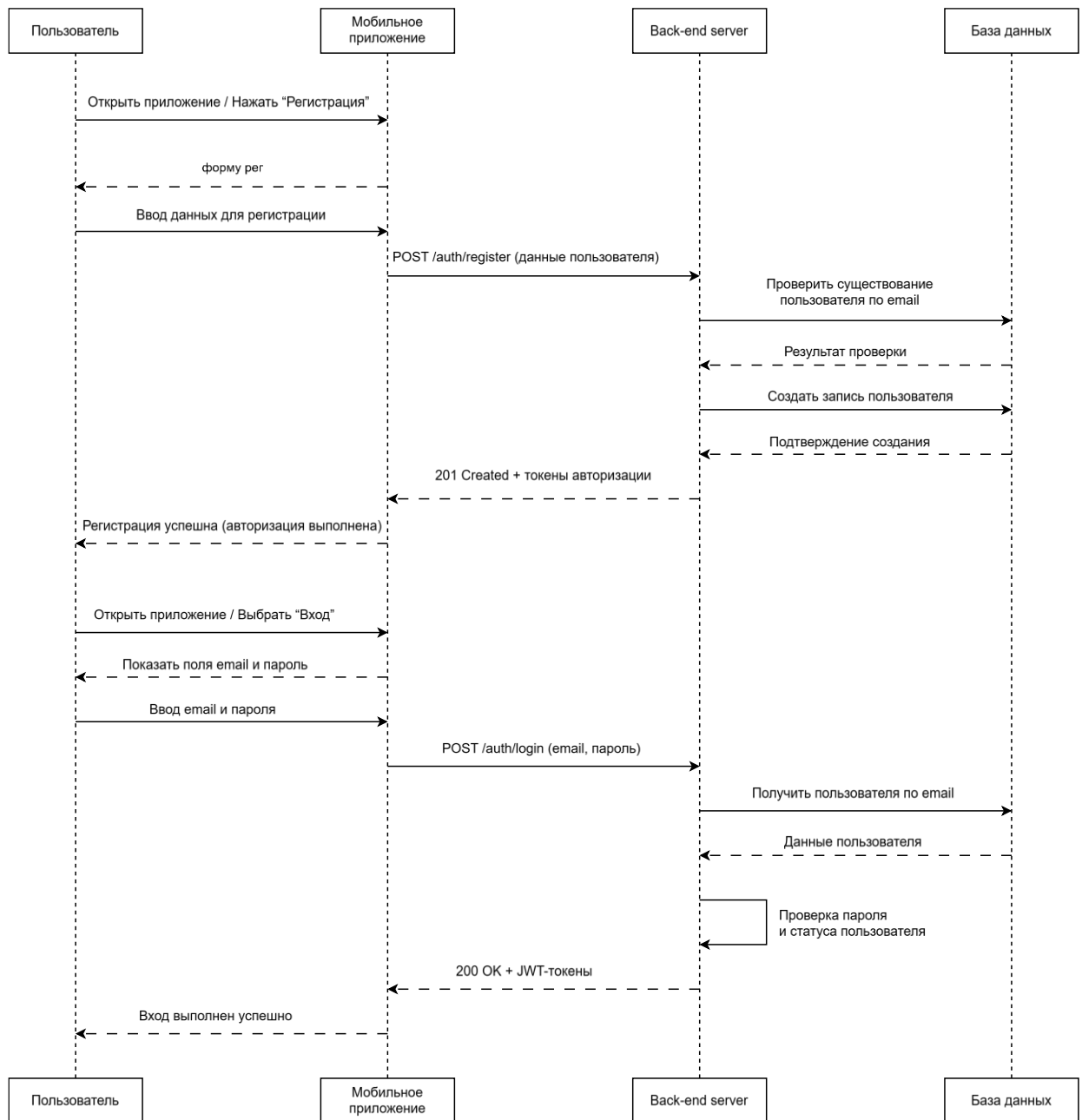


Рисунок 3.3 — Диаграмма последовательности для процесса аутентификации и авторизации пользователя

Сценарий прохождения теста, иллюстрирующий потоки данных между мобильным клиентом, сервером приложений и базой данных от момента выбора теста до получения окончательного результата, представлен на рисунке 3.4.

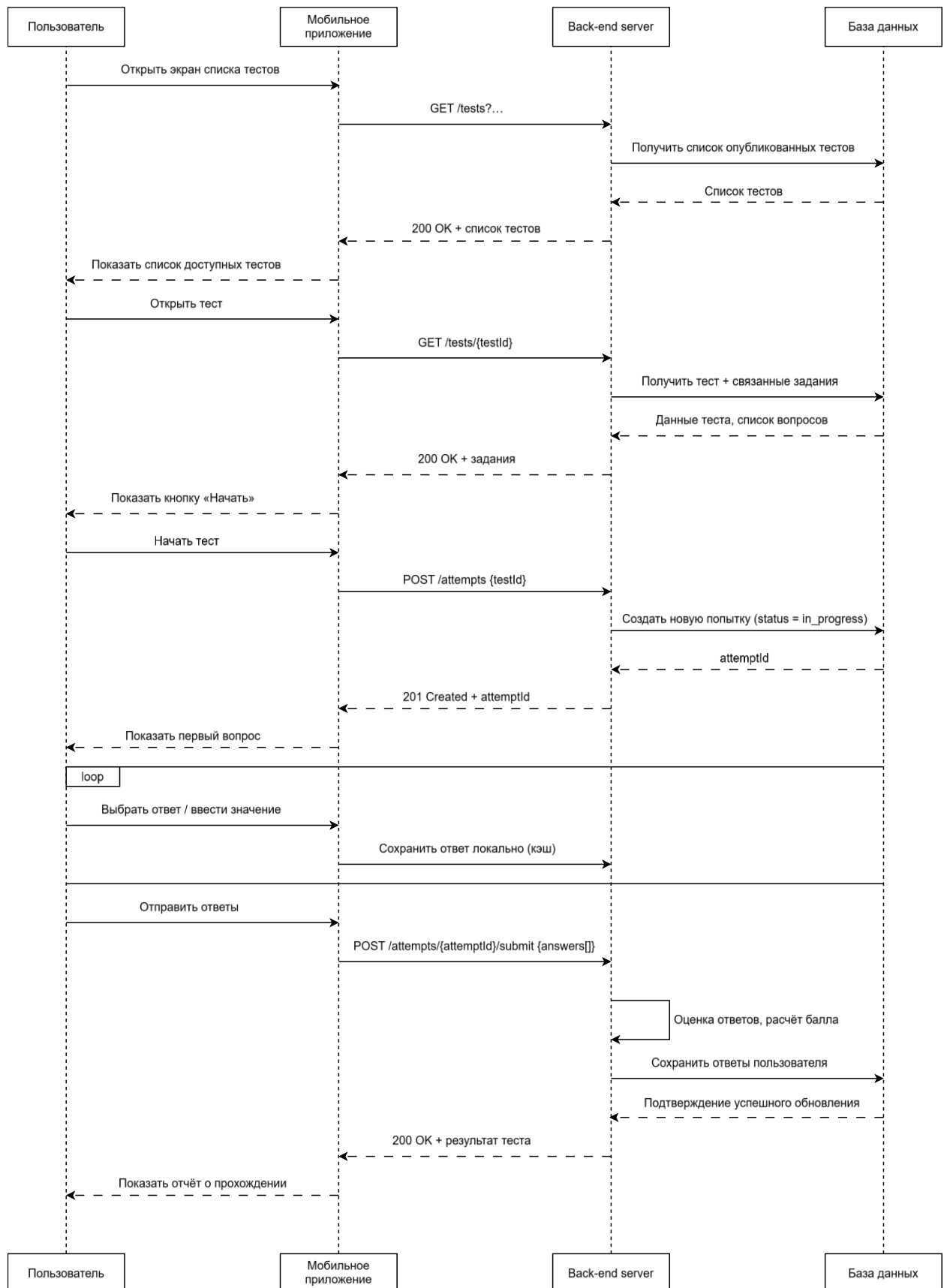


Рисунок 3.4 – Диаграмма последовательности для процесса прохождения теста

Таким образом, взаимодействие компонентов можно описать следующим образом:

- мобильное приложение ↔ серверная часть ↔ база данных;
- веб-инструмент (создание тестов) ↔ серверная часть ↔ база данных;
- административная панель Django Admin ↔ база данных напрямую через ORM.3.3 .

3.4 Сценарии использования системы

Ниже приведены ключевые сценарии взаимодействия пользователей и компонентов системы. Для каждого сценария указаны участники, предпосылки, последовательность действий и ожидаемые результаты, а также основные ветвления и пограничные ситуации:

1 Регистрация пользователя

Участники: гость.

Предпосылки: доступ к мобильному приложению Android.

Шаги:

1 Пользователь открывает экран регистрации и вводит следующие данные: имя, адрес электронной почты, пароль. Также осуществляется выбор роли из двух возможных вариантов: «ученик» или «преподаватель».

2 Клиентское приложение формирует и отправляет на сервер соответствующий запрос с использованием протокола HTTPS и формата JSON.

3 Сервер выполняет валидацию полученных данных, проверяя уникальность адреса электронной почты, соответствие пароля установленной политике сложности, а также корректность выбранной роли.

4 При успешном прохождении валидации в системе создаётся новая учётная запись, после чего сервер возвращает клиенту подтверждение регистрации и JWT-токен для обеспечения первичной сессии.

Результат: создана учётная запись, пользователь аутентифицирован.

Ветвления: e-mail уже зарегистрирован; слабый пароль; сетевой сбой – клиент показывает ошибку и предлагает повторить.

2 Вход и выход из системы

Участники: учитель, ученик.

Шаги:

1 Пользователь вводит в соответствующие поля адрес электронной почты и пароль для выполнения входа в систему.

2 Сервер проверяет корректность переданных учётных данных и в случае успеха выдаёт клиенту JWT-токен, имеющий ограниченный срок действия.

3 Клиентское приложение сохраняет полученный токен безопасным способом с использованием EncryptedSharedPreferences и Keystore, а затем применяет его при последующих запросах, добавляя в заголовки.

4 При выполнении выхода из системы клиент удаляет все локальные маркеры сессии, хранящиеся на устройстве.

Ветвления: неверные данные; превышен лимит попыток; истёкший токен – клиент запрашивает новый по повторному входу.

3 Привязка ученика к преподавателю

Участники: ученик, преподаватель.

Предпосылки: обе стороны зарегистрированы и вошли в систему.

Шаги:

1 Преподаватель в мобильном приложении создаёт ограниченный по времени код приглашения.

2 Ученик вводит код на экране «Привязка к преподавателю».

3 Сервер валидирует код, устанавливает связь «преподаватель – ученик».

4 Обеим сторонам отправляется подтверждение (push/внутреннее уведомление).

Ветвления: просроченный/неверный код; ученик уже привязан – система сообщает текущее состояние.

4 Просмотр доступных тестов

Участники: ученик, учитель.

Шаги:

1 Пользователь открывает раздел «Тесты».

2 Клиент запрашивает у сервера перечни: готовые тесты прошлых лет (с фильтрами «предмет», «год»), назначенные пользовательские тесты, конструктор смешанного теста.

3 Сервер возвращает списки с метаданными (длительность, число заданий, категории).

4 Результат: на экране список отфильтрованных тестов, каждая позиция в котором представлена названием теста и его метаданными.

5 Генерация смешанного теста

Участники: ученик.

Шаги:

1 Ученик выбирает предмет, задаёт количество заданий по категориям/темам и уровень сложности; при необходимости включает опцию «добавлять задания из тестов преподавателя».

2 Клиент отправляет параметры генерации на сервер.

3 Сервер валидирует доступность пула заданий под заданные параметры; фиксирует «состав» с уникальным seed (для воспроизводимости); возвращает сводку теста.

4 Ученик подтверждает запуск.

5 Ветвления: недостаточно заданий в одной из категорий – сервер возвращает допустимые пределы и предлагает скорректировать параметры.

Логика работы алгоритма генерации смешанного теста представлена на рисунке 3.5, а также на чертеже ГУИР.253504-01 СА. Кроме того, исходный код метода приведен в приложении Б.

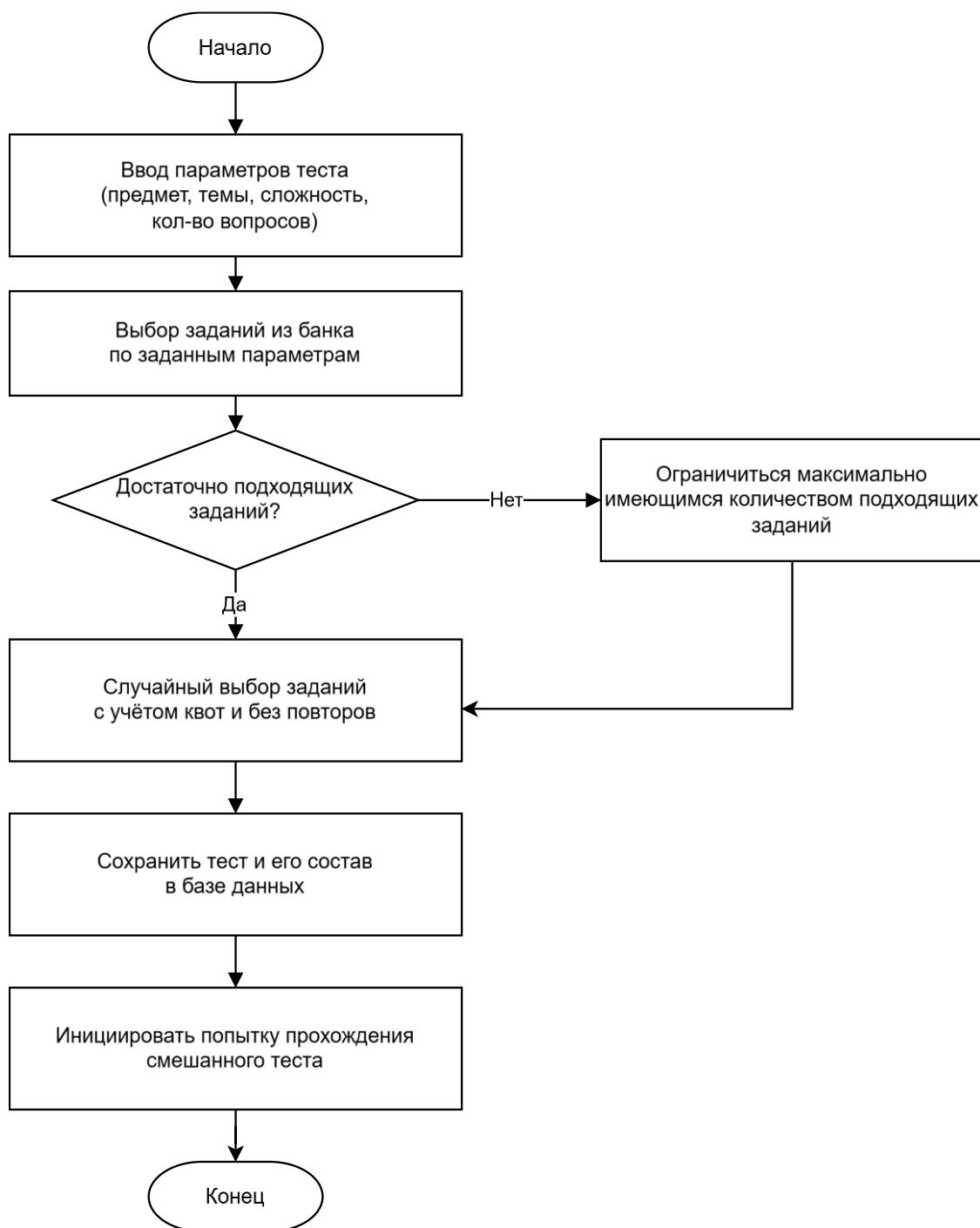


Рисунок 3.5 — Схема алгоритма генерации смешанного теста

6 Прохождение полного варианта теста прошлых лет

Участники: ученик.

Шаги:

1 Ученик выбирает вариант (предмет/год/номер), запускает тест; сервер фиксирует старт попытки.

2 Клиент отображает таймер, обеспечивает навигацию по вопросам, локально кэширует ответы.

3 При каждом ответе клиент сохраняет черновик локально.

4 По завершении ученик отправляет ответы на сервер.

Результат: сервер проверяет ответы, вычисляет результат, фиксирует попытку с временными метками и возвращает подробности.

7 Прохождение пользовательского теста (назначенного преподавателем)

Участники: ученик.

Шаги:

1 Ученик получает уведомление о назначенном тесте и видит его в разделе «Пользовательские».

2 Запускает тест; сервер фиксирует попытку, состав и параметры (таймер, порядок).

3 Дальнейшая логика идентична сценарию, описанному в пункте 6.

Ветвления: тест отозван преподавателем до запуска – скрыть из списка; попытка повторного запуска при политике «одна попытка» – отказ.

8 Продолжение прерванного теста

Участники: ученик.

Шаги:

1 При аварийном завершении/обрыве сети клиент хранит локальный снимок состояния: текущий вопрос, ответы, остаток времени.

2 При повторном входе клиент запрашивает у сервера статус попытки; если попытка активна – восстанавливает состояние; при строгом таймере – учитывается прошедшее время.

3 После восстановления сети локальные изменения синхронизируются.

Ветвления: истёк лимит времени – клиент переводит пользователя к экрану результатов.

9 Завершение теста и подсчёт результата

Участники: ученик, сервер.

Шаги:

1 Клиент отправляет финальный набор ответов с идентификатором попытки.

2 Сервер проверяет целостность запроса и выполняет оценку: первичные баллы, при наличии – шкалирование.

3 Сервер сохраняет попытку, помечает её завершённой.

4 Клиент получает результат и отображает детальную раскладку.
Серверный алгоритм обработки завершения попытки схематично изображён на рисунке 3.6, а также на чертеже ГУИР.253504-01 СА.

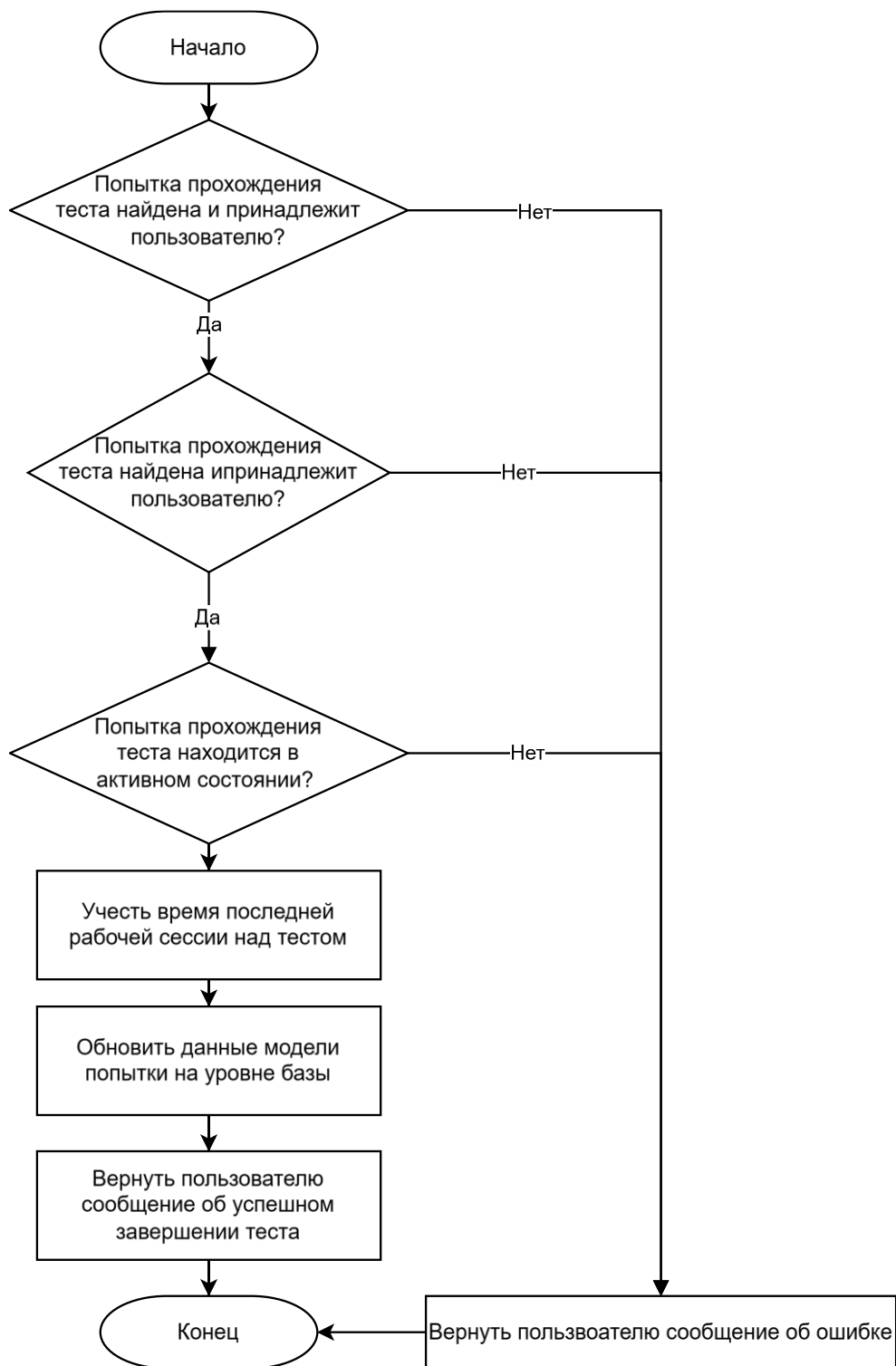


Рисунок 3.6 — Схема алгоритма завершения попытки тестирования на стороне API

10 Просмотр результатов и детальный разбор

Участники: ученик.

Шаги:

1 Ученик открывает экран результатов последней попытки или из истории.

2 Клиент запрашивает у сервера: общий балл, время выполнения, список заданий с пометками «верно/ошибка», привязку к темам/категориям и доступные пояснения.

3 Отображается послевывборочный анализ (ошибочные темы, доля верных ответов по категориям, среднее время ответа).

11 История решений и фильтрация

Участники: ученик.

Шаги:

1 Ученик открывает раздел «История».

2 Клиент отправляет фильтры: предмет, тип теста (прошлый год/смешанный/пользовательский), период дат.

3 Сервер возвращает список попыток с метаданными; при раскрытии записи – детали.

4 Результат: пользователю доступна полная трассируемость подготовки.

12 Получение и применение рекомендаций

Участники: ученик.

Шаги:

1 После завершения попытки сервер пересчитывает профиль тем: доля ошибок, давность, частота.

2 Формируется упорядоченный список «рекомендуемых к повторению» тем.

3 Клиент запрашивает рекомендации (или получает уведомление) и отображает блок с приоритетами; при переходе – предложение запустить тренировку по выбранным темам (смешанный мини-тест).

13 Уведомления

Участники: ученик/преподаватель.

Шаги:

1 Сервер регистрирует токен push-сервиса клиента.

2 При назначении теста, появлении результата или новых рекомендаций сервер отправляет уведомление.

3 Пользователь открывает уведомление и попадает на соответствующий экран (тест/результат/рекомендации).

Ветвления: отключённые уведомления – клиент показывает пункт «Новые события» внутри приложения и счётчик непрочитанных.

14 Создание пользовательского теста

Участники: преподаватель.

Шаги:

- 1 Преподаватель авторизуется в веб-инструменте.
- 2 Открывает «Создание теста», выбирает предмет и добавляет задания: из банка (поиск/фильтр) и/или новые вопросы (форма с полями: формулировка, варианты/ответ, категория, сложность, пояснение).
- 3 Сохраняет тест как черновик; сервер присваивает идентификатор и версию.
- 4 При необходимости редактирует порядок, ограничение времени, допустимое число попыток.
- 5 Публикует тест; статус меняется на «опубликован».

15 Назначение теста ученикам

Участники: преподаватель.

Шаги:

- 1 Преподаватель открывает в мобильном приложении страницу «Мои тесты», выбирает опубликованный тест, выбирает опцию назначить и из контекстного окна выбирает учеников, которым будет доступен этот тест.
- 2 Подтверждает назначение; сервер создаёт задания к исполнению для указанных учеников и отправляет уведомления.
- 3 В мобильном приложении учеников тест появляется в разделе «Пользовательские».

Ветвления: попытка назначить тест не своему ученику – отказ; повторное назначение при политике «одна попытка» – предупреждение.

Детальная логика операции назначения теста, реализованная в методе AssignTestToStudent сервиса AssignmentService, представлена на рисунке 3.7, а также на чертеже ГУИР.253504-01 СА. Кроме того, исходный код соответствующего метода представлен в приложении Б. Алгоритм включает последовательные проверки существования теста и наличия связи «преподаватель–ученик» с возвратом соответствующих кодов ошибок, после чего создаёт запись назначения с указанием deadline и количества попыток.

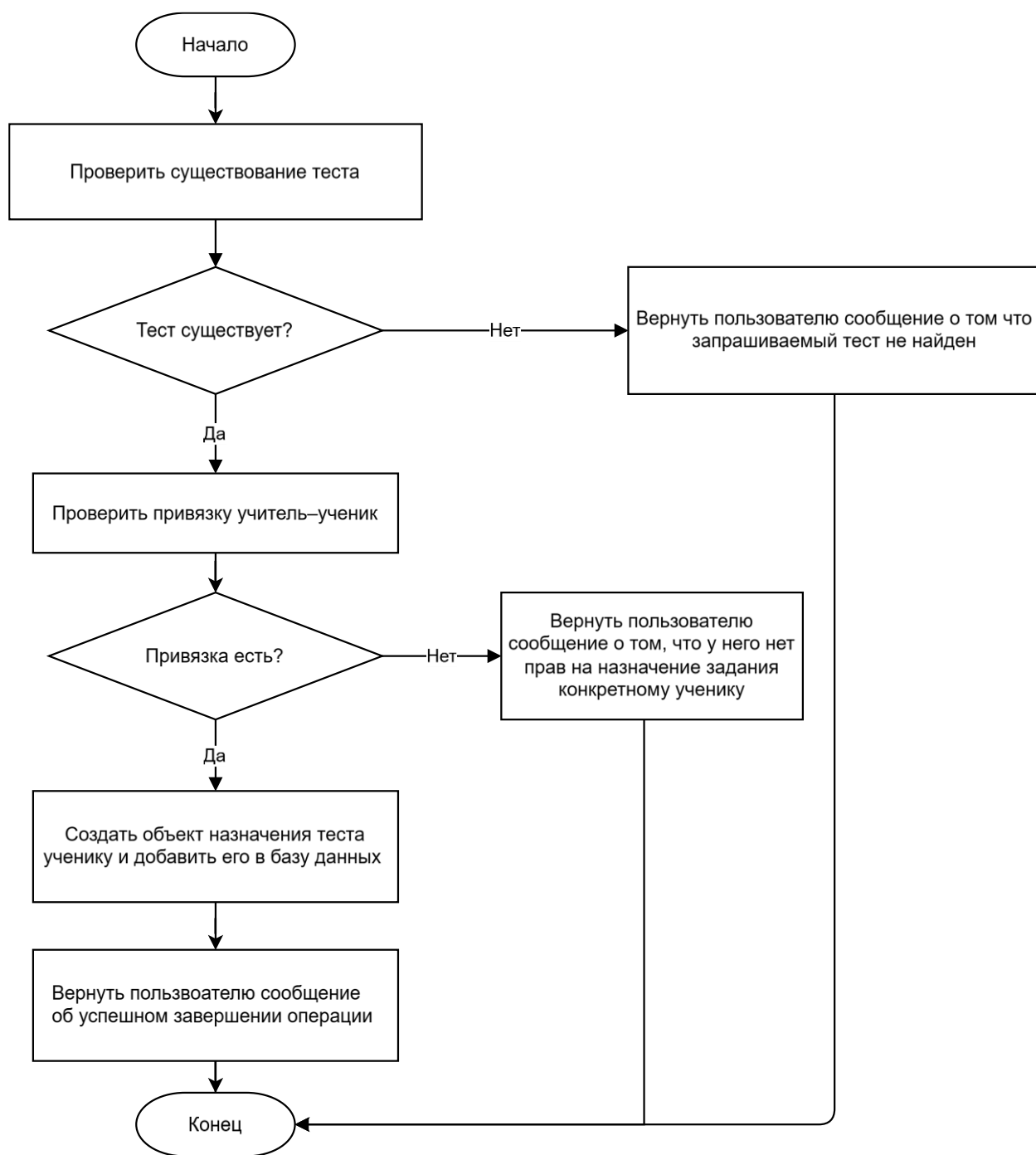


Рисунок 3.7 — Схема алгоритма назначения теста ученику

16 Просмотр результатов учеников преподавателем

Участники: преподаватель.

Шаги:

1 Преподаватель открывает раздел «Ученики».

2 Клиент запрашивает у сервера список прикрепленных учеников с основными метриками (последний результат, активность).

3 При выборе ученика запрашивается детальная статистика: история попыток, распределение ошибок по темам, динамика за период.

4 Преподаватель просматривает результаты по конкретному тесту и может рекомендовать темы к повторению (установка флага рекомендаций – опционально).

17 Управление банком заданий через Django Admin

Участники: администратор/разработчик.

Шаги:

1 Авторизация в Django Admin (отдельные доступы, вне пользовательского потока).

2 Просмотр и редактирование сущностей: задания, тесты прошлых лет, категории/темы, пользователи.

3 Сохранение изменений; ORM Django фиксирует записи в PostgreSQL.

Ветвления: конфликты уникальности; несоответствие схемы – система отклоняет строки с отчётом.

18 Обработка ошибок генерации/прохождения теста

Участники: ученик, сервер.

Случаи:

1 Недостаточно заданий под параметры смешанного теста – сервер возвращает допустимые пределы и список дефицитных категорий; клиент предлагает изменить параметры.

2 Повторная отправка результатов – сервер использует идемпотентность по идентификатору попытки и возвращает уже сохранённый результат.

3 Потеря сети при сдаче – клиент помещает результат в очередь синхронизации, показывает статус «Ожидание отправки», при восстановлении сети выполняет отправку автоматически.

19 Контроль доступа и защитные проверки

Участники: ученик/преподаватель, сервер.

Случаи:

1 Попытка доступа к чужим результатам – сервер возвращает 403 Forbidden.

2 Использование просроченного JWT – сервер возвращает 401 Unauthorized; клиент предлагает повторный вход.

3 Попытка назначения теста пользователю, не привязанному к преподавателю – отказ с объяснением причины.

20 Удаление привязки «преподаватель – ученик»

Участники: преподаватель или ученик.

Шаги:

1 Инициатор выбирает действие «Отвязать».

2 Сервер проверяет права, помечает связь неактивной; новые назначения невозможны.

3 История и результаты остаются в системе (для отчётности).

21 Сброс пароля

Участники: ученик/преподаватель.

Шаги:

1 Пользователь указывает e-mail на экране «Забыли пароль».

2 Сервер отправляет письмо со ссылкой на сброс; после перехода и задания нового пароля – инвалидирует предыдущие JWT.

3 Клиент выполняет новый вход.

22 Просмотр назначенных тестов и статуса выполнения

Участники: ученик.

Шаги:

1 Пользователь открывает раздел «Пользовательские».

2 Запрос статусов: «не начат», «в процессе», «завершён», дата назначения.

3 Переход к выполнению или просмотр результата, если завершён.

23 Отзыв/изменение опубликованного теста преподавателем

Участники: преподаватель.

Шаги:

1 Преподаватель в веб-инструменте открывает тест и меняет статус на «отозван».

2 Сервер прекращает выдачу теста в «Пользовательские» для не начавших; уже начатые попытки могут завершиться в соответствии с текущей политикой.

3 Ученики получают уведомление об изменении.

Для формализации функциональной декомпозиции программного комплекса на верхнем уровне применено моделирование в нотации IDEF0. На рисунке 3.8 приведена контекстная диаграмма IDEF0-A0 «Обеспечивать подготовку к централизованному тестированию». Она определяет основной бизнес-процесс системы, его входы (запросы пользователя), выходы (результаты и рекомендации), а также механизмы исполнения — мобильное приложение на Android/Kotlin, серверную часть на ASP.NET Core и базу данных PostgreSQL.

Декомпозиция контекстного процесса на функциональные блоки первого уровня представлена на диаграммах IDEF0-A1, IDEF0-A2 и IDEF0-

А3. На рисунке 3.9 изображена диаграмма IDEF0-A1 «Обработать пользовательский запрос», которая детализирует этапы аутентификации, получения списка тестов, валидации запроса и подготовки данных для последующего тестирования. В качестве механизмов задействованы контроллер аутентификации AuthController, HTTP-клиент ApiClient и репозиторий тестов TestRepository.

Диаграмма IDEF0-A2 «Выполнить тестирование», приведённая на рисунке 3.10, а также на чертеже ГУИР.253504-04 СРС, раскрывает внутреннюю структуру процесса прохождения теста. Она включает подпроцессы: начало попытки тестирования, обработку ответов пользователя, проверку правильности ответов и завершение попытки с сохранением результатов. Механизмами выступают сервис StudentAttemptService и репозитории AttemptRepository, UserAnswerRepository.

На рисунке 3.11, а также на чертеже ГУИР.253504-04 СРС представлена диаграмма IDEF0-A3 «Сформировать аналитику и рекомендации». Данный блок описывает завершающий этап обработки результатов: расчёт статистики, анализ допущенных ошибок, формирование персонализированных рекомендаций по темам и подготовку итогового отчёта.

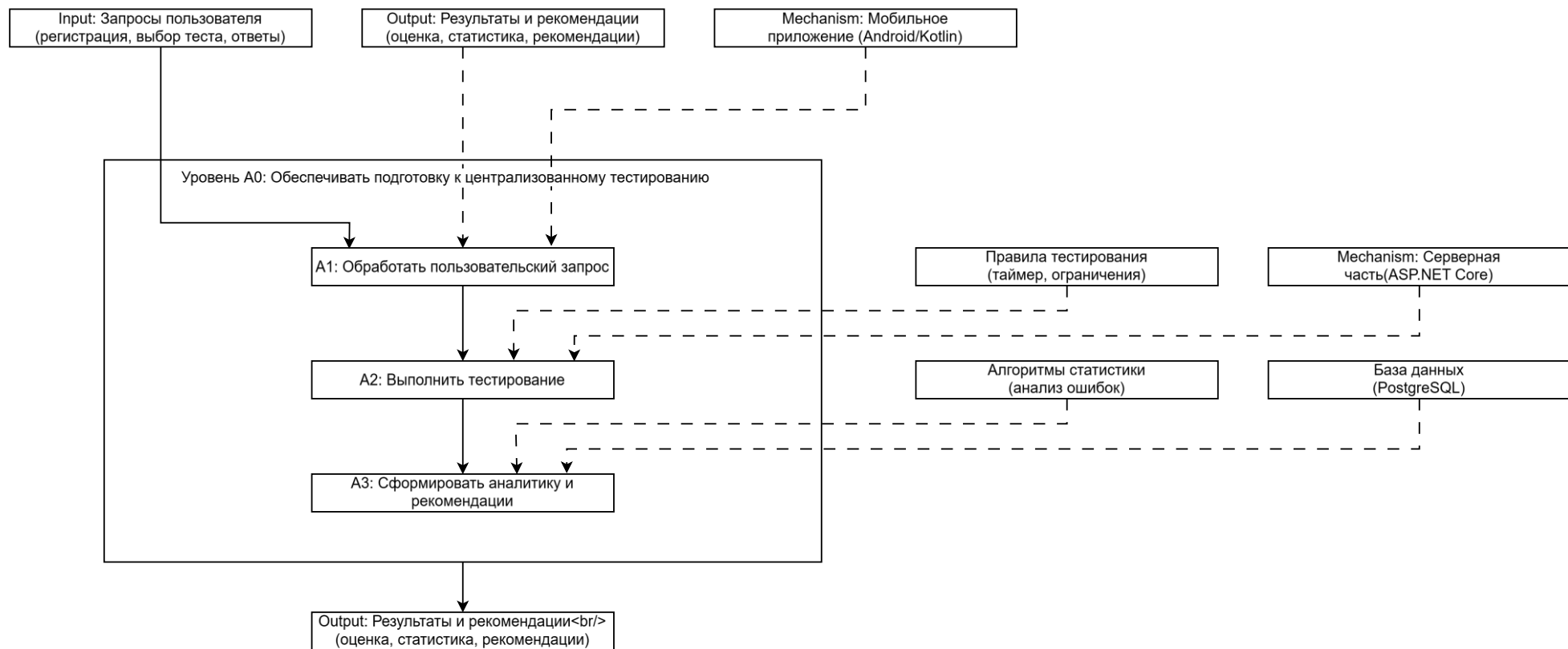


Рисунок 3.8 — Контекстная диаграмма IDEF0-A0 «Обеспечивать подготовку к централизованному тестированию»

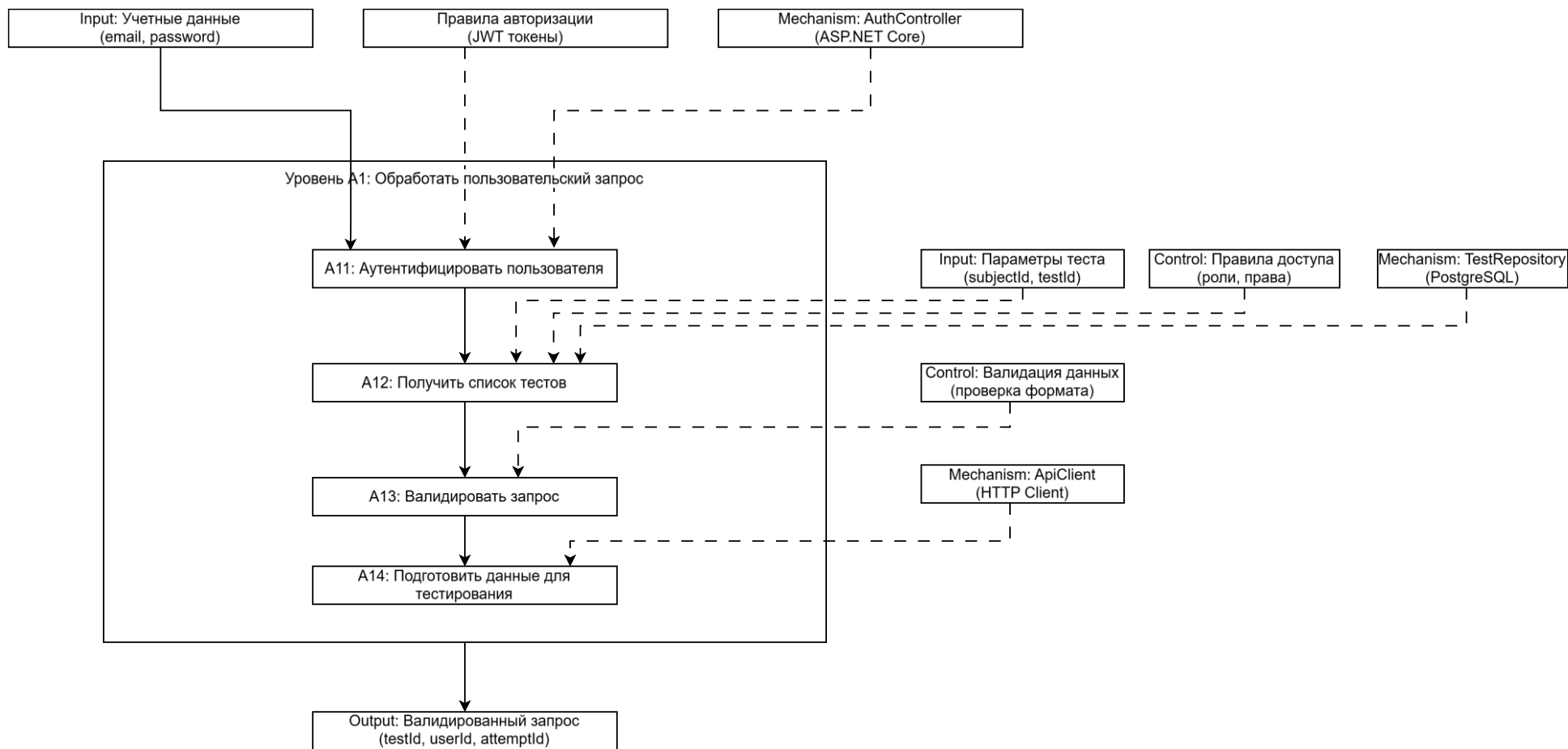


Рисунок 3.9 — Диаграмма IDEF0-A1 «Обработать пользовательский запрос»

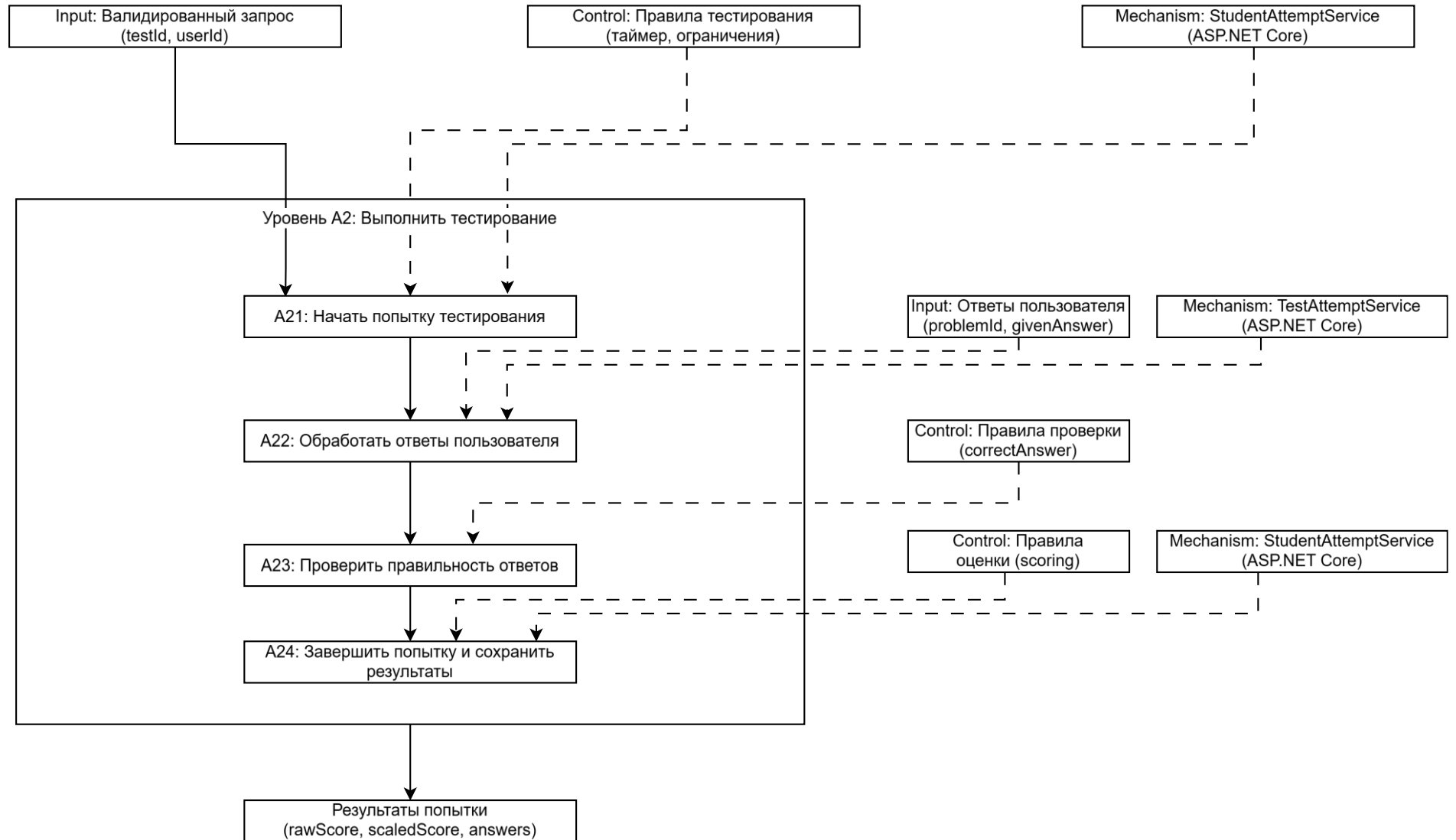


Рисунок 3.10 — Диаграмма IDEF0-A2 «Выполнить тестирование»

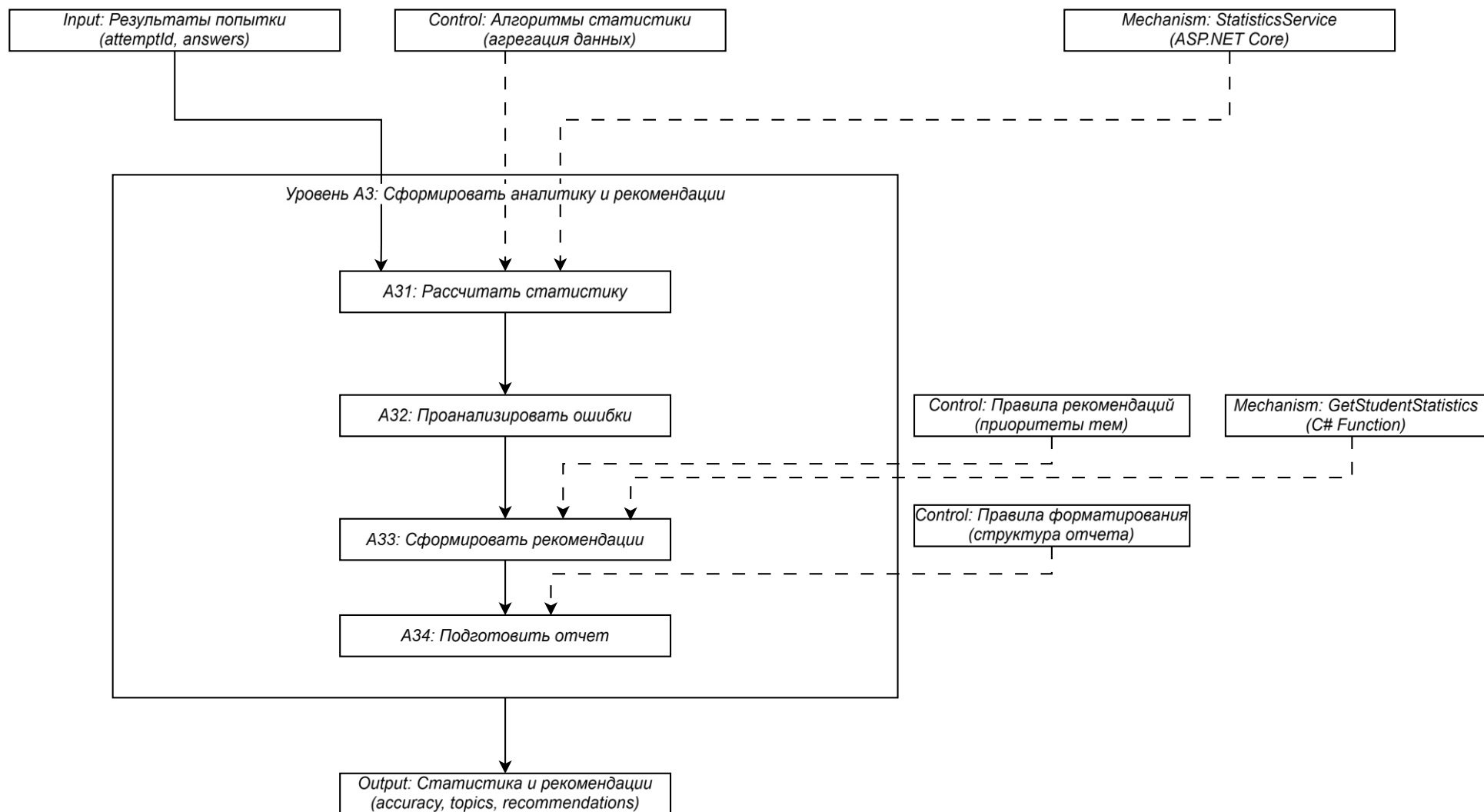


Рисунок 3.11 — Диаграмма IDEF0-A3 «Сформировать аналитику и рекомендации»

3.5 Обеспечение расширяемости и сопровождения

Система обеспечивает расширение функциональности без полной переработки существующих модулей. Предусмотрена возможность добавления новых предметов, категорий и типов заданий (например, с кратким ответом или на сопоставление). База данных и бизнес-логика поддерживают такое расширение без изменения существующей структуры и с сохранением совместимости с ранее введёнными данными. Интерфейсы мобильного приложения и серверная часть поддерживают обработку различных форматов заданий без модификации базового механизма тестирования.

Модель пользователей поддерживает расширение перечня ролей без изменения основной логики аутентификации и авторизации. Подсистемы статистики и уведомлений реализованы так, чтобы позволять добавлять новые отчёты и типы событий без изменения существующих функций.

Для сопровождения программного комплекса предусмотрено регулярное обновление банка заданий и тестов через административную панель. Администратор может редактировать, удалять и добавлять задания без вмешательства в код или прямого доступа к базе данных. Также предусмотрено администрирование пользователей: изменение статуса, сброс пароля, блокировка доступа. Серверная часть фиксирует в журналах все ошибки, исключения и критические события. Логирование централизовано и обеспечивает анализ при эксплуатации системы. Реализовано регулярное резервное копирование базы данных и её восстановление при сбое.

В данном разделе спроектирована архитектура программного комплекса на основе многослойной клиент-серверной модели с разделением на уровни представления, прикладной логики и хранения данных. Разработана модульная структура системы, включающая модули аутентификации, управления пользователями, тестами, прохождения тестов, статистики и уведомлений. Представлены диаграммы развёртывания (рисунок 3.1), модульной структуры (рисунок 3.2), последовательности для аутентификации (рисунок 3.3) и прохождения теста (рисунок 3.4). Описаны 23 сценария использования системы. Разработаны схемы алгоритмов генерации смешанного теста (рисунок 3.5), завершения попытки тестирования (рисунок 3.6) и назначения теста ученику (рисунок 3.7). Выполнено функциональное моделирование в нотации IDEF0: контекстная диаграмма A-0 (рисунок 3.8) и декомпозиции A1, A2, A3 (рисунки 3.9–3.11). Предусмотрены механизмы расширяемости и сопровождения. В результате сформирована завершённая архитектура, обеспечивающая модульность, масштабируемость и удобство дальнейшей разработки.

4 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА БАЗЫ ДАННЫХ

4.1 Обоснование выбора СУБД

Для хранения данных программного комплекса используется реляционная система управления базами данных PostgreSQL.

Выбор PostgreSQL обусловлен следующими факторами:

- система поддерживает транзакционную модель с соблюдением свойств ACID, что обеспечивает целостность и надёжность данных при одновременной работе большого числа пользователей;
- СУБД является свободно распространяемой и кроссплатформенной, что исключает необходимость приобретения лицензий и упрощает развёртывание на различных типах серверов;
- в системе поддерживается работа со сложными структурами данных (JSONB, массивы, полнотекстовый поиск), что позволяет хранить не только строго структурированные записи, но и дополнительные метаданные тестов и заданий [12];
- PostgreSQL обладает развитым инструментарием для проектирования и оптимизации запросов, включая поддержку индексов различных типов, оптимизатор запросов и механизмы репликации [13];
- СУБД имеет встроенные средства резервного копирования и восстановления, а также поддерживает масштабирование с помощью репликации и шардинга, что соответствует требованиям к отказоустойчивости и дальнейшему развитию системы.
- Для взаимодействия серверной части на ASP.NET Core с PostgreSQL используется библиотека Entity Framework Core. Это обеспечит унифицированный доступ к данным, поддержку миграций и независимость прикладного кода от конкретной реализации хранилища.
- Использование PostgreSQL позволяет организовать централизованное хранение информации о пользователях, ролях, тестах, заданиях, результатах прохождения и уведомлениях в единой базе данных с обеспечением безопасности и отказоустойчивости.

4.2 Логическая модель данных

Логическая модель данных программного комплекса обеспечивает хранение информации о пользователях, их ролях и связях, тестах и заданиях, попытках прохождения, результатах, статистике и уведомлениях.

Перед детальным проектированием логической структуры базы данных целесообразно рассмотреть потоки данных, циркулирующие между пользователями, процессами обработки и хранилищем. Диаграмма потоков данных верхнего уровня, на которой выделены внешние сущности, основные процессы преобразования информации и хранилище базы данных, приведена на рисунке 4.1.

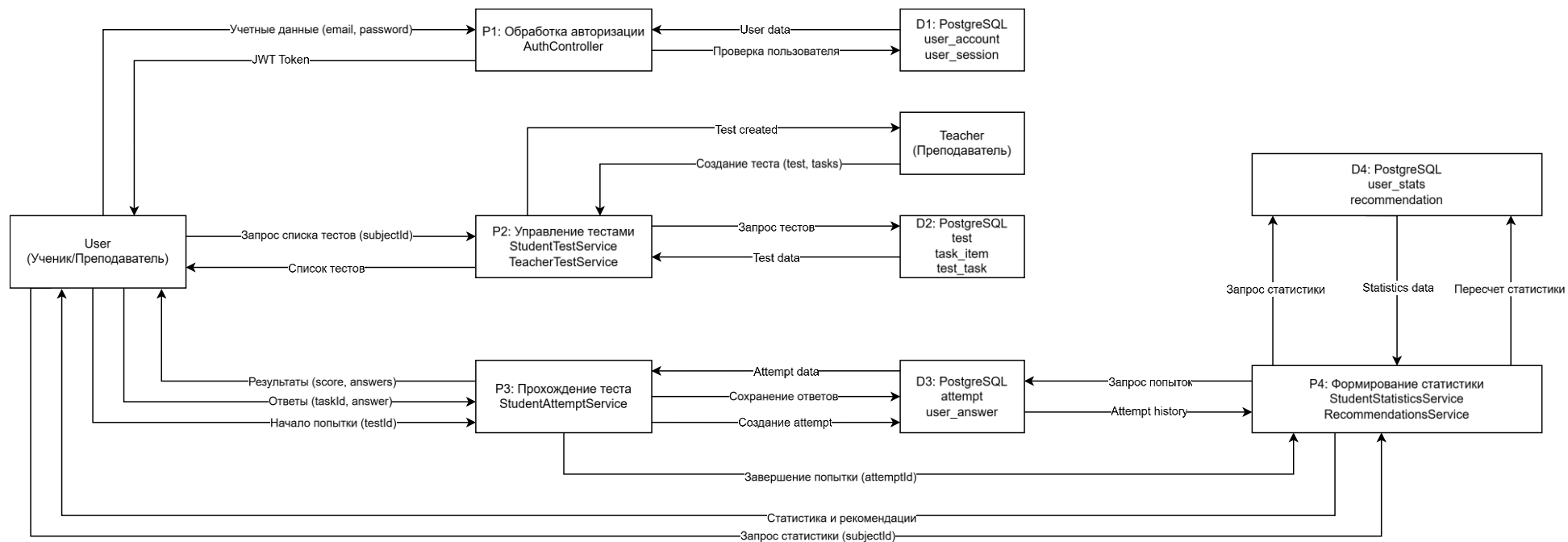


Рисунок 4.1 — Диаграмма потоков данных программного комплекса (DFD)

4.2.1 Сущность «Пользователь» (User).

Сущность предназначена для хранения информации о зарегистрированных участниках системы.

Основные атрибуты:

- уникальный идентификатор;
- имя пользователя;
- адрес электронной почты (уникальный);
- хэш пароля;
- роль (smallint: Student, Teacher, Admin);
- флаг подтверждения электронной почты;
- дата последнего входа;
- идентификатор аватара (ссылка на Image);
- флаг удаления учётной записи;
- дата создания и последнего обновления.

4.2.2 Сущность «Связь преподаватель–ученик» (TeacherStudent).

Сущность фиксирует установленную связь между преподавателем и учеником после подтверждения приглашения.

Основные атрибуты:

- идентификатор связи;
- идентификатор преподавателя (User);
- идентификатор ученика (User);
- статус (Active, Blocked);
- флаг удаления;
- дата создания и обновления.

4.2.3 Сущность «Код приглашения» (InvitationCode).

Используется преподавателем для приглашения учеников.

Основные атрибуты:

- идентификатор кода;
- идентификатор преподавателя;
- строковое значение кода;
- количество оставшихся использований;
- срок действия;
- флаг отзыва;
- дата создания и обновления.

4.2.4 Сущность «Предмет» (Subject).

Нормализованное представление учебного предмета.

Основные атрибуты:

- идентификатор предмета;

- наименование предмета;
- флаг удаления;
- дата создания и обновления.

4.2.5 Сущность «Раздел» (Section).

Группировка тем внутри предмета.

Основные атрибуты:

- идентификатор раздела;
- идентификатор предмета;
- наименование раздела;
- флаг удаления;
- дата создания и обновления.

4.2.6 Сущность «Тема» (Topic).

Тематический блок, к которому относятся задания.

Основные атрибуты:

- идентификатор темы;
- идентификатор раздела;
- наименование темы;
- флаг удаления;
- дата создания и обновления.

4.2.7 Сущность «Задание» (Problem).

Содержит метаинформацию о задании; сами формулировки и ответы вынесены в версии.

Основные атрибуты:

- идентификатор задания;
- идентификатор темы;
- идентификатор автора (преподаватель);
- флаги публичности, опубликованности и удаления.

4.2.8 Сущность «Версия задания» (ProblemVersion).

Обеспечивает историю изменений задания; активна всегда одна версия.

Основные атрибуты:

- идентификатор версии;
- идентификатор задания;
- тип (одиночный выбор, множественный выбор, открытый);
- сложность (VeryEasy ... VeryHard);
- текст условия (JSONB);
- правильный ответ;
- пояснение (JSONB);

- флаг активности;
- дата создания.

4.2.9 Сущность «Тест» (Test).

Тестовый набор, создаваемый преподавателем или сгенерированный системой.

Основные атрибуты:

- идентификатор теста;
- название;
- идентификатор предмета;
- идентификатор автора;
- тип (State, Custom, Mixed);
- флаги: тренировочный, опубликован, публичный, удалён;
- длительность (секунды);
- количество попыток;
- дата создания и обновления.

4.2.10 Сущность «Состав теста» (TestProblem).

Связывает тест с конкретными заданиями и определяет их порядок.

Основные атрибуты:

- идентификатор записи;
- идентификатор теста;
- идентификатор задания;
- код (например, A1, B2);
- дата создания.

4.2.11 Сущность «Группа» (Group).

Позволяет преподавателю объединять учеников для массовых назначений.

Основные атрибуты:

- идентификатор группы;
- идентификатор преподавателя;
- идентификатор предмета;
- наименование;
- флаг удаления;
- дата создания и обновления.

4.2.12 Сущность «Назначение студенту» (StudentAssignment).

Индивидуальное назначение теста ученику.

Основные атрибуты:

- идентификатор назначения;

- идентификаторы студента, преподавателя, теста;
- идентификатор группового назначения (если создано через группу);
- срок сдачи;
- оставшееся число попыток;
- дата создания и обновления.

4.2.13 Сущность «Назначение группе» (GroupAssignment).

Групповое назначение теста; на его основе создаются отдельные StudentAssignment.

Основные атрибуты:

- идентификатор;
- идентификаторы преподавателя, группы, теста;
- срок сдачи;
- количество попыток по умолчанию;
- дата создания и обновления.

4.2.14 Сущность «Попытка» (TestAttempt).

Фиксирует факт прохождения теста конкретным учеником.

Основные атрибуты:

- идентификатор попытки;
- идентификаторы теста и студента;
- статус (InProgress, Paused, Completed, Canceled);
- продолжительность (секунды);
- сырой балл (процент);
- дата начала и последнего возобновления.

4.2.15 Сущность «Ответ пользователя» (UserAnswer).

Сохраняет ответ ученика на конкретную версию задания в рамках попытки.

Основные атрибуты:

- идентификатор ответа;
- идентификаторы попытки и версии задания;
- текст ответа;
- флаг правильности;
- дата создания.

4.2.16 Сущность «Избранное» (FavoriteProblem / FavoriteTest).

Позволяет ученику отмечать понравившиеся задания и тесты (связь многие-ко-многим без дополнительных атрибутов).

4.2.17 Сущность «Уведомление» (Notification).

Хранит сообщения, отправляемые пользователям.

Основные атрибуты:

- идентификатор уведомления;
- идентификатор получателя;
- уровень приоритета;
- текст (JSON payload);
- флаг прочтения;
- дата создания.

Связи между описанными сущностями выстроены следующим образом:

- пользователь имеет одну роль (представлена кодом);
- преподаватель связан с учениками через TeacherStudent (многие-ко-многим);
- одна связь TeacherStudent может порождаться одним кодом приглашения;
- предмет содержит несколько разделов, раздел — несколько тем;
- задание принадлежит одной теме и одному автору-преподавателю;
- каждая версия задания ссылается на задание;
- тест создаётся преподавателем, относится к одному предмету;
- состав теста (TestProblem) связывает тест и задания (многие-ко-многим);
- попытка (TestAttempt) привязана к тесту и студенту;
- ответ (UserAnswer) связывает попытку с конкретной версией задания;
- назначения (StudentAssignment и GroupAssignment) связывают тест, преподавателя и ученика/группу;
- группа (Group) включает учеников через промежуточную таблицу student_group_student;
- избранное реализовано отдельными связками многие-ко-многим между пользователем и задачами/тестами;
- уведомление направлено одному пользователю-получателю.

Совокупность описанных сущностей, их атрибутов и связей в графическом виде представлена на ER-диаграмме, отражающей логическую модель данных программного комплекса, на рисунке 4.2.

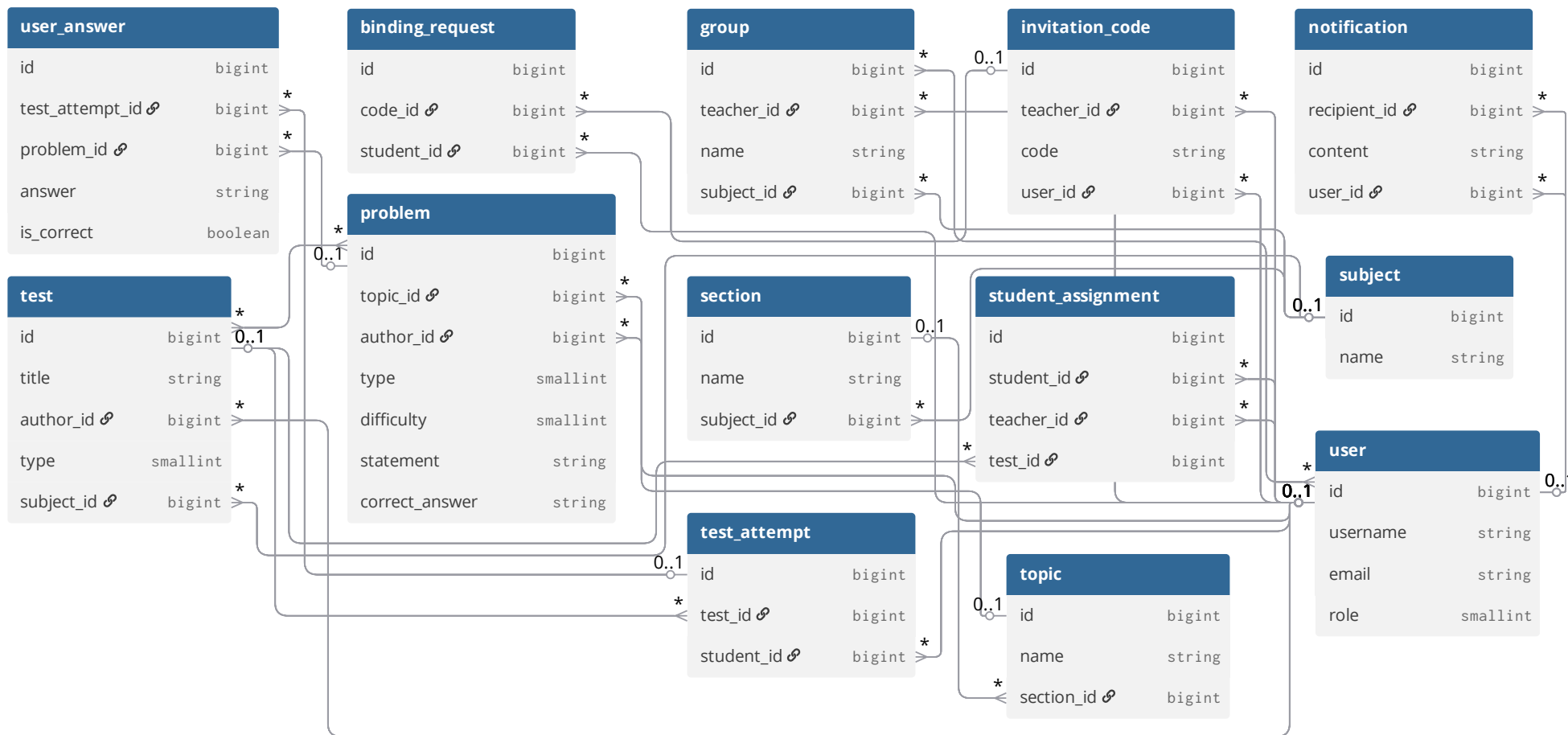


Рисунок 4.2 — Entity–relationship диаграмма программного средства

4.3 Физическая модель данных

Физическая модель определяет состав таблиц, типы данных, ключи и ограничения, а также индексацию и правила ссылочной целостности в СУБД PostgreSQL. Ниже приведены требования к структуре базы. Формулировки даны в нормативной форме без SQL-кода, но с указанием рекомендуемых типов PostgreSQL.

4.3.1 Общие требования и соглашения

1 Идентификаторы записей имеют тип `bigint` и генерируются автоматически (`GENERATED BY DEFAULT AS IDENTITY`).

2 Для временных меток используется тип `timestampz`, все значения хранятся в UTC.

3 Текстовые поля: короткие справочные значения – `varchar(n)` с ограничением длины, длинные тексты – `text`, структурированные данные – `jsonb`.

4 Логические флаги – `boolean NOT NULL` со значениями по умолчанию.

5 Перечислимые атрибуты (роль пользователя, тип задания, статус попытки и т.п.) хранятся как `smallint` в соответствии с подходом `code first`.

6 Внешние ключи заданы с явным указанием правил удаления и обновления: для справочных данных – `ON DELETE RESTRICT`, для зависимых записей – `ON DELETE CASCADE` либо `ON DELETE SET NULL`.

7 На все внешние ключи и поля, используемые в условиях `WHERE`, созданы B-Tree индексы.

4.3.2 Структура моделей базы данных

1 Таблица «Пользователь» (`public.user`).

Назначение: зарегистрированные пользователи системы.

Колонки:

- `id bigint` (PK, автоинкремент);
- `username varchar(100) NOT NULL`;
- `password_hash char(128) NOT NULL`;
- `email varchar(254) UNIQUE NOT NULL`;
- `role smallint NOT NULL` (`Student=1`, `Teacher=2`, `Admin=3`);
- `is_email_verified boolean NOT NULL DEFAULT false`;
- `last_login_at timestampz`;
- `avatar_id bigint (FK → image(id) ON DELETE RESTRICT)`;
- `is_deleted boolean NOT NULL DEFAULT false`;
- `created_at timestampz NOT NULL DEFAULT now()`;
- `last_update_at timestampz NOT NULL`.

Индексы: уникальный по `email`, неуникальные по `username` и `avatar_id`.

2 Таблица «Изображение» (`public.image`).

Назначение: метаданные загруженных изображений (аватары и пр.).

Колонки:

- id bigint (PK, автоинкремент);
- object_key text UNIQUE NOT NULL;
- bucket text NOT NULL;
- owner_id bigint NOT NULL (FK → user(id) ON DELETE RESTRICT);
- content_type text NOT NULL;
- size bigint NOT NULL;
- is_deleted boolean NOT NULL DEFAULT false;
- created_at timestamptz NOT NULL DEFAULT now();

3 Таблица «Код приглашения» (public.invitation_code).

Назначение: коды для привязки учеников к преподавателю.

Колонки:

- id bigint (PK);
- teacher_id bigint NOT NULL (FK → user(id) ON DELETE CASCADE);
- code varchar(36) NOT NULL;
- uses_left smallint (CHECK uses_left >= 0);
- expired_at timestamptz;
- is_revoked boolean NOT NULL DEFAULT false;
- created_at timestamptz NOT NULL DEFAULT now();
- last_update_at timestamptz NOT NULL.

Индексы: по teacher_id, code.

4 Таблица «Связь преподаватель–ученик» (public.teacher_student).

Назначение: подтверждённые связи.

Колонки:

- id bigint (PK);
- teacher_id bigint NOT NULL (FK → user(id) ON DELETE RESTRICT);
- student_id bigint NOT NULL (FK → user(id) ON DELETE RESTRICT);
- status smallint NOT NULL (Active=1, Blocked=2);
- is_deleted boolean NOT NULL DEFAULT false;
- created_at timestamptz NOT NULL DEFAULT now();
- last_update_at timestamptz NOT NULL.

Уникальный индекс: (teacher_id, student_id).

5 Таблица «Запрос на привязку» (public.binding_request).

Назначение: ожидающие подтверждения запросы от учеников.

Колонки:

- id bigint (PK);
- code_id bigint NOT NULL (FK → invitation_code(id) ON DELETE RESTRICT);
- student_id bigint NOT NULL (FK → user(id) ON DELETE RESTRICT);
- created_at timestamptz NOT NULL DEFAULT now();
- last_update_at timestamptz NOT NULL.

6 Таблица «Предмет» (public.subject).

Назначение: учебные предметы.

Колонки:

- id bigint (PK);
- name varchar(200) NOT NULL;
- is_deleted boolean NOT NULL DEFAULT false;
- created_at timestamptz NOT NULL DEFAULT now();
- last_update_at timestamptz NOT NULL.

7 Таблица «Раздел» (public.section).

Назначение: разделы внутри предмета.

Колонки:

- id bigint (PK);
- name varchar(200) NOT NULL;
- subject_id bigint NOT NULL (FK → subject(id) ON DELETE RESTRICT);
- is_deleted boolean NOT NULL DEFAULT false;
- created_at timestamptz NOT NULL DEFAULT now();
- last_update_at timestamptz NOT NULL.

8 Таблица «Тема» (public.topic).

Назначение: темы внутри раздела.

Колонки:

- id bigint (PK);
- name varchar(200) NOT NULL;
- section_id bigint NOT NULL (FK → section(id) ON DELETE RESTRICT);
- is_deleted boolean NOT NULL DEFAULT false;
- created_at timestamptz NOT NULL DEFAULT now();
- last_update_at timestamptz NOT NULL.

9 Таблица «Задание» (public.problem).

Назначение: метаданные задания.

Колонки:

- id bigint (PK);
- topic_id bigint NOT NULL (FK → topic(id) ON DELETE RESTRICT);
- author_id bigint NOT NULL (FK → user(id) ON DELETE RESTRICT);
- is_deleted boolean NOT NULL DEFAULT false;
- is_public boolean NOT NULL DEFAULT false;
- is_published boolean NOT NULL DEFAULT false.

10 Таблица «Версия задания» (public.problem_version).

Назначение: содержимое конкретной версии задания.

Колонки:

- id bigint (PK);
- problem_id bigint NOT NULL (FK → problem(id) ON DELETE RESTRICT);
- type smallint NOT NULL;
- difficulty smallint NOT NULL;
- statement jsonb NOT NULL;
- correct_answer varchar(32) NOT NULL;
- explanation jsonb NOT NULL DEFAULT '{}';
- is_active boolean NOT NULL DEFAULT false;
- created_at timestampz NOT NULL DEFAULT now().

11 Таблица «Тест» (public.test).

Назначение: тестовые наборы.

Колонки:

- id bigint (PK);
- title varchar(200) NOT NULL;
- subject_id bigint NOT NULL (FK → subject(id) ON DELETE RESTRICT);
- author_id bigint NOT NULL (FK → user(id) ON DELETE RESTRICT);
- type smallint NOT NULL (State=1, Custom=2, Mixed=3);
- is_training boolean NOT NULL DEFAULT false;
- is_published boolean NOT NULL DEFAULT false;
- is_public boolean NOT NULL DEFAULT false;
- is_deleted boolean NOT NULL DEFAULT false;
- duration integer NOT NULL DEFAULT 0;
- attempts_count integer NOT NULL DEFAULT 0;
- created_at timestampz NOT NULL DEFAULT now();
- last_update_at timestampz NOT NULL.

12 Таблица «Состав теста» (public.test_problem).

Назначение: связь теста с заданиями.

Колонки:

- id bigint (PK);
 - test_id bigint NOT NULL (FK → test(id) ON DELETE RESTRICT);
 - problem_id bigint NOT NULL (FK → problem(id) ON DELETE RESTRICT);
 - code varchar(3) NOT NULL;
 - created_at timestampz NOT NULL DEFAULT now().
- Уникальный индекс: (test_id, code).

13 Таблица «Группа» (public.group).

Назначение: учебные группы преподавателя.

Колонки:

- id bigint (PK);
- teacher_id bigint NOT NULL (FK → user(id) ON DELETE CASCADE);
- subject_id bigint NOT NULL (FK → subject(id) ON DELETE RESTRICT);
- name varchar(200) NOT NULL;
- is_deleted boolean NOT NULL DEFAULT false;
- created_at timestamptz NOT NULL DEFAULT now();
- last_update_at timestamptz NOT NULL.

Уникальный индекс: (teacher_id, name).

14 Таблица состава группы (public.student_group_student).

Назначение: многие-ко-многим между группами и студентами.

Колонки:

- group_id bigint NOT NULL (FK → group(id) ON DELETE CASCADE);
- student_id bigint NOT NULL (FK → user(id) ON DELETE CASCADE);

Первичный ключ: (group_id, student_id).

15 Таблица «Назначение группе» (public.group_assignment).

Назначение: назначение теста группе учеников.

Колонки:

- id bigint (PK);
- teacher_id bigint NOT NULL (FK → user(id) ON DELETE RESTRICT);
- group_id bigint (FK → group(id) ON DELETE SET NULL);
- test_id bigint NOT NULL (FK → test(id) ON DELETE RESTRICT);
- expired_at timestamptz NOT NULL;
- default_attempts_allowed smallint (CHECK >= 0);
- created_at timestamptz NOT NULL DEFAULT now();
- last_update_at timestamptz NOT NULL.

16 Таблица «Назначение студенту» (public.student_assignment).

Назначение: индивидуальное назначение теста.

Колонки:

- id bigint (PK);
- student_id bigint NOT NULL (FK → user(id) ON DELETE RESTRICT);
- teacher_id bigint NOT NULL (FK → user(id) ON DELETE RESTRICT);
- group_assignment_id bigint (FK → group_assignment(id) ON DELETE RESTRICT);
- test_id bigint NOT NULL (FK → test(id) ON DELETE RESTRICT);
- expired_at timestamptz NOT NULL;
- attempts_left smallint (CHECK >= 0);
- created_at timestamptz NOT NULL DEFAULT now();
- last_update_at timestamptz NOT NULL.

17 Таблица «Попытка» (public.test_attempt).

Назначение: попытки прохождения тестов.

Колонки:

- id bigint (PK);
- test_id bigint NOT NULL (FK → test(id) ON DELETE RESTRICT);
- student_id bigint NOT NULL (FK → user(id) ON DELETE RESTRICT);
- status smallint NOT NULL (InProgress=1, Paused=2, Completed=3, Canceled=4);
- duration integer NOT NULL;
- raw_score smallint;
- created_at timestampz NOT NULL DEFAULT now();
- last_resumed_at timestampz NOT NULL DEFAULT now().

18 Таблица «Ответ пользователя» (public.user_answer).

Назначение: ответ на конкретную версию задания.

Колонки:

- id bigint (PK);
- test_attempt_id bigint NOT NULL (FK → test_attempt(id) ON DELETE RESTRICT);
- problem_version_id bigint NOT NULL (FK → problem_version(id) ON DELETE RESTRICT);
- answer varchar(32) NOT NULL;
- is_correct boolean NOT NULL;
- created_at timestampz NOT NULL DEFAULT now().

19 Таблица «Сессия пользователя» (public.user_session).

Назначение: сессии аутентифицированных пользователей.

Колонки:

- id bigint (PK);
- user_id bigint NOT NULL (FK → user(id) ON DELETE RESTRICT);
- jti uuid UNIQUE NOT NULL;
- client_type smallint NOT NULL;
- ip_address inet;
- device_info jsonb;
- last_activity_at timestampz NOT NULL;
- created_at timestampz NOT NULL DEFAULT now();
- last_update_at timestampz NOT NULL;
- revoked_at timestampz.

20 Таблица «Refresh-токен» (public.refresh_token).

Назначение: токены обновления для сессий.

Колонки:

- id bigint (PK);
- session_id bigint UNIQUE NOT NULL (FK → user_session(id) ON DELETE RESTRICT);

- token_hash char(128) UNIQUE NOT NULL;
- device_id varchar(255);
- expires_at timestampz NOT NULL;
- created_at timestampz NOT NULL DEFAULT now();
- revoked_at timestampz.

Уникальный индекс на session_id с фильтром WHERE revoked_at IS NULL.

21 Таблица «Токен сброса пароля» (public.password_reset_token).

Назначение: одноразовые токены для восстановления пароля.

Колонки:

- id bigint (PK);
- user_id bigint NOT NULL;
- token_hash char(128) UNIQUE NOT NULL;
- expires_at timestampz NOT NULL;
- used_at timestampz;
- attempt_left smallint NOT NULL DEFAULT 5.

22 Таблица «Токен верификации email» (public.email_verification_code).

Назначение: токены подтверждения адреса электронной почты.

Колонки:

- id bigint (PK);
- user_id bigint NOT NULL (FK → user(id) ON DELETE CASCADE);
- token_hash char(128) UNIQUE NOT NULL;
- expires_at timestampz NOT NULL;
- verified_at timestampz;
- attempt_left smallint NOT NULL DEFAULT 5.

Уникальный индекс на user_id с фильтром WHERE verified_at IS NULL.

23 Таблица «Уведомление» (public.notification).

Назначение: внутренние уведомления пользователей.

Колонки:

- id bigint (PK);
- recipient_id bigint NOT NULL (FK → user(id) ON DELETE CASCADE);
- priority_level smallint NOT NULL;
- payload text NOT NULL;
- is_seen boolean NOT NULL DEFAULT false;
- created_at timestampz NOT NULL DEFAULT now().

24 Таблица «Событие пользователя» (public.user_event).

Назначение: аудит действий пользователей.

Колонки:

- id bigint (PK);
- user_id bigint NOT NULL (FK → user(id) ON DELETE RESTRICT);

- event_type smallint NOT NULL;
- target_id bigint;
- ip_address inet;
- device_info jsonb;
- created_at timestampz NOT NULL DEFAULT now();

25 Таблица «Журнал аудита» (public.audit_log).

Назначение: низкоуровневый аудит изменений сущностей.

Колонки:

- id bigint (PK);
- entity_type varchar(32) NOT NULL;
- entity_id bigint NOT NULL;
- action smallint NOT NULL;
- changes jsonb NOT NULL DEFAULT '{}';
- timestamp timestampz NOT NULL.

26 Таблица «Избранные задания» (public.favorite_problem).

Назначение: связь многие-ко-многим между пользователем и заданиями.

Колонки:

- problem_id bigint NOT NULL (FK → problem(id) ON DELETE CASCADE);
 - student_id bigint NOT NULL (FK → user(id) ON DELETE CASCADE);
- Первичный ключ: (problem_id, student_id).

27 Таблица «Избранные тесты» (public.favorite_test).

Назначение: связь многие-ко-многим между пользователем и тестами.

Колонки:

- test_id bigint NOT NULL (FK → test(id) ON DELETE CASCADE);
 - student_id bigint NOT NULL (FK → user(id) ON DELETE CASCADE);
- Первичный ключ: (student_id, test_id).

4.3.3 Правила ссылочной целостности

Удаление записей регулируется следующими политиками внешних ключей:

- ON DELETE RESTRICT применяется для ключевых справочников и объектов, удаление которых недопустимо при наличии зависимых записей (например, пользователь не может быть удалён, пока на него ссылаются созданные им задания или попытки). Реальное удаление пользователя выполняется логически (is_deleted = true).

- ON DELETE CASCADE используется для сущностей, полностью теряющих смысл без родительской записи: состав группы, избранное, коды приглашений, уведомления, токены верификации.

- ON DELETE SET NULL задан для group_assignment.group_id, что

позволяет удалить группу, сохранив индивидуальные назначения студентам.

Во всех случаях жёсткое удаление записей из базы не применяется к основным бизнес-сущностям (пользователи, задания, тесты), вместо этого используется мягкое удаление (флаг `is_deleted`).

4.3.4 Индексация

Для обеспечения производительности созданы следующие индексы:

- уникальные: `user(email)`, `invitation_code(code)`, `image(object_key)`, `refresh_token(token_hash)`, `refresh_token(session_id) WHERE revoked_at IS NULL`, `user_session(jti)`, `email_verification_code(token_hash)`, `email_verification_code(user_id) WHERE verified_at IS NULL`, `password_reset_token(token_hash)`, `password_reset_token(user_id) WHERE used_at IS NULL`, `group(teacher_id, name)`, `test_problem(test_id, code)`, `teacher_student(teacher_id, student_id)`;

- неуникальные (B-Tree) для внешних ключей и частых фильтров: по всем FK, а также `test(title)`, `notification(recipient_id, is_seen)`, `test_attempt(test_id, student_id)`, `test_attempt(student_id)`, `audit_log(entity_type, entity_id)`, `user(username)`.

Составные индексы ускоряют запросы выборки уведомлений (порядок по `created_at DESC`) и истории попыток (порядок по `started_at DESC`, индекс создаётся автоматически по первичному ключу, но при необходимости дополняется отдельным индексом по `student_id` и времени).

В данном разделе выполнено проектирование базы данных программного комплекса. Обоснован выбор СУБД PostgreSQL (свободная, ACID-совместимая, поддержка JSONB, полнотекстового поиска, репликации, масштабирования). Построена логическая модель данных, включающая 27 сущностей: пользователь, роли, предметы, разделы, темы, задания и их версии, тесты и их состав, группы, назначения тестов (групповые и индивидуальные), попытки прохождения, ответы пользователей, уведомления, токены (сессий, `refresh`, сброса пароля, верификации), избранное, аудит и события. Разработана диаграмма потоков данных верхнего уровня (рисунок 4.1) и ER-диаграмма (рисунок 4.2). Физическая модель реализована в PostgreSQL с определением типов данных (`bigint`, `timestamptz`, `jsonb`, `char(128)` для хэшей), первичных и внешних ключей с правилами `ON DELETE RESTRICT/CASCADE/SET NULL`. Созданы уникальные и не уникальные B-Tree индексы для ускорения запросов по внешним ключам, `email`, кодам приглашений и составным полям. Реализовано мягкое удаление (`is_deleted`) для основных бизнес-сущностей. В результате спроектирована эффективная, нормализованная и масштабируемая структура базы данных, полностью соответствующая функциональным требованиям системы.

5 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

5.1 Выбор средств и инструментов разработки

Для реализации серверной части программного комплекса использована платформа ASP.NET Core. Она обеспечивает создание REST API, модульную архитектуру и высокую производительность при работе с запросами. Для доступа к базе данных должна применяться технология Entity Framework Core как объектно-реляционный маппер, позволяющий выполнять миграции схемы и работать с данными на уровне объектов.

Для разработки административного веб-инструмента преподавателя используется среда Django с системой шаблонов Django Templates [14]. Такой выбор обоснован наличием встроенной административной панели для обслуживания системы, а также развитым механизмом маршрутизации и форм. Это решение позволяет обеспечить веб-доступ к функциям формирования и редактирования тестов, а также управлению пользовательскими связями.

Клиентская часть для учеников и преподавателей реализована в виде мобильного приложения на языке Kotlin с использованием официальных библиотек Android. Для построения интерфейсов используется Jetpack Compose, обеспечивающий декларативный подход и модульность [15]. Дополнительно могут применяться Android Jetpack-компоненты для навигации, управления состоянием и работы с локальным хранилищем.

В качестве системы управления базами данных используется PostgreSQL. Эта СУБД является свободно распространяемой, поддерживает транзакции, индексацию и репликацию, что соответствует требованиям к надёжности и расширяемости [16].

При разработке используются современные системы контроля версий. Основной системой хранения исходного кода является Git, с размещением репозитория в сервисах GitHub. Для документирования API и взаимодействия компонентов используется спецификация Swagger, которая позволяет обеспечить единый формат описания интерфейсов и упростить разработку клиентских приложений.

Для сборки и управления зависимостями применяются стандартные средства: Gradle для мобильного приложения, NuGet для ASP.NET Core и pip/venv для Python-проектов. Это обеспечивает управляемость и воспроизводимость окружений.

Для тестирования используются встроенные средства каждой платформы: xUnit для .NET, pytest для Python и JUnit для Kotlin.

Для обеспечения удобства развертывания допускается использование контейнеризации с помощью Docker, что упростит настройку окружения и переносимость между серверами.

5.2 Структуре серверной части

Серверная часть реализована на платформе ASP.NET Core и обеспечивает выполнение бизнес-логики программного комплекса. Архитектура сервера многослойна, с выделением следующих уровней: уровень представления API, уровень прикладной логики и уровень доступа к данным.

На уровне представления API реализованы контроллеры для следующих групп функций:

- аутентификация и авторизация пользователей;
- управление пользователями и ролями;
- управление тестами прошлых лет, пользовательскими тестами и смешанными тестами;
- управление заданиями и их атрибутами;
- управление связями преподаватель—ученик и назначениями тестов;
- обработка попыток и ответов пользователей;
- предоставление статистики и аналитики;
- генерация и доставка уведомлений.

На уровне прикладной логики реализованы сервисы, обеспечивающие работу основных функциональных модулей:

- сервис пользователей (регистрация, вход, сброс пароля, изменение профиля);
- сервис тестов (создание, редактирование, удаление, генерация смешанных тестов);
- сервис заданий (управление содержимым банка заданий и их категориями);
- сервис попыток (фиксация старта, проверка ответов, подсчёт результатов, завершение попытки);
- сервис статистики (агрегация данных, формирование аналитики по темам и предметам);
- сервис рекомендаций (определение тем к повторению на основе статистики);
- сервис уведомлений (формирование событий, отправка push-сообщений, фиксация статуса уведомлений).

На уровне доступа к данным используется объектно-реляционное отображение (ORM) Entity Framework Core, которое обеспечивает абстракцию от конкретной реализации базы данных и позволяет оперировать объектами предметной области вместо прямых SQL-запросов. В рамках данного уровня определены все сущности, используемые в программном комплексе: пользователь, роль, предмет, тема, задание, тест, состав теста, назначение, попытка, ответ, статистика, рекомендация, уведомление. Каждая из перечисленных сущностей представлена соответствующей моделью, описывающей её структуру и атрибуты. Для каждой модели в явном виде определены конфигурации, включающие описание первичных и внешних

ключей, необходимые индексы для ускорения выполнения запросов, а также ограничения целостности данных, такие как уникальность полей, допустимые значения и обязательность заполнения.

Серверная часть программного комплекса поддерживает документирование интерфейсов прикладного программирования с помощью спецификации OpenAPI (реализованной посредством Swagger). Документация позволяет разработчикам и обслуживающему персоналу ознакомиться со всеми доступными конечными точками API. Для каждой конечной точки в документации указаны используемый HTTP-метод, маршрут (URL), набор входных параметров с их типами и обязательностью, формат выходных данных, а также перечень возможных кодов ответов с описанием соответствующих ситуаций (успешное выполнение, ошибки валидации, отсутствие прав доступа, внутренняя ошибка сервера и другие).

Взаимодействие с базой данных в системе выполняется исключительно через уровень доступа к данным, описанный выше. Прямое обращение к базе данных из контроллеров (обработчиков HTTP-запросов) или сервисов (содержащих бизнес-логику) не допускается архитектурой комплекса. Такой подход обеспечивает чёткое разделение ответственности между компонентами, упрощает тестирование и модификацию логики доступа к данным без изменения остальных частей системы.

Серверная часть также содержит подсистему логирования, предназначенную для регистрации всех критических событий, возникающих в процессе работы, ошибок выполнения и исключений [17]. Логирование выполняется с обязательной привязкой к идентификатору пользователя, инициировавшего действие или столкнувшегося с ошибкой, а также к точному времени наступления события. Это позволяет в дальнейшем выполнять анализ поведения системы, выявлять причины сбоев и обеспечивать аудит действий пользователей.

Программная архитектура серверной части, реализованная на платформе ASP.NET Core, в графическом виде представлена на рисунке 5.1. Диаграмма классов отражает ключевые сущности предметной области (User, Subject, Test, Task, Attempt, UserAnswer), сервисный слой с классами AuthService, TestService, AttemptService и UserAnswerRepository, интерфейсы репозитория для доступа к данным (IUserRepository, ITestRepository, IAttemptRepository), а также контекст базы данных ApplicationDbContext, агрегирующий коллекции DbSet для всех сущностей и обеспечивающий сохранение изменений через метод SaveChanges().

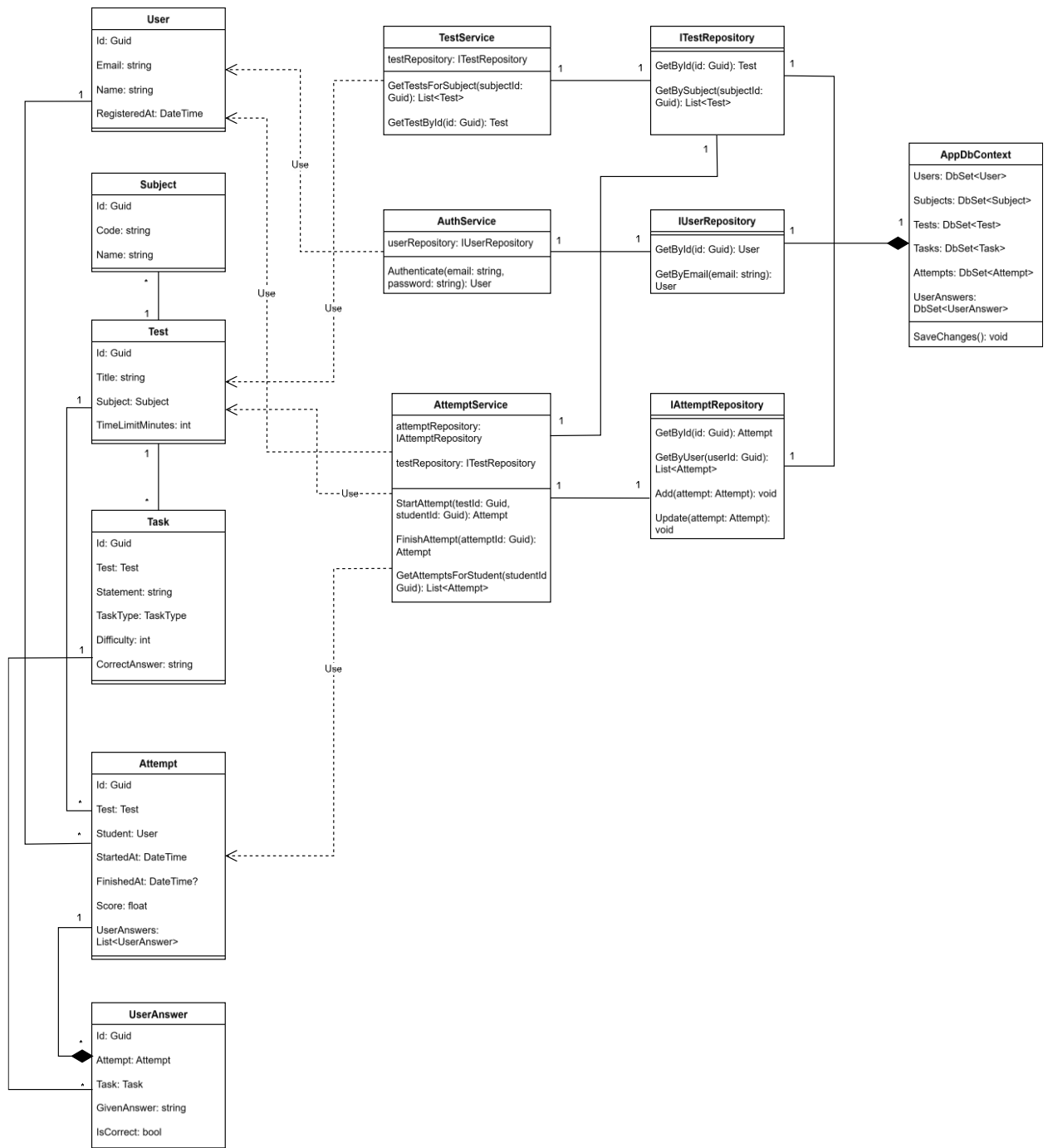


Рисунок 5.1 — Диаграмма классов серверной части (ASP.NET Core)

5.3 Структура мобильного приложения

Мобильное приложение реализовано на языке Kotlin для операционной системы Android с использованием архитектурного паттерна MVVM [18]. В приложении выделены следующие уровни: пользовательский интерфейс, уровень управления состоянием и уровень работы с данными.

Пользовательский интерфейс реализован с использованием Jetpack Compose и включать экраны:

– экран регистрации и входа, обеспечивающий ввод данных и взаимодействие с сервером для получения токена авторизации;

– главный экран с выбором предмета и типа теста;

– экран списка тестов прошлых лет с возможностью фильтрации по предмету и году;

– экран генерации смешанного теста с указанием количества заданий по категориям и уровням сложности;

– экран назначенных тестов, отображающий задания, полученные от преподавателя;

– экран прохождения теста, включающий таймер, навигацию по вопросам и ввод ответов;

– экран результатов с указанием количества правильных и неправильных ответов, общей оценки и времени выполнения;

– экран истории решений, содержащий список попыток и возможность просмотра подробных результатов каждой;

– экран статистики, отображающий динамику подготовки, распределение ошибок по темам и рекомендации к повторению;

– экран уведомлений, отображающий список событий, полученных от сервера.

Уровень управления состоянием реализован с использованием ViewModel и обеспечивать хранение состояния экранов, обработку бизнес-логики на клиенте, а также вызовы к API. Каждое основное направление функциональности (аутентификация, тесты, попытки, статистика, уведомления) имеет собственную ViewModel.

Уровень работы с данными включает:

– сетевой модуль для взаимодействия с серверной частью через Retrofit, с поддержкой авторизации по JWT-токену;

– локальное хранилище на базе Room/SQLite для кэширования тестов, сохранения незавершённых попыток и работы в офлайн-режиме;

– модуль синхронизации, обеспечивающий отправку данных на сервер после восстановления сети.

Приложение поддерживает разграничение интерфейса в зависимости от роли пользователя. Для роли «ученик» доступны функции прохождения тестов, просмотра статистики и получения рекомендаций. Для роли «преподаватель» дополнительно доступны функции просмотра статистики прикреплённых учеников и их истории решений.

Архитектура приложения обеспечивает модульность. Добавление новых экранов и расширение функционала не должно требовать изменений в существующих модулях. Взаимодействие между компонентами осуществляется через чётко определённые интерфейсы.

Приложение поддерживает обработку ошибок: сетевых сбоев, истечения срока действия токена, недоступности сервера. Все ошибки отображаются пользователю в виде уведомлений или диалогов.

5.4 Структура веб-инструмента преподавателя

Веб-инструмент преподавателя реализован на платформе Django с использованием Django Templates. Интерфейс доступен в современных браузерах (Google Chrome, Mozilla Firefox, Microsoft Edge) и поддерживает минимальное разрешение экрана 1366×768 пикселей.

В системе реализованы следующие разделы интерфейса:

1 Раздел авторизации – форма входа по электронной почте и паролю с проверкой учётных данных через серверную часть.

2 Раздел управления тестами – список созданных тестов с возможностью сортировки, поиска и фильтрации по предмету, дате создания и статусу публикации.

3 Раздел создания и редактирования теста – форма для добавления заданий из банка и создания новых вопросов. Форма должна включать поля для формулировки задания, типа задания, уровня сложности, правильного ответа и пояснения. Преподаватель должен иметь возможность изменять порядок заданий, указывать лимит времени и число попыток для теста.

Веб-инструмент должен обеспечивать валидацию всех вводимых данных на клиентской и серверной стороне. Ошибки валидации должны отображаться в интерфейсе пользователя.

Все взаимодействие с сервером выполняется через REST API. Запросы передаются по протоколу HTTPS с использованием JWT-токенов для авторизации. Данные о создании, редактировании и назначении тестов сохраняются в базе данных через серверную часть.

Веб-инструмент использует адаптивные шаблоны. Интерфейс должен корректно отображаться на различных диагоналях экрана без потери функциональности. Элементы управления сгруппированы по назначению и обеспечивают минимальное количество действий для выполнения базовых операций.

Для предотвращения потери данных при создании тестов реализован механизм сохранения черновиков. Черновик теста должен оставаться в системе до момента публикации или удаления преподавателем.

Веб-инструмент должен обеспечивать разграничение доступа. Все функции управления тестами и учениками доступны только авторизованным пользователям с ролью преподавателя.

Структура классов мобильного приложения, спроектированного по архитектурному паттерну MVVM на языке Kotlin, изображена на рисунке 5.2

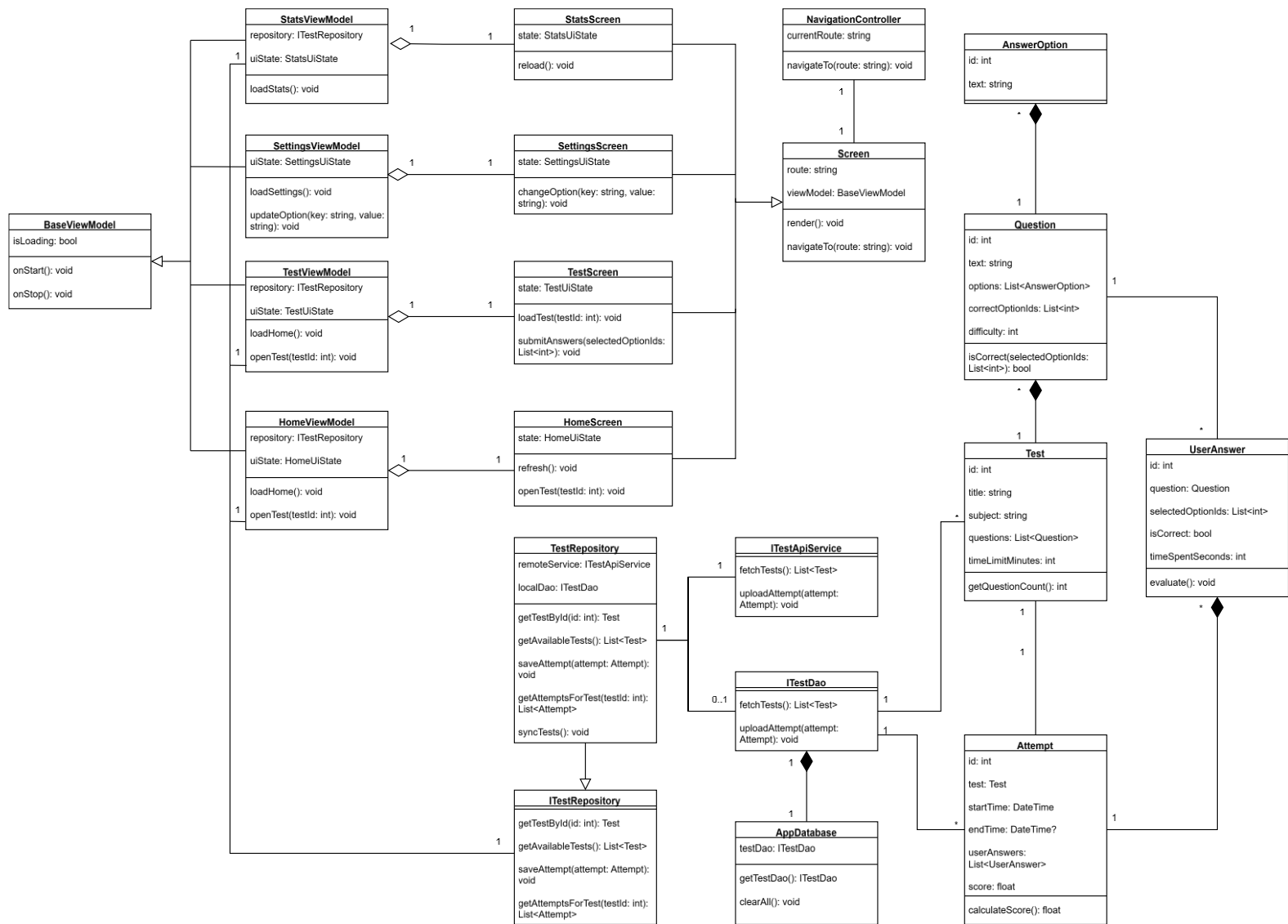


Рисунок 5.2 — Диаграмма классов мобильного приложения (Kotlin, MVVM)

5.5 Структура административной панели

Административная панель реализована с использованием встроенного инструмента Django Admin и предназначена для управления основными сущностями системы. Доступ к панели предоставляется только пользователям с ролью администратора или разработчика. Авторизация выполняется по имени пользователя и паролю с последующей проверкой прав доступа.

В административной панели реализованы следующие функции:

- управление пользователями: создание, редактирование, блокировка и удаление учётных записей, изменение ролей;
- управление предметами: добавление и редактирование наименований предметов, их кодов и статусов активности;
- управление темами: добавление, редактирование и удаление тем, привязка тем к предметам, настройка иерархии тем;
- управление заданиями: создание, редактирование и удаление заданий, изменение формулировки, правильного ответа, уровня сложности, привязки к предмету и теме;
- управление тестами: просмотр и редактирование тестов прошлых лет и пользовательских тестов, изменение их параметров, статуса публикации и состава;
- просмотр назначений тестов: проверка связей «преподаватель—ученик» и списка назначенных тестов;
- просмотр и редактирование справочников (например, источников экзаменационных вариантов).

Панель предоставляет стандартный интерфейс Django Admin с возможностью поиска, фильтрации и сортировки записей. Для всех сущностей реализованы формы редактирования с валидацией данных.

Все изменения, выполняемые через административную панель, фиксируются в журнале событий. В журнале хранятся сведения о пользователе, дате и времени операции, изменённых сущностях и типе действия (создание, редактирование, удаление).

Административная панель обеспечивает поддержку массовых операций: удаление нескольких записей, активация и деактивация выбранных элементов.

5.6 Поддерживаемые платформы и окружения

Мобильное приложение поддерживает операционную систему Android версии не ниже 8.0 (API level 26). Приложение должно корректно функционировать на устройствах с архитектурой процессора ARM и x86, с минимальным объёмом оперативной памяти 2 ГБ и минимальным разрешением экрана 720×1280 пикселей. Приложение должно быть оптимизировано для работы как на смартфонах, так и на планшетах.

Серверная часть поддерживает развёртывание на операционных

системах Windows Server 2019 и выше и Ubuntu Linux 22.04 и выше [19]. Допускается использование других совместимых дистрибутивов Linux. Для удобства развёртывания и сопровождения серверная часть поддерживает работу в контейнерах Docker.

База данных реализована в СУБД PostgreSQL версии 15 или выше. База данных поддерживает развёртывание на выделенных серверах, в контейнерах Docker, а также в облачных средах (например, Amazon RDS или Microsoft Azure Database for PostgreSQL).

Веб-инструмент для преподавателей и административная панель поддерживают работу в современных браузерах: Google Chrome (версии не ниже 110), Mozilla Firefox (версии не ниже 110) и Microsoft Edge (версии не ниже 110). Интерфейс корректно отображается при минимальном разрешении экрана 1366×768 и выше.

Для системы определены следующие минимальные аппаратные требования для развёртывания:

- сервер приложений: 4 ядра CPU, 8 ГБ оперативной памяти, 50 ГБ дискового пространства;
- сервер базы данных: 4 ядра CPU, 8 ГБ оперативной памяти, 100 ГБ дискового пространства;
- возможность горизонтального масштабирования серверной части при возрастании нагрузки.

Все компоненты программного комплекса поддерживают работу в сетевой среде с использованием протокола HTTPS и шифрования TLS версии 1.3 и выше.

В разделе описана программная реализация компонентов комплекса и обоснован выбор технологий разработки. Представлена структура серверной части, мобильного приложения и веб-инструмента преподавателя. Рассмотрены архитектурные решения, включая организацию уровней данных и пользовательских интерфейсов. Также определены системные требования и поддерживаемые платформы для корректной работы системы. Все компоненты готовы к сборке, тестированию и дальнейшему развёртыванию.

6 ТЕСТИРОВАНИЕ И ПРОВЕРКА РАБОТОСПОСОБНОСТИ ПРОГРАММНОГО КОМПЛЕКСА

В данном разделе приведены результаты тестирования программного комплекса, охватывающего модульное, функциональное и нагрузочное тестирование всех ключевых компонентов: серверного API-приложения на ASP.NET Core, панели преподавателя на Django, мобильного Android-клиента и базы данных PostgreSQL. Целью тестирования являлась проверка соответствия реализованной функциональности требованиям, сформулированным в разделах 2 и 3, а также оценка производительности и стабильности системы под расчётной нагрузкой.

6.1 Цели и методы тестирования

Тестирование программного комплекса проводилось с целью подтверждения соответствия реализованной функциональности требованиям, изложенным в разделах 2 и 3, а также для оценки стабильности и производительности системы. Были применены следующие виды тестирования:

- модульное тестирование серверной части (ASP.NET Core) и панели преподавателя (Django);
- функциональное тестирование веб-инструмента преподавателя с автоматизацией через Selenium;
- нагрузочное тестирование REST API посредством Apache JMeter;
- ручное функциональное тестирование мобильного Android-приложения на реальных устройствах.

Для модульного тестирования использовались фреймворки xUnit (.NET) и pytest (Python). Автоматизация действий браузера выполнялась с помощью Selenium WebDriver для Google Chrome. Нагрузочное тестирование проводилось инструментом JMeter 5.6, эмулировавшим одновременную работу до 500 виртуальных пользователей. Мобильное приложение тестировалось вручную на устройствах с Android 10 и 13.

6.2 Модульное тестирование серверной части

Модульные тесты написаны для ключевых сервисов приложения, реализующих бизнес-логику. Ниже приведены характерные примеры.

6.2.1 Тестирование сервиса аутентификации (AuthService). Проверялась корректность входа пользователя с правильными и неправильными учётными

данными, а также обработка случаев с неподтверждённым адресом электронной почты. Фрагмент теста для метода LoginAsync представлен на рисунке 6.1.

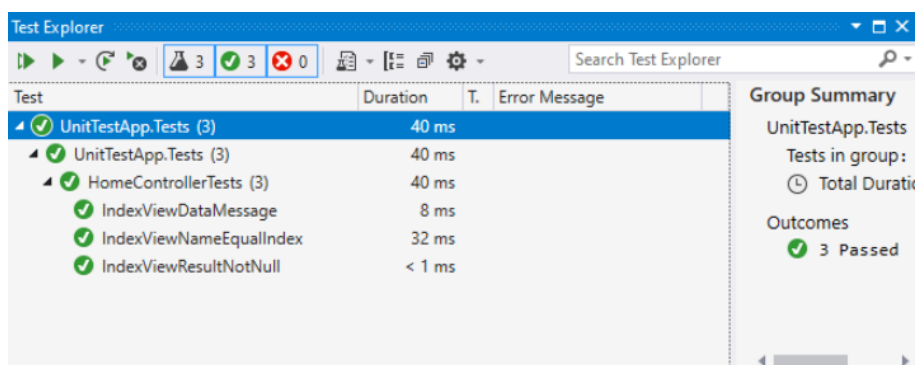


Рисунок 6.1 – Листинг юнит-теста сервиса AuthService

Всего для AuthService реализовано 12 тестов, охватывающих сценарии успешного входа, неверного пароля, отсутствия пользователя и блокировки после исчерпания попыток. Тесты подтвердили корректность работы механизма JWT-токенов и обновления сессий.

6.2.2 Тестирование сервиса задач (ProblemService). Тесты проверяли создание задачи, её редактирование и удаление, а также контроль прав доступа. В частности, тест `CreateProblem_ValidRequest_ReturnsSuccess` удостоверяет, что при корректных входных данных задача сохраняется в базе, а активная версия получает флаг `IsActive`. Тест `DeleteProblem_ByNonAuthor_ReturnsForbidden` гарантирует, что удалить задачу может только её автор. Аналогичные тесты написаны для сервисов `TestService`, `AssignmentService` и `StatisticsService`.

Для Django-приложения (панель преподавателя) модульные тесты проверяют корректность форм создания теста и назначения ученикам, а также работу представлений (views) с авторизацией. Использовался фреймворк `pytest` совместно с `Django TestCase`. Обнаруженные на ранних этапах ошибки валидации и контроля доступа были исправлены.

6.3 Функциональное тестирование веб-инструмента преподавателя

Функциональное тестирование пользовательского интерфейса панели преподавателя проводилось методом автоматизированного выполнения сценариев в браузере Google Chrome с помощью Selenium WebDriver [20]. Основное внимание уделялось формам регистрации, входа и создания теста, поскольку они являются наиболее критичными для работы преподавателя.

Сценарий тестирования формы авторизации включал:

- переход на страницу /login;
- ввод корректной пары email/пароль;
- проверку перенаправления в раздел «Мои тесты» и появления приветственного сообщения;
- ввод заведомо неверного пароля и проверку появления сообщения об ошибке;
- попытку доступа к защищённой странице без авторизации (ожидался код 302 и редирект на логин).

Сценарий создания теста:

- авторизация под учётной записью преподавателя;
- нажатие кнопки «Создать тест», заполнение полей названия, предмета и параметров;
- добавление двух заданий из банка;
- сохранение теста и проверка его появления в списке со статусом «Черновик»;
- публикация теста и проверка изменения статуса на «Опубликован».

Все сценарии выполнялись в трёх основных браузерах: Chrome 120, Firefox 121 и Edge 120. Результаты функционального тестирования сведены в таблицу 6.1.

Таблица 6.1 – Результаты автоматизированного функционального тестирования веб-панели

Тестируемая функция	Число сценариев	Успешно	Не успешно
Авторизация (вход/выход)	6	6	0
Регистрация нового преподавателя	3	3	0
Создание теста (черновик)	5	5	0
Редактирование и публикация теста	4	4	0
Назначение теста ученикам	4	4	0
Просмотр списка учеников и их результатов	3	3	0

Все автоматизированные тесты выполнены успешно, интерфейс стабилен в указанных браузерах.

Ручное функциональное тестирование мобильного Android-приложения охватило 25 сценариев, включая регистрацию учащегося, прохождение теста

с таймером, просмотр статистики и получение push-уведомлений. Критических дефектов не выявлено.

6.4 Нагрузочное тестирование REST API

Для проверки соответствия нефункциональным требованиям по производительности (раздел 2.3) выполнено нагрузочное тестирование ключевых конечных точек API. Использовался Apache JMeter с профилем, имитирующим 500 одновременных пользователей, постепенно наращиваемых в течение 2 минут и удерживаемых на пике 5 минут [21]. Тестовый сценарий включал:

- авторизацию (POST /auth/login) и получение JWT;
- получение списка тестов (GET /tests/student/list);
- запуск попытки и отправку ответов (POST /attempts/start, POST /attempts/{id}/complete);
- запрос статистики (GET /statistics/me).

Тестирование проводилось на сервере с 4 ядрами CPU и 8 ГБ ОЗУ (конфигурация, рекомендованная в разделе 5.6). Результаты приведены в таблице 6.2.

Таблица 6.2 – Результаты нагрузочного тестирования API (500 пользователей)

Эндпоинт	Среднее время ответа, мс	95-й перцентиль, мс	Пропускная способность, запр./сек	Ошибки, %
POST /auth/login	185	340	210	0,0
GET /tests/student/list	220	410	185	0,0
POST /attempts/start	260	470	150	0,0
POST /attempts/.../complete	310	580	120	0,0
GET /statistics/me	280	520	160	0,0

Среднее время ответа всех запросов не превысило 320 мс, что находится в пределах установленного требования (не более 1 с). Ошибки отсутствовали. Пропускная способность API достаточна для обслуживания расчётной нагрузки. Таким образом, производительность серверной части признана удовлетворительной.

6.5 Результаты тестирования

Сводные данные обо всех видах тестирования и их результатах представлены в таблице 6.3.

Таблица 6.3 – Сводные результаты тестирования программного комплекса

Вид тестирования	Инструменты	Количество тестов/сценариев	Успешно	Выявлено дефектов (критичных)	Статус
Модульное (ASP.NET Core)	xUnit	78	78	0	Пройдено
Модульное (Django)	pytest	22	22	0	Пройдено
Функциональное (веб)	Selenium	25	25	0	Пройдено
Функциональное (мобильное)	Ручное	25	25	0	Пройдено
Нагрузочное	JMeter	5 профилей	—	—	Пройдено

В ходе тестирования все выявленные несоответствия и дефекты были устранены. Программный комплекс демонстрирует стабильную работу, удовлетворяет заявленным функциональным и нефункциональным требованиям и может быть рекомендован к внедрению.

В данном разделе проведено комплексное тестирование программного комплекса. Модульные тесты серверной части (78 тестов) и панели преподавателя (22 теста) успешно пройдены. Функциональное тестирование веб-инструмента (25 сценариев) и ручное тестирование мобильного приложения (25 сценариев) подтвердили корректность работы всех функций, критических дефектов не выявлено. Нагрузочное тестирование API при 500 одновременных пользователях показало среднее время ответа до 310 мс, что соответствует установленному требованию (не более 1 секунды), ошибки отсутствовали. Таким образом, программный комплекс полностью соответствует заявленным требованиям и может быть рекомендован к внедрению.

7 РУКОВОДСТВО ПО РАЗВЕРТЫВАНИЮ ПРИЛОЖЕНИЯ

Этот раздел описывает последовательность действий, позволяющую администратору развернуть готовый программный комплекс из скомпилированных пакетов без повторной сборки исходного кода. Комплекс включает четыре компонента: серверное API-приложение на ASP.NET Core, панель преподавателя на Django, мобильный клиент для Android и базу данных PostgreSQL. Все шаги рассчитаны на Windows Server 2019+ или Ubuntu 22.04 LTS; команды для Linux приведены в скобках.

7.1 Обзор компонентов и требований

В поставку входят:

- каталог `server` (исполняемый файл `CtHelper.Server.exe` или `linux-сборка`);
- каталог `teacher-panel` (приложение Django со статическими файлами);
- установочный APK-файл (`CtHelper.apk`);
- каталог `db` (SQL-скрипты `create_tables.sql` и `seed.sql`).

Требования к окружению:

- СУБД PostgreSQL 15+;
- .NET Runtime 8.0 (ASP.NET Core Hosting Bundle для Windows, пакет `aspnetcore-runtime-8.0` для Ubuntu);
- Python 3.10+ с менеджером пакетов `pip`;
- веб-сервер Nginx (для проксирования Django в production);
- Android 8.0 (Oreo) и выше на мобильном устройстве (рекомендуется 4 ГБ ОЗУ, FullHD экран).

7.2 Подготовка окружения

Создайте на сервере каталог `C:\cthelper` (`/opt/cthelper`). Установите PostgreSQL 15+, запомните пароль суперпользователя `postgres`. Установите .NET Runtime: для Windows скачайте ASP.NET Core Hosting Bundle с `dotnet.microsoft.com`, для Ubuntu выполните `sudo apt install aspnetcore-runtime-8.0`. Установите Python 3.10+ и `pip` (на Ubuntu – `sudo apt install python3 python3-pip`). Проверьте, что на сервере открыты порты 5001 (ASP.NET Core) и 8000 (Django dev)

7.3 Развертывание базы данных

Откройте pgAdmin (или `psql`) и создайте новую базу `cthelper`: «CREATE DATABASE cthelper». Создайте отдельного пользователя приложения: «CREATE USER cthelper_user WITH PASSWORD 'securepass';

`GRANT ALL PRIVILEGES ON DATABASE cthelper TO cthelper_user;»`.
Скопируйте каталог `db` в `/opt/cthelper/db`. Выполните скрипты: `«psql -U cthelper_user -d cthelper -f /opt/cthelper/db/create_tables.sql»` и `«psql -U cthelper_user -d cthelper -f /opt/cthelper/db/seed.sql»`.

Скрипты создадут таблицы (пользователи, тесты, задания, попытки, статистика, уведомления) и заполнят справочники (предметы, темы, администратор по умолчанию). Убедитесь, что в `pgAdmin` отображаются таблицы `public.user`, `public.problem`, `public.test` и др.

7.4 Развёртывание серверного API

Скопируйте каталог `server` в `C:\cthelper\server`. Отредактируйте файл `appsettings.Production.json`:

- `ConnectionStrings.DefaultConnection`:
`Host=localhost;Port=5432;Database=cthelper;Username=cthelper_user;Password=securepass;`
- `JwtSettings.SecretKey` (длинная случайная строка `base64`),
`AccessTokenExpirationMinutes` (например, 60), `RefreshTokenExpirationDays` (например, 30);
- настройте `Kestrel`: укажите порт 5001 и сертификат SSL (или временно используйте самоподписанный для отладки).

Запустите сервер командой `CtHelper.Server.exe` (Windows) или `dotnet CtHelper.Server.dll` (Linux). После успешного запуска в консоли отобразится `«Now listening on: https://0.0.0.0:5001»`, а на экране входа в API (`https://localhost:5001/swagger`) станет доступна документация Swagger со всеми эндпоинтами.

7.5 Развёртывание панели преподавателя

Скопируйте каталог `teacher-panel` в `/opt/cthelper/teacher-panel`. Перейдите в этот каталог и установите зависимости: `«pip install -r requirements.txt»`.

В файле `settings.py` укажите:

- `DATABASES`: `ENGINE django.db.backends.postgresql`, `NAME cthelper`,
`USER cthelper_user`, `PASSWORD securepass`;
- `ALLOWED_HOSTS` = `['*']` (или укажите IP вашего сервера);
- `STATIC_ROOT` = `os.path.join(BASE_DIR, 'static')`.

Выполните миграции Django (они синхронизируют модели с уже существующей схемой и создадут служебные таблицы):
`python manage.py migrate`

Соберите статические файлы: `python manage.py collectstatic`. Запустите сервер разработки для теста: `«python manage.py runserver 0.0.0.0:8000»`

Панель преподавателя будет доступна по адресу `http://<ip_сервера>:8000`.

7.6 Развёртывание мобильного клиента

Установите APK-файл на целевое устройство Android. Для дальнейшего использования следуйте инструкциям, приведенным в разделе 7.

7.7 Проверка работоспособности

Выполните сквозной тест всех компонентов:

1 Откройте веб-панель преподавателя (Django), авторизуйтесь как admin (пароль из seed.sql). Создайте новый тест по предмету «Математика», добавив несколько заданий.

2 В мобильном приложении зарегистрируйте ученика и войдите. На главном экране должен отобразиться созданный преподавателем тест в разделе «Пользовательские».

3 Запустите тест, ответьте на вопросы, завершите. Проверьте, что в разделе «Результаты» отображается попытка с баллами и временем.

4 На сервере через Swagger выполните GET /statistics/me и убедитесь, что статистика пересчиталась.

5 В Django Admin (<http://<ip>:8000/admin>) зайдите в раздел пользователей и проверьте наличие записей об ученике и преподавателе.

7.8 Обслуживание и резервное копирование

Раз в сутки выполняйте резервное копирование базы: `pg_dump -Fc cthelper > /backup/cthelper_$(date +%Y%m%d).dump`. Логи ASP.NET Core сохраняются в каталог logs рядом с CtHelper.Server.dll. При ошибках анализируйте файлы за соответствующий день. Django логирует ошибки в файл, указанный в settings.py.

Для обновления APK-файла на устройствах Android достаточно установить новую версию поверх старой – архитектура БД и API обратно совместимы.

В разделе представлено руководство по установке программного комплекса. Описаны этапы подготовки сервера, развертывания базы данных, серверного API и панели управления. Указан порядок установки мобильного приложения и сценарий проверки работы всей системы. Также приведены инструкции по обслуживанию: резервному копированию, анализу логов и обновлению системы. Раздел содержит всё необходимое для настройки и поддержки комплекса администратором.

8 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

В данном разделе приведены пошаговые инструкции для каждой роли системы. Все операции выполняются через мобильное Android-приложение или веб-панель преподавателя. Для выполнения действия нажмите указанную кнопку или ссылку. При успешном завершении отобразится уведомление/новый экран, при ошибке – красное сообщение с пояснением.

8.1 Общие шаги авторизации

Система имеет два интерфейса: мобильное приложение (для учеников и преподавателей в полевых условиях) и веб-панель (для преподавателей за компьютером при создании/редактировании тестов). Административные функции выполняются через панель Django Admin, доступную по URL /admin.

8.1.1 Регистрация.

1 Откройте мобильное приложение. На экране входа нажмите кнопку «Регистрация».

2 Заполните поля: Имя пользователя, Email, Пароль. Выпадающим списком выберите роль – «Ученик» или «Преподаватель».

3 Нажмите «Зарегистрироваться». На указанный Email придет письмо с кодом подтверждения из 6 цифр.

4 Вернитесь в приложение, введите полученный код в поле «Подтверждение Email» и нажмите «Подтвердить». Код действителен 10 минут, дается 5 попыток ввода.

5 После подтверждения откроется главный экран приложения, соответствующий вашей роли.

8.1.2 Вход в систему.

На начальном экране приложения введите Email и пароль, нажмите «Войти». Если Email подтвержден, вы попадете на главный экран. Для веб-панели преподавателя перейдите по адресу <http://<сервер>:8000>, введите Email и пароль на форме входа – откроется раздел «Мои тесты».

8.1.3 Восстановление пароля.

На экране входа нажмите «Забыли пароль». Введите Email, нажмите «Отправить». На почту придет код сброса из 6 цифр. В приложении введите код и новый пароль, нажмите «Сменить пароль». После этого выполните вход заново.

8.2 Ученик. Прохождение тестов и анализ своих результатов

Рабочим инструментом ученика в программном комплексе является мобильный Android клиент. Ниже описаны ключевые функции предоставляемые в данном клиенте

8.2.1 Просмотр доступных тестов.

1 На главном экране в верхней части выберите предмет из выпадающего списка (например, «Математика»).

2 Отобразятся три вкладки: «Архив» – варианты ЦТ прошлых лет; «Пользовательские» – тесты, назначенные вашим преподавателем; «Генератор» – создание смешанного теста.

3 На вкладке «Архив» доступны фильтры: год проведения, тема. Нажмите интересующий вариант – откроется карточка с количеством заданий, средней сложностью и кнопкой «Начать».

4 Если на вкладке «Пользовательские» есть непройденные тесты, рядом с названием отображается метка «Новое» и крайний срок выполнения.

8.2.2 Генерация смешанного теста.

1 На вкладке «Генератор» выберите предмет. Укажите количество заданий для каждой нужной темы, задайте сложность (ползунком или кнопками Очень легко – Очень трудно) и отметьте флаг «Включать задания преподавателя», если хотите добавить кастомные вопросы.

2 Нажмите «Сгенерировать». Приложение покажет сводку: общее число заданий, темы, примерное время прохождения.

3 Нажмите «Начать» для запуска.

8.2.3 Прохождение теста

1 На экране теста в верхней панели отображается таймер обратного отсчета (для тренировочных тестов может быть скрыт), номер текущего вопроса и общее число заданий.

2 Читайте условие и вводите/выбирайте ответ. Для заданий с выбором – нажмите на вариант ответа, для открытых – введите текст/число, для множественного выбора – отметьте галочками варианты.

3 Для перехода к следующему вопросу нажмите стрелку «Далее». Можно вернуться стрелкой «Назад» и изменить предыдущие ответы. В нижней части экрана – полоса прогресса с цветовой индикацией отвеченных вопросов.

4 Если тест тренировочный, доступна кнопка «Пауза» в левом верхнем углу – нажмите, чтобы приостановить таймер и временно выйти. Позже тест можно возобновить из раздела «История».

5 После ответа на последний вопрос нажмите «Завершить». Система автоматически отправит ответы на сервер. Изображение экрана прохождения теста с таймером, навигацией по заданиям и индикатором прогресса приведено ниже, на рисунке 8.1, а также на плакате ГУИР.253504.002 ПЛ.

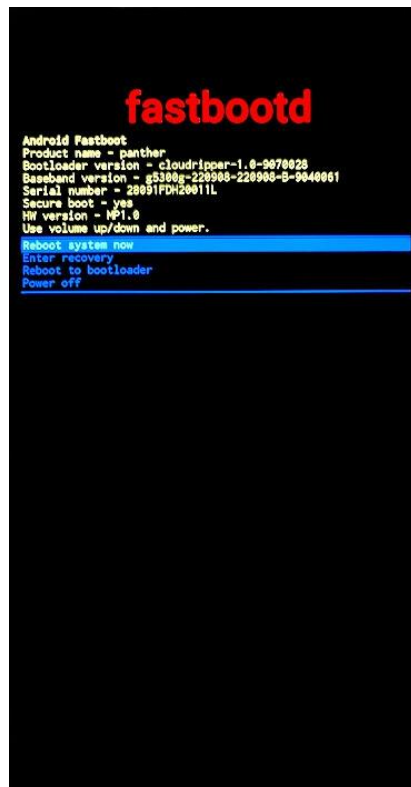


Рисунок 8.1 – Экран прохождения теста с таймером, навигацией по заданиям и индикатором прогресса

8.2.4 Результаты и разбор ошибок

1 Сразу после завершения откроется экран результата. В верхней части – итоговый балл в процентах, количество правильных/неправильных ответов, затраченное время.

2 Ниже – список всех заданий. Каждое помечено зеленой галочкой (верно) или красным крестиком (неверно). Для неверных показан ваш ответ и правильный ответ с пояснением.

3 Нажмите «К статистике» для перехода к аналитике.

Ниже, на рисунке рисунок 8.2, а также на плакате ГУИР.253504.002 ПЛ прикреплено изображение экрана результатов попытки с разбором правильных и неправильных ответов.

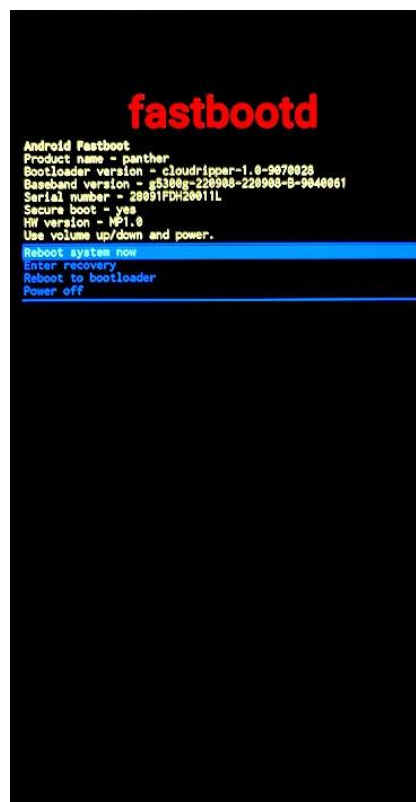


Рисунок 8.2 – Экран результатов попытки с разбором правильных и неправильных ответов

8.2.5 История решений

- 1 В нижнем меню (или боковой панели) выберите раздел «История».
- 2 Отобразится список всех завершенных и прерванных попыток, отсортированных по дате. Каждая строка содержит название теста, дату, результат и статус (Завершен / Пауза).
- 3 Нажмите на любую попытку – повторится экран детального разбора (как в 8.2.4).

8.2.6 Статистика и рекомендации

- 1 В нижнем меню выберите раздел «Статистика».
- 2 Выберите предмет. Система отобразит график динамики успеваемости (баллы за последние 10 попыток) и блок «Темы».
- 3 Блок «Темы» разбит на две вкладки:
 - Обзор: все темы с цветовой шкалой успешности (зеленый – >70%, желтый – 40-70%, красный – <40%).
 - Рекомендации: список тем, требуемых повторения, отсортированный по приоритету. Для каждой темы указан процент ошибок и кнопка «Тренировать» – создает смешанный тест только по этой теме.
- 4 Ниже блока тем можно выбрать период анализа: последняя неделя, месяц или всё время.

Скриншот, иллюстрирующий работу с экраном статистики ученика представлен ниже на рисунке 8.3, а также на плакате ГУИР.253504.002 ПЛ.

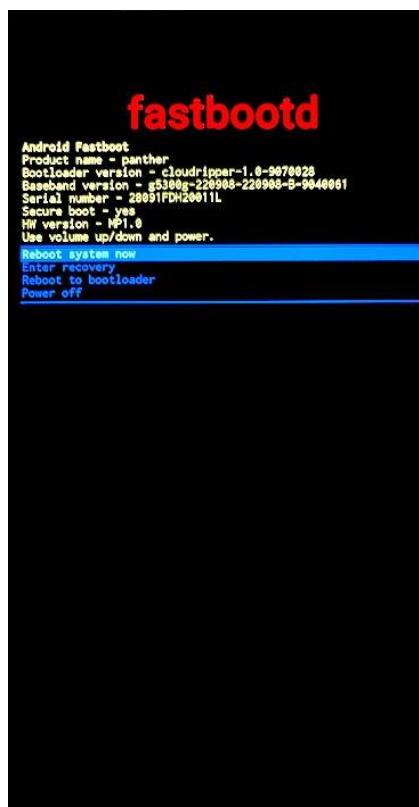


Рисунок 8.3 – Экран персональной статистики ученика с цветовой шкалой успеваемости по темам и рекомендациями к повторению

8.3 Преподаватель. Создание тестов, управление учениками и контроль успеваемости

Преподаватель работает в двух средах: веб-панель (Django) – для полноценного создания и редактирования тестов; мобильное приложение – для быстрого назначения тестов, просмотра списков учеников и их результатов.

8.3.1 Создание нового теста (Web-панель).

1 Авторизуйтесь в веб-панели преподавателя. Нажмите «Создать тест».
2 Укажите предмет, название теста, тип (тренировочный или экзаменационный), длительность (в минутах, 0 – без ограничения) и количество попыток (0 – неограниченно).

3 Для добавления заданий из банка: выберите слева тему или введите поисковый запрос – отобразится список доступных заданий. Каждое можно развернуть, чтобы увидеть формулировку. Нажмите «+» рядом с заданием – оно добавится в правую часть экрана (состав теста).

4 Чтобы создать новое задание: нажмите «+ Новое задание», заполните поля: формулировка, тип (один/множественный выбор, открытый ответ), варианты ответов с пометкой правильного, уровень сложности, тему, пояснение к ответу. Нажмите «Сохранить» – задание добавится и в банк, и в тест.

5 Перетаскивайте задания в правой части для сортировки. Каждому заданию можно назначить код (например, A1, B2) для идентификации.

6 Нажмите «Сохранить тест». Он появится в разделе «Мои тесты» со статусом «Черновик».

7 Для публикации нажмите на тесте кнопку «Опубликовать». Опубликованные тесты становятся доступны для назначения ученикам. Экран создания теста показан на рисунке 8.4, а также на плакате ГУИР.253504.003 ПЛ.



Рисунок 8.4 – Экран создания теста: добавление заданий из банка, сортировка и настройка параметров

8.3.2 Назначение теста ученикам (Мобильное приложение)

1 В мобильном приложении (роль Преподаватель) перейдите в раздел «Мои тесты».

2 Выберите опубликованный тест и нажмите «Назначить».

3 Откроется список ваших учеников. Отметьте галочками тех, кому нужно назначить тест.

4 Укажите крайний срок (опционально) и нажмите «Назначить».

5 Ученики мгновенно получают push-уведомление «Новый тест: <название>», а тест появится у них на вкладке «Пользовательские».

Экран управления назначениями тестов ученикам и группам приведен ниже на рисунке 8.5, а также на плакате ГУИР.253504.003 ПЛ.



Рисунок 8.5 – Экран управления назначениями тестов ученикам и группам

8.3.3 Просмотр результатов учеников (Мобильное приложение)

1 В разделе «Ученики» выберите ученика из списка. Отобразится его профиль: имя, email, средний балл по предметам.

2 Нажмите «Попытки» – увидите историю всех попыток ученика с результатами. Нажмите на конкретную попытку – откроется детальный разбор с ответами.

3 Нажмите «Статистика» – отобразится аналитика по предметам с цветовой разбивкой по темам (аналогично п. 8.2.6).

8.3.4 Приглашение учеников (Мобильное приложение)

1 В разделе «Приглашения» нажмите «Создать код». Задайте лимит использований и срок действия (опционально).

2 Нажмите «Сгенерировать» – система выдаст 9-значный код, например ABC-123-DEF. Поделитесь им с учеником (в мессенджере, на доске в классе).

3 Ученик в своем приложении в разделе «Профиль» > «Привязка к преподавателю» вводит этот код и нажимает «Отправить». У вас в разделе «Запросы» появится заявка. Нажмите «Подтвердить» – ученик привяжется к вам и сможет получать ваши тесты. Если нужно прекратить сотрудничество, в списке учеников свайпом влево выберите «Отвязать».

Экран создания пригласительного кода приведен ниже, на рисунке 8.6.



Рисунок 8.6 – Экран управления назначениями тестов ученикам и группам

8.4 Администратор: управление системой

Административные функции выполняются через панель Django Admin, доступную по адресу <http://<сервер>:8000/admin>.

8.4.1 Управление пользователями

В разделе «Users» можно просматривать, редактировать, блокировать/разблокировать и удалять учетные записи учеников и преподавателей. Создание пользователя здесь не рекомендуется (должно происходить через регистрацию в приложении с подтверждением email).

8.4.2 Управление контентом

- Subjects: добавление, редактирование предметов.
- Sections, Topics: управление иерархией тем.

- Problems: просмотр, создание, редактирование заданий, изменение статусов is_active и is_public.
 - Tests: просмотр всех тестов, изменение статусов публикации.
 - Invitation codes: просмотр и ручная деактивация кодов приглашений.
- Изображения экрана управления ключевыми сущностями приложения приведено ниже, на рисунке 8.8.

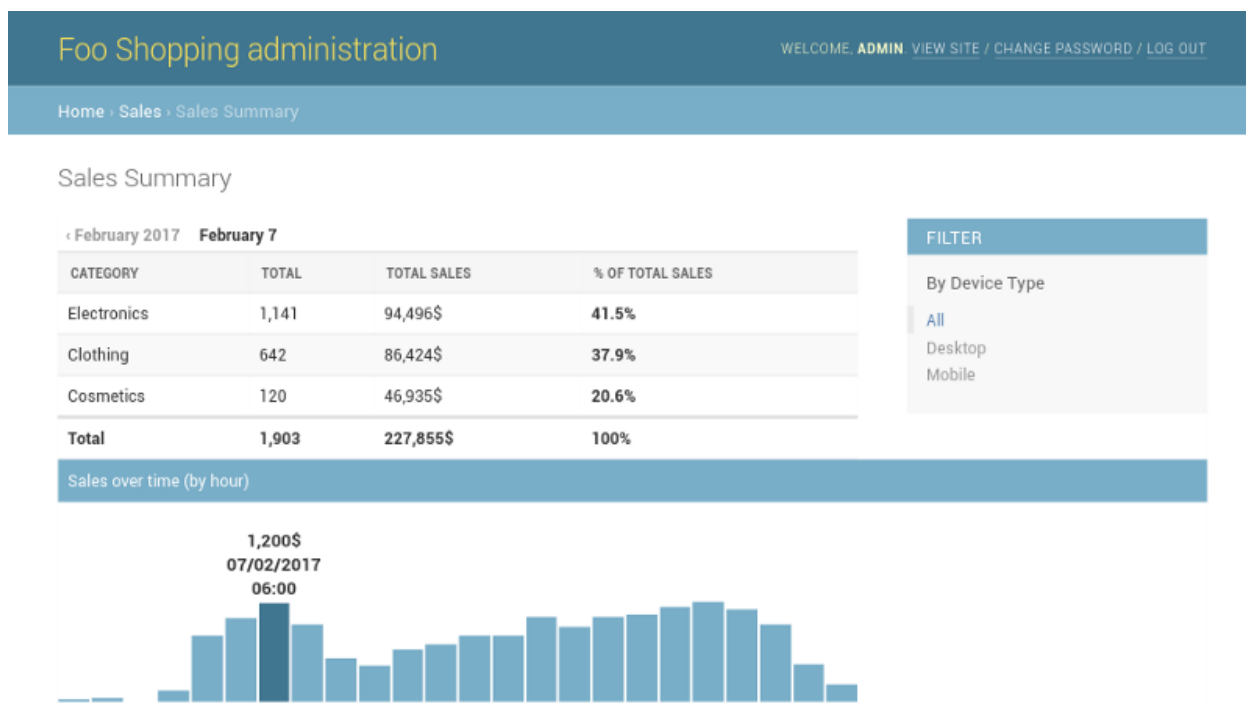


Рисунок 8.7 – Основной экран административной панели

8.4.3 Мониторинг

Раздел «User sessions» покажет все активные сессии пользователей, их JTI, IP-адреса и клиентские устройства. При подозрении на компрометацию сессии можно отозвать.

В данном разделе разработано полноценное руководство пользователя для трёх ролей: ученик, преподаватель, администратор. Описаны общие шаги авторизации: регистрация с подтверждением, вход в мобильное приложение и веб-панель, восстановление пароля. Для ученика детально представлены: просмотр доступных тестов, генерация смешанного теста с выбором тем и сложности, прохождение теста с таймером, экран результатов с разбором ошибок, история решений, статистика с графиком динамики и цветовой шкалой успеваемости по темам, а также персонализированные рекомендации к повторению. Для преподавателя описана работа в веб-панели: создание теста с добавлением заданий из банка и созданием новых, сортировка, публикация. Раздел полностью покрывает большинство сценариев использования и позволяет пользователям любой роли самостоятельно освоить систему.

9 ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ И РЕАЛИЗАЦИИ ПРОГРАММНОГО КОМПЛЕКСА НА МАССОВОМ РЫНКЕ

9.1 Характеристика программного средства

Разрабатываемый программный комплекс представляет собой цифровой сервис для подготовки к централизованному экзамену (ЦЭ) и централизованному тестированию (ЦТ). Система объединяет функции самостоятельной подготовки учащегося и инструменты контроля со стороны преподавателя в единой образовательной среде.

Функционально сервис включает:

- структурированную базу тестовых заданий прошлых лет с возможностью поиска по темам;
- генерацию индивидуальных тестов (смешанных, по выбранным темам, случайных);
- автоматический сбор статистики решений с детализацией по темам и типам ошибок;
- формирование персонализированных рекомендаций по повторению материала;
- личный кабинет преподавателя с возможностью создания собственных тестов, привязки учеников, мониторинга их прогресса и генерации аналитических отчётов.

Целевая аудитория – три основные группы. Учащиеся (выпускники школ, абитуриенты) получают инструмент для системной самоподготовки. Преподаватели и репетиторы используют сервис для создания тестов, контроля успеваемости и взаимодействия с учениками. Образовательные учреждения (школы, гимназии, центры) внедряют комплекс для организации пробных экзаменов и внутреннего мониторинга знаний. Общая численность потенциальных пользователей в Беларуси составляет более 68 тыс. учащихся и 28,5 тыс. преподавателей, что обеспечивает ёмкий рынок.

Программный комплекс ориентирован на три категории: учащихся, преподавателей и образовательные учреждения. Монетизация построена по гибридной модели Freemium с платными подписками, где бесплатный базовый функционал дополнен расширенными возможностями, а для преподавателей предусмотрена градация тарифов по количеству учеников. Продвижение осуществляется через магазины приложений, таргетированную и контекстную рекламу, партнёрства с образовательными центрами, профильные мероприятия и прямые продажи школам. Реализация продукта обеспечивает стабильный доход от подписок, масштабируемость бизнеса и рост доли компании на рынке.

9.2 Расчет инвестиций в разработку программного средства

Планируемый срок разработки MVP (Minimum Viable Product) программного комплекса составит 6 месяцев. Расчет инвестиций в разработку программного средства выполнен в соответствии с утвержденной методикой экономического обоснования дипломных проектов [22].

Соотношение премии к зарплате принято 50%.

Расчетная норма рабочего времени при пятидневной рабочей неделе составляет 2016 часов в год. (4.1).

$$Z_o = K_{пр} \cdot \sum_{i=1}^n Z_{чи} \cdot t_i, \quad (4.1)$$

где $K_{пр}$ – коэффициент премий и иных стимулирующих выплат;

n – количество исполнителей, занятых разработкой конкретного ПО, шт;

$Z_{чи}$ – часовой оклад исполнителя i -й категории, р;

t_i – трудоемкость работ, выполняемых исполнителем i -й категории, ч.

Месячные оклады разработчиков определены на основе анализа рыночных данных по IT-специалистам в г. Минске [23]. Расчет основной заработной платы представлен в таблице 4.1.

Таблица 4.1 – Расчет основной заработной платы команды разработчиков программного средства

Категория исполнителя	Месячный оклад, р.	Часовой оклад, р.	Трудоемкость работ, ч	Итого, р.
Бизнес аналитик	4000	23,81	150	3571,50
Backend-разработчик (ASP.NET Core)	5 500	32,74	450	14733,00
Android-разработчик (Kotlin)	5 000	29,76	450	13392,00
Fullstack-разработчик (Django/Python)	3 500	20,83	300	6249,00
Тестировщик (QA)	2 300	13,69	200	2738,00
Итого				40683,50
Премия и иные стимулирующие выплаты (50%)				20341,75
Всего затрат на основную заработную плату разработчиков				61025,25

Расчет общей суммы инвестиций (затрат) на разработку программного средства осуществляется по методике, представленной в таблице 4.2.

Таблица 4.2 – Расчет инвестиций (затрат) на разработку программного средства

Наименование статьи затрат	Формула/таблица для расчета, р.	Сумма, р.
1. Основная заработная плата разработчиков	Формула (4.1), таблица 4.1	61025,25
2. Дополнительная заработная плата разработчиков	$З_д = \frac{З_о \cdot Н_д}{100} = \frac{61025,25 \cdot 10}{100} = 6102,53, \quad (4.2)$ где $Н_д$ – норматив дополнительной заработной платы, (10%)	6102,53
3. Отчисления на социальные нужды	$Р_{соц} = \frac{(З_о + З_д) \cdot Н_{соц}}{100}$ $= \frac{(61025,25 + 6102,53) \cdot 34,6}{100} = 23230,21, \quad (4.3)$ где $Н_{соц}$ – ставка отчислений в ФСЗН и Белгосстрах (в соответствии с действующим законодательством – 34,6% [24])	23230,21
4. Прочие расходы	$Р_{пр} = \frac{З_о \cdot Н_{пр}}{100}$ $= \frac{61025,25 \cdot 30}{100} = 18307,58, \quad (4.4)$ где $Н_{пр}$ – норматив прочих расходов, (30%)	18307,58
5. Расходы на реализацию	$Р_p = \frac{З_о \cdot Н_p}{100}$ $= \frac{61025,25 \cdot 5}{100} = 3051,26, \quad (4.5)$ где $Н_p$ – норматив расходов на реализацию, (5 %)	3051,26
6. Общая сумма инвестиций (затрат) на разработку	$З_p = З_о + З_д + Р_{соц} + Р_{пр} + Р_p$ $= 61025,25 + 6102,53 + 23230,21 + 18307,58 + 3051,26 = 111716,83, \quad (4.6)$	111716,83

Таким образом полная сумма затрат на разработку программного обеспечения составляет 111716,83 рублей.

9.3 Расчет экономического эффекта от реализации программного средства на рынке

Экономический эффект организации-разработчика программного средства представляет собой прирост чистой прибыли от его продажи на рынке потребителям, величина которого зависит от объема продаж, цены реализации и затрат на разработку программного средства.

По данным Республиканского института контроля знаний, в 2025 году в централизованном тестировании приняли участие 58491 абитуриент. Централизованный экзамен сдавали 56700 выпускников 11-х классов.

С учётом того, что подготовку к ЦТ начинают уже в 10-м классе, и доля десятиклассников в общей массе готовящихся составляет 17,5% (по экспертным оценкам), общая численность целевой аудитории учащихся рассчитывается следующим образом:

- учащиеся 11-х классов: 56700 чел.
- учащиеся 10-х классов, готовящиеся к ЦТ: $56700 \times 0,175 = 9923$ чел.
- выпускники прошлых лет и учащиеся колледжей: $58491 - 56700 = 1791$ чел.

Общая численность целевой аудитории учащихся: $56700 + 9923 + 1791 \approx 68400$ человек.

Количество преподавателей, потенциально заинтересованных в использовании системы: по данным Министерства образования, в сфере образования Беларуси работают 114 000 педагогов. Учитывая, что не все преподаватели ведут выпускные классы и готовят к ЦТ, по имеющимся оценкам доля целевых преподавателей составляет 25% от общего числа, то есть 28 500 человек.

Принимая во внимание наличие бесплатного тарифа (Freemium) и конкуренцию на рынке (Adukar, TurboCT, 0101CT), прогнозируемый уровень проникновения платной подписки в первый год составит:

- среди учащихся: 1,5% от целевой аудитории
- среди преподавателей: 1,0% от целевой аудитории

Таким образом, прогнозируемое количество платящих пользователей в первый год:

- учащиеся: $68400 \times 0,015 = 1026$ чел.
- преподаватели: $28500 \times 0,01 = 285$ чел.

Общее число платящих пользователей $N = 1311$ человек.

На основе анализа цен на образовательные услуги в Беларуси (курсы подготовки при вузах стоят 350–600 руб. за сезон), а также с учётом разработанной тарифной сетки, установим следующие цены:

1. Подписка «Ученик» (доступ к полной аналитике, рекомендациям, неограниченным тестам): 25 руб./мес. (годовая подписка – 250 руб. со скидкой)

2. Подписка «Преподаватель» (создание тестов, привязка учеников, аналитика по группе): 40 руб./мес. (годовая – 400 руб.)

Средневзвешенная стоимость подписки (ARPU) с учётом структуры пользователей [21]:

$$ARPU = \frac{(1026 \times 250 + 285 \times 400)}{1311} \approx 283 \text{ р.}$$

Прирост чистой прибыли, полученной разработчиком от реализации программного средства на рынке, рассчитывается по формуле:

$$\Delta\Pi_{\text{ч}}^{\text{р}} = (\Pi_{\text{отп}} \cdot N - \text{НДС}) \cdot P_{\text{пр}} \cdot \left(1 - \frac{H_{\text{п}}}{100}\right), \quad (4.7)$$

где $\Pi_{\text{отп}}$ – отпускная цена подписки программного средства, р.;

N – количество подписки программного средства, реализуемое за год, шт.;

НДС – сумма налога на добавленную стоимость, р.;

$P_{\text{пр}}$ – рентабельность продаж (принята на уровне 30%);

$H_{\text{п}}$ – ставка налога на прибыль согласно действующему законодательству (20%).

Налог на добавленную стоимость определяется по формуле:

$$\text{НДС} = \frac{\Pi_{\text{отп}} \cdot N \cdot H_{\text{д.с}}}{100\% + H_{\text{д.с}}}, \quad (4.8)$$

где $H_{\text{д.с}}$ – ставка налога на добавленную стоимость в соответствии с действующим законодательством (20%).

Рассчитаем НДС с помощью формулы (4.8):

$$\text{НДС} = \frac{283 \cdot 1311 \cdot 20}{120} = 61750 \text{ р.}, \quad (4.9)$$

Годовая выручка без НДС: $283 \cdot 1311 - 61317 = 308750 \text{ р.}$

Прирост чистой прибыли, полученной разработчиком от реализации программного средства на рынке, рассчитаем по формуле (4.7):

$$\Delta\Pi_{\text{ч}}^{\text{р}} = 308750 \cdot 0,3 \cdot \left(1 - \frac{20}{100}\right) = 74100 \text{ р.}, \quad (4.10)$$

Таким образом, прирост чистой прибыли, полученной разработчиком от реализации программного средства в первый год, составляет 147160 рублей.

9.4 Расчет показателей экономической эффективности разработки и реализации программного средства на рынке

Экономическая эффективность инвестиций в разработку программного средства и его реализацию на рынке осуществляется на основе расчета и оценки показателей эффективности инвестиций по методике, представленной в таблице 4.3.

Исходные данные для расчета:

- инвестиции в разработку – 111717,00 рублей;
- годовой экономический эффект – 74100,00 рублей;
- норма дисконта – 9,75% (принята равной ставке рефинансирования Национального банка Республики Беларусь на 2026 год);
- расчетный период – 2 года.

Подсчет чистого дисконтированного дохода осуществим в табличной форме. Расчет эффективности инвестиций показан в таблице 4.5.

Таблица 4.3 – Расчет эффективности инвестиций (затрат) в реализацию проектного решения

Показатель	Значение по годам расчетного периода	
	07.2026 – 07.2027	07.2027 – 07.2028
Результат		
1 Прирост чистой прибыли, р.	74100,00	74100,00
2 Дисконтированный результат, р.	74100,00	67518,95
Затраты		
3 Инвестиции в реализацию проектного решения, р.	111716,83	0
4 Дисконтированные инвестиции, р.	111716,83	0
5 Чистый дисконтированный доход по годам, р.	74100,00	67518,95
6 Чистый дисконтированный доход нарастающим итогом, р.	-37616,83	29901,17
7 Коэффициент дисконтирования, доли единицы	1,0000	0,9112

Чистый дисконтированный доход за 2 года, полученный по формуле (4.11) составляет ЧДД(NPV) = 29901,17 рубля, что удовлетворяет целевому значению «> 0».

$$\text{ЧДД(NPV)} = \sum_{t=1}^n \Delta\Pi_{\text{чт}} \cdot \alpha_t - \sum_{t=1}^n Z_t \cdot \alpha_t, \quad (4.11)$$

где α_t – коэффициент дисконтирования, рассчитанный для года t

Также табличный расчет по формуле (4.12) позволяет получить значение динамического срока окупаемости инвестиций. DPP = 2 г, так как именно во 2-й год рассчитанное значение удовлетворяет целевому «≤ 3 года».

$$\text{DPP} = n, \text{ при котором } \sum_{t=1}^n \Delta\Pi_{\text{чт}} \cdot \frac{1}{(1+d)^{t-t_p}} \geq \sum_{t=1}^n Z_t \cdot \frac{1}{(1+d)^{t-t_p}}, \quad (4.12)$$

Рассчитаем индекс доходности инвестиций за весь расчётный период (2 года):

$$\text{ИД(PI)} = \frac{\sum_{t=1}^n \Delta\Pi_{\text{чт}} \cdot \alpha_t}{\sum_{t=1}^n Z_t \cdot \alpha_t} = \frac{74100,00 + 67518,00}{111716,83} = 1,27, \quad (4.13)$$

Коэффициент дисконтирования α_t , получаемый для года t , рассчитывается по формуле:

$$\alpha_t = \frac{1}{(1+d)^{t-t_p}}, \quad (4.14)$$

где d – требуемая норма дисконта, доли единицы;

t – порядковый номер года, доходы и затраты которого приводятся к расчетному году;

t_p – расчетный год, к которому приводятся доходы и инвестиционные затраты ($t_p = 1$).

Значение, полученное в формуле (4.13), соответствует целевому «> 1». По формуле (4.11) получим простой срок окупаемости инвестиций:

$$T_{\text{ок}}(\text{PP}) = \frac{\sum_{t=1}^n Z_t}{\frac{1}{n} \cdot \sum_{t=1}^n \Delta\Pi_{\text{чт}}} = \frac{111716,83}{74100,00} \approx 1,51 \text{ года } (\approx 18 \text{ месяцев}), \quad (4.15)$$

Где

– n – расчетный период, лет

- Z_t – затраты (инвестиции) в году t , р.
- $\Delta\Pi_{\text{чt}}$ – прирост чистой прибыли в году t в результате реализации проекта, р.

Значение простого срока окупаемости инвестиций, полученное в формуле (4.7), соответствует целевому « ≤ 3 года». Получим и сравним с целевым значением среднюю норму прибыли/рентабельности инвестиций:

$$P_{\text{и}}(\text{ARR}) = \frac{\frac{1}{n} \cdot \sum_{t=1}^n \Delta\Pi_{\text{чt}}}{\sum_{t=1}^n Z_t} \cdot 100\% = \frac{74100,00}{111716,83} \cdot 100\% = 66,3\%, \quad (4.16)$$

Полученное значение средней нормы прибыли/рентабельности инвестиций, полученное в формуле (4.8), значительно превышает среднюю процентную ставку в национальной валюте для юридических лиц на срок свыше 1 года на 2026 год (10,65%).

Данные, полученные за два года работы продукта, свидетельствуют о том, что инвестиции окупаются уже на втором году реализации проекта. Показатели доходности подтверждают привлекательность вложений, а средняя норма рентабельности инвестиций значительно превышает норму дисконта. Таким образом, разработка и реализация на рынке программного комплекса являются экономически целесообразными.

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломного проекта разработан программный комплекс для подготовки к централизованному тестированию и централизованному экзамену, ориентированный на специфику образовательной системы Республики Беларусь. Комплекс объединяет функции самостоятельной подготовки учащихся и инструменты контроля со стороны преподавателя в единой цифровой среде.

Проведённый анализ существующих решений (онлайн-платформ и мобильных приложений) показал, что большинство аналогов предоставляют лишь базовый доступ к тестам прошлых лет. При этом они не обеспечивают персонализированной аналитики, адаптивного формирования тестов, роли преподавателя с возможностью создания собственных заданий и отслеживания прогресса учеников, а также системы уведомлений и рекомендаций. Выявленные недостатки обосновали необходимость разработки собственного программного комплекса и позволили сформулировать complete набор функциональных требований. Система должна поддерживать три роли пользователей (ученик, преподаватель, администратор), обеспечивать хранение банка тестов, генерацию смешанных тестов, прохождение тестирования с фиксацией результатов, детальную статистику и аналитику, автоматические рекомендации по повторению тем, механизм привязки учеников к преподавателю, создание пользовательских тестов преподавателем, а также систему уведомлений. Дополнительно были определены нефункциональные требования к производительности, надёжности, удобству интерфейсов, масштабируемости и безопасности.

Архитектурно программный комплекс построен по многослойной клиент-серверной модели. Уровень представления реализован в виде мобильного приложения для операционной системы Android и веб-инструмента для преподавателей. Уровень прикладной логики представлен серверной частью с REST API. Уровень хранения данных реализован на реляционной системе управления базами данных. Такая архитектура обеспечивает модульность, масштабируемость и удобство сопровождения. Модульная структура системы включает подсистемы аутентификации, управления пользователями, тестами, прохождения тестов, статистики, уведомлений и административную панель. Разработаны диаграммы развёртывания, последовательности для основных сценариев, схемы ключевых алгоритмов, а также выполнено функциональное моделирование в нотации IDEF0. Предусмотрены механизмы расширяемости и сопровождения.

База данных спроектирована с учётом всех бизнес-сущностей системы: пользователи, роли, предметы, темы, задания, тесты, назначения, попытки прохождения, ответы, уведомления и аудит. Построены диаграммы потоков данных и ER-диаграмма. Физическая модель реализована с учётом требований целостности, производительности (индексация) и возможности мягкого удаления записей.

Программная реализация выполнена в полном соответствии с архитектурными решениями. Серверная часть построена по трехуровневой архитектуре. Мобильное приложение реализует паттерн MVVM, поддерживает работу в офлайн-режиме с локальным кэшированием и автоматической синхронизацией при восстановлении сети. Веб-инструмент преподавателя обеспечивает создание и редактирование тестов, их публикацию и назначение ученикам. Административная панель предоставляет функции управления пользователями и контентом.

Тестирование программного комплекса проведено всесторонне. Модульные тесты ключевых сервисов, функциональное тестирование веб-инструмента и мобильного приложения, а также нагрузочное тестирование REST API подтвердили корректность работы системы, её стабильность и соответствие заявленным требованиям производительности. Система успешно выдерживает расчётную нагрузку и не имеет критических дефектов.

Разработаны руководства по развёртыванию и пользованию системой. Руководство по развёртыванию содержит инструкции по подготовке окружения, установке базы данных, развёртыванию серверной части, веб-инструмента преподавателя и мобильного приложения, а также процедуры проверки работоспособности и обслуживания. Руководство пользователя описывает все функции системы для трёх ролей: ученик (прохождение тестов, статистика, рекомендации), преподаватель (создание тестов, назначение ученикам, контроль результатов) и администратор (управление пользователями и контентом).

Экономическое обоснование подтвердило целесообразность разработки и вывода программного комплекса на рынок. Рассчитанные показатели эффективности инвестиций (чистый дисконтированный доход, индекс доходности, сроки окупаемости, средняя норма рентабельности) удовлетворяют всем целевым критериям и значительно превышают альтернативные вложения. Разработка и реализация программного комплекса признаны экономически целесообразными.

Таким образом, в результате выполнения дипломного проекта полностью достигнута поставленная цель. Разработанный программный комплекс учитывает специфику белорусской системы образования, объединяет самостоятельную подготовку учащихся и контроль со стороны преподавателя, обеспечивает высокий уровень безопасности и производительности, подтвердил свою работоспособность в ходе тестирования и доказал экономическую эффективность при выводе на массовый рынок. Практическая значимость работы заключается в создании доступного цифрового инструмента, позволяющего учащимся систематизировать знания и отслеживать динамику подготовки в реальном времени, а преподавателям – эффективно организовывать учебный процесс и взаимодействовать с учениками в единой цифровой среде.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] СТП 01–2024. Дипломные проекты (работы). Общие требования. – Введ. 2024-01-01. – Минск : БГУИР, 2024. – 178 с.
- [2] PostgreSQL 15 Documentation [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/15/index.html> – Дата доступа: 05.04.2025.
- [3] .NET Documentation [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/> – Дата доступа: 05.04.2025.
- [4] OAuth 2.0 [Электронный ресурс]. – Режим доступа: <https://oauth.net/2/> – Дата доступа: 05.04.2025.
- [5] What is OpenID Connect [Электронный ресурс]. – Режим доступа: <https://openid.net/developers/how-connect-works/> – Дата доступа: 05.04.2025.
- [6] Transport Layer Security (TLS) [Электронный ресурс]. – Режим доступа: <https://www.cloudflare.com/learning/ssl/what-is-ssl/> – Дата доступа: 05.04.2025.
- [7] Современные веб-приложения на ASP.NET Core [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/architecture/modern-web-apps-azure/> – Дата доступа: 05.04.2025.
- [8] Introduction to Swagger and OpenAPI [Электронный ресурс]. – Режим доступа: <https://swagger.io/docs/specification/about/> – Дата доступа: 05.04.2025.
- [9] Npgsql Documentation [Электронный ресурс]. – Режим доступа: <https://www.npgsql.org/doc/index.html> – Дата доступа: 05.04.2025.
- [10] JSON Web Tokens Introduction [Электронный ресурс]. – Режим доступа: <https://jwt.io/introduction> – Дата доступа: 05.04.2025.
- [11] Руководство. Создание веб-API на основе контроллера с помощью ASP.NET Core [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/aspnet/core/tutorials/first-web-api?view=aspnetcore-9.0&tabs=visual-studio> – Дата доступа: 05.04.2025.
- [12] PostgreSQL Documentation: JSON Types [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/15/datatype-json.html> – Дата доступа: 05.04.2025.
- [13] PostgreSQL Documentation: pg_trgm [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/15/pgtrgm.html> – Дата доступа: 05.04.2025.
- [14] PostgreSQL Documentation: PL/pgSQL [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/15/plpgsql.html> – Дата доступа: 05.04.2025.
- [15] Django Documentation [Электронный ресурс]. – Режим доступа: <https://docs.djangoproject.com/en/5.0/> – Дата доступа: 05.04.2025.

[16] Руководство по публикации приложений .NET [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/core/deploying/> – Дата доступа: 05.04.2025.

[17] Android Developers Documentation [Электронный ресурс]. – Режим доступа: <https://developer.android.com/docs> – Дата доступа: 05.04.2025.

[18] Jetpack Compose Documentation [Электронный ресурс]. – Режим доступа: <https://developer.android.com/develop/ui/compose/documentation> – Дата доступа: 05.04.2025.

[19] Logging in .NET [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/core/extensions/logging> – Дата доступа: 05.04.2025.

[20] Selenium WebDriver Documentation [Электронный ресурс]. – Режим доступа: <https://www.selenium.dev/documentation/webdriver/> – Дата доступа: 05.04.2025.

[21] Apache JMeter Documentation [Электронный ресурс]. – Режим доступа: <https://jmeter.apache.org/usermanual/index.html> – Дата доступа: 05.04.2025.

[22] Экономика проектных решений: методические указания по экономическому обоснованию дипломных проектов: учеб.-метод. пособие / В. Г. Горовой [и др.] – Минск.: БГУИР, 2021. – 107 с.

[23] Wadline [Электронный ресурс]. – Режим доступа : <https://wadline.ru/salary/minsk> – Дата доступа: 05.04.2025.

[24] Закон Республики Беларусь от 15 июля 2021 г. № 118-З «О взносах в бюджет государственного внебюджетного фонда социальной защиты населения Республики Беларусь»

ПРИЛОЖЕНИЕ А

СПРАВКА О ПРОВЕРКЕ НА ВЗАИМСТВОВАНИЯ



Отчет предоставлен сервисом
«Антиплагиат» - <http://my.antiplagiat.ru>



Отчет о проверке

Автор: dmitrukb91@gmail.com / ID: 13128995

Проверяющий: dmitrukb91@gmail.com / ID: 13128995

Название документа: Для реализации серверной части программного
компле

РЕЗУЛЬТАТЫ ПРОВЕРКИ

Тариф: DEMO



Совпадения:
Не менее 0%



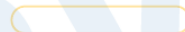
Оригинальность:
Не более 100%



Цитирования:
Недоступно для DEMO*



Самоцитирования:
Недоступно для DEMO*



❗ «Совпадения», «Цитирования», «Самоцитирования», «Оригинальность» являются отдельными показателями, отображаются в процентах и в сумме дают 100%, что соответствует проверенному тексту документа.

❗ *Результаты проверки на тарифе DEMO являются неполными и ограниченными по сравнению с платным тарифом и корпоративной версией, так как проверка идет по источникам, добавленным до 15 ноября 2021 года, с использованием урезанных возможностей системы

- **Совпадения** — фрагменты проверяемого текста, полностью или частично сходные с найденными источниками, за исключением фрагментов, которые система отнесла к цитированию или самоцитированию. Показатель «Совпадения» — это доля фрагментов проверяемого текста, отнесенных к совпадениям, в общем объеме текста.
- **Самоцитирование** — фрагменты проверяемого текста, совпадающие или почти совпадающие с фрагментом текста источника, автором или соавтором которого является автор проверяемого документа. Показатель «Самоцитирования» — это доля фрагментов текста, отнесенных к самоцитированию, в общем объеме текста.
- **Цитирование** — фрагменты проверяемого текста, которые не являются авторскими, но которые система отнесла к корректно оформленным. К цитированиям относятся также шаблонные фразы; библиография; фрагменты текста, найденные модулем поиска «СПС Гарант: нормативно-правовая документация». Показатель «Цитирования» — это доля фрагментов проверяемого текста, отнесенных к цитированию, в общем объеме текста.
- **Текстовое пересечение** — фрагмент текста проверяемого документа, совпадающий или почти совпадающий с фрагментом текста источника.
- **Источник** — документ, проиндексированный в системе и содержащийся в модуле поиска, по которому проводится проверка.
- **Оригинальный текст** — фрагменты проверяемого текста, не обнаруженные ни в одном источнике и не отмеченные ни одним из модулей поиска. Показатель «Оригинальность» — это доля фрагментов проверяемого текста, отнесенных к оригинальному тексту, в общем объеме текста.

Обращаем Ваше внимание, что система находит текстовые совпадения проверяемого документа с проиндексированными в системе источниками. При этом система является вспомогательным инструментом, определение корректности и правомерности совпадений или цитирований, а также авторства текстовых фрагментов проверяемого документа остается в компетенции проверяющего.

Рисунок 1 – Результат проверки на заимствование

ПРИЛОЖЕНИЕ Б

ЛИСТИНГ ПРОГРАММНОГО КОДА

```
public async Task<OperationResult>
AssignTestToStudent(AssignTestToStudentRequestModel
requestModel)
{
    var testExists = await _dbContext.Tests.AnyAsync(t
=> t.Id == requestModel.TestId && !t.IsDeleted);
    if (!testExists)
    {
        return new OperationResult { ErrorCode =
ErrorCodeConstants.TestNotFound, ErrorMessage = "Test not
found", HttpStatusCode = HttpStatusCode.NotFound };
    }

    var binding = await _dbContext.TeacherStudents
.AnyAsync(ts => ts.TeacherId ==
requestModel.TeacherId && ts.StudentId == requestModel.StudentId
&& !ts.IsDeleted);
    if (!binding)
    {
        return new OperationResult { ErrorCode =
ErrorCodeConstants.BindingNotFound, ErrorMessage = "No binding
with this student", HttpStatusCode = HttpStatusCode.NotFound };
    }

    var assignment = new StudentAssignment
    {
        StudentId = requestModel.StudentId,
        TeacherId = requestModel.TeacherId,
        TestId = requestModel.TestId,
        ExpiredAt = requestModel.Deadline ??
DateTimeOffset.MaxValue,
        AttemptsLeft = requestModel.AttemptsAllowed,
        CreatedAt = DateTimeOffset.UtcNow,
        LastUpdateAt = DateTimeOffset.UtcNow
    };

    await
_dbContext.StudentAssignments.AddAsync(assignment);
    await _dbContext.SaveChangesAsync();
    return new OperationResult();
}

public async Task<OperationResult<Test>>
CreateMixedTest(CreateMixedTestRequestModel requestModel)
{
    var allProblems = new List<Problem>();
    foreach (var topic in requestModel.TopicItems)
    {
        var topicProblems = await _dbContext.Problems
```

```

        .Include(p => p.Versions.Where(v =>
v.IsActive))
        .Where(p => p.TopicId == topic.TopicId
            && !p.IsDeleted
            && p.IsPublished
            && p.Versions.Any(v =>
v.IsActive
            &&
(int)v.Difficulty == (int)requestModel.AverageDifficulty))
        .OrderBy(r => Guid.NewGuid())
        .Take((int)topic.ProblemCount)
        .ToListAsync();
    allProblems.AddRange(topicProblems);
}

if (!allProblems.Any())
{
    return new OperationResult<Test>
    {
        ErrorCode = ErrorCodeConstants.TestNotFound,
        ErrorMessage = "No problems found for
selected parameters",
        HttpStatusCode = HttpStatusCode.NotFound
    };
}

var singleChoiceProblems = allProblems
    .Where(p => p.Versions.FirstOrDefault(pv =>
pv.IsActive)!.Type == ProblemTypeEnum.SingleChoice)
    .ToList();

var multipleChoiceProblems = allProblems
    .Where(p => p.Versions.FirstOrDefault(pv =>
pv.IsActive)!.Type == ProblemTypeEnum.MultipleChoice)
    .ToList();

var openEndedProblems = allProblems
    .Where(p => p.Versions.FirstOrDefault(pv =>
pv.IsActive)!.Type == ProblemTypeEnum.OpenEnded)
    .ToList();

var aTypeProblems = singleChoiceProblems
    .Concat(multipleChoiceProblems)
    .OrderBy(x => Guid.NewGuid())
    .ToList();

var bTypeProblems = openEndedProblems
    .OrderBy(x => Guid.NewGuid())
    .ToList();

var testProblems = new List<TestProblem>();

int aCounter = 1;

```

```

foreach (var problem in aTypeProblems)
{
    testProblems.Add(new TestProblem
    {
        ProblemId = problem.Id,
        Code = $"A{aCounter}"
    });
    aCounter++;
}

int bCounter = 1;
foreach (var problem in bTypeProblems)
{
    testProblems.Add(new TestProblem
    {
        ProblemId = problem.Id,
        Code = $"B{bCounter}"
    });
    bCounter++;
}

testProblems = testProblems
    .OrderBy(x => Guid.NewGuid())
    .ToList();

var subject = await _dbContext.Subjects
    .Where(s => s.Id == requestModel.SubjectId)
    .Select(s => s.Name)
    .FirstOrDefaultAsync();

var difficultyText = requestModel.AverageDifficult
switch
{
    ProblemDifficultEnum.VeryEasy => "очень легкий",
    ProblemDifficultEnum.Easy => "легкий",
    ProblemDifficultEnum.Normal => "средний",
    ProblemDifficultEnum.Hard => "сложный",
    ProblemDifficultEnum.VeryHard => "очень
сложный",
    _ => "смешанный"
};

var testTitle = $"Смешанный тест по {subject ??
"предмету"} ({difficultyText}) от
{DateTimeOffset.UtcNow:dd.MM.yyyy HH:mm}";

var test = new Test
{
    Title = testTitle,
    SubjectId = requestModel.SubjectId,
    AuthorId = requestModel.AuthorId,
    Type = TestTypeEnum.Mixed,
    IsTraning = true,

```

```
        IsPublished = true,  
        IsPublic = false,  
        Duration = 3600,  
        AttemptsCount = 1,  
        CreatedAt = DateTimeOffset.UtcNow,  
        LastUpdatedAt = DateTimeOffset.UtcNow,  
        TestProblems = testProblems  
    };  
  
    await _dbContext.Tests.AddAsync(test);  
    await _dbContext.SaveChangesAsync();  
  
    return new OperationResult<Test>(test);  
}
```